

Learn Paper with me.

All 52 videos from Paper's YouTube channel, from the first MCP setup in March to the September releases, rebuilt as a course and a reference. The personal parts are addressed to me and come from 14,018 of my own Paper calls.

Start reading

[Read the web edition](#)

[Open the video library](#)

write_html3,092

get_screenshot2,805

lookups2,668

get_selection1

Point at it instead of describing it.

Mar

Apr

May

Jun

Jul

Aug

Sep

Know how old each video's Paper is.

--ink

--slate

--line

--sheet

--cobalt

Put tokens in before the boards.

01	Start here	08	Shaders and motion
02	Your Paper, measured	09	Agents in Paper
03	Ten changes for you	10	Design to code, and back
04	Four-week plan	11	Snapshot, collaboration and releases
05	Canvas, layout and files	12	Reference
06	Tokens, color and type	13	Glossary and timeline
07	Vectors, images and AI generation	14	Video library

Start here

This guide covers everything Paper's team has shown on YouTube, from the first MCP setup videos in March to the September releases. It is organized so you can learn Paper in order, then come back to look things up. It is also fitted to your work: the examples use your files, the prompts are written for Claude Code with the Paper MCP, and the recommendations come from what your own sessions show.

How the guide is organized

For you. What your sessions say about how you use Paper, the ten changes that would help most, and a four-week plan that tells you which video to watch when. The plan's checkboxes are remembered in this browser.

Learn. Seven chapters, from the canvas to agents and code. Each one opens with its videos, teaches the topic with exact UI names, then ends with exercises on your own files and, where agents are involved, prompts you can paste into Claude Code.

Reference. Keyboard shortcuts, the full Paper MCP tool list, the image models, a glossary, a release timeline, and a searchable library of all 52 videos with their key moments.

A link like ► 3:42 opens the video at that second. Use it whenever a step depends on where something sits on screen: the guide describes the interface, the video shows it.

How it was made

1. **Extraction.** Metadata, captions (27 written by Paper's team, 25 generated by YouTube), viewer comments, and 1080p video for all 52 videos. From the video: 1,952 frames, one every 2 to 8 seconds, plus 185 contact sheets.
2. **Seven analysts.** Seven Opus agents each took one theme (6 to 11 related videos), read every transcript, skimmed every contact sheet, opened the full frames where the screen mattered, and wrote one notes file per video and one chapter per theme.
3. **Reference.** An eighth agent read Paper's documentation, its MCP guides, the Paper plugin's two agent skills, and the Paper Shaders library.
4. **Audit.** A ninth agent checked coverage (every video and feature lands in a chapter), resolved contradictions against the frames, and fact-checked claims.
5. **Your side.** Your project memories, your 14 Paper files, and the 14,018 Paper MCP calls recorded in your Claude sessions between 3 July and 2 October 2026.

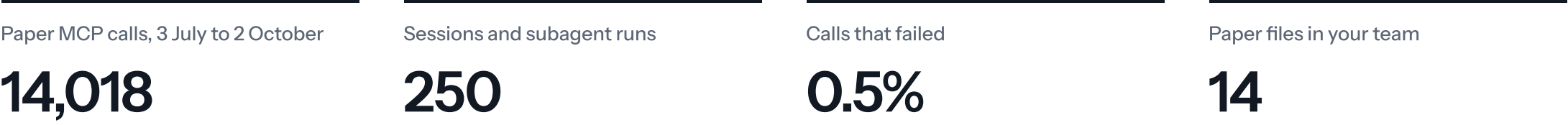
Nothing here is a transcript. The chapters are written from the videos, and they cite them, so the exact moment is always one click away.

Three ways to use it

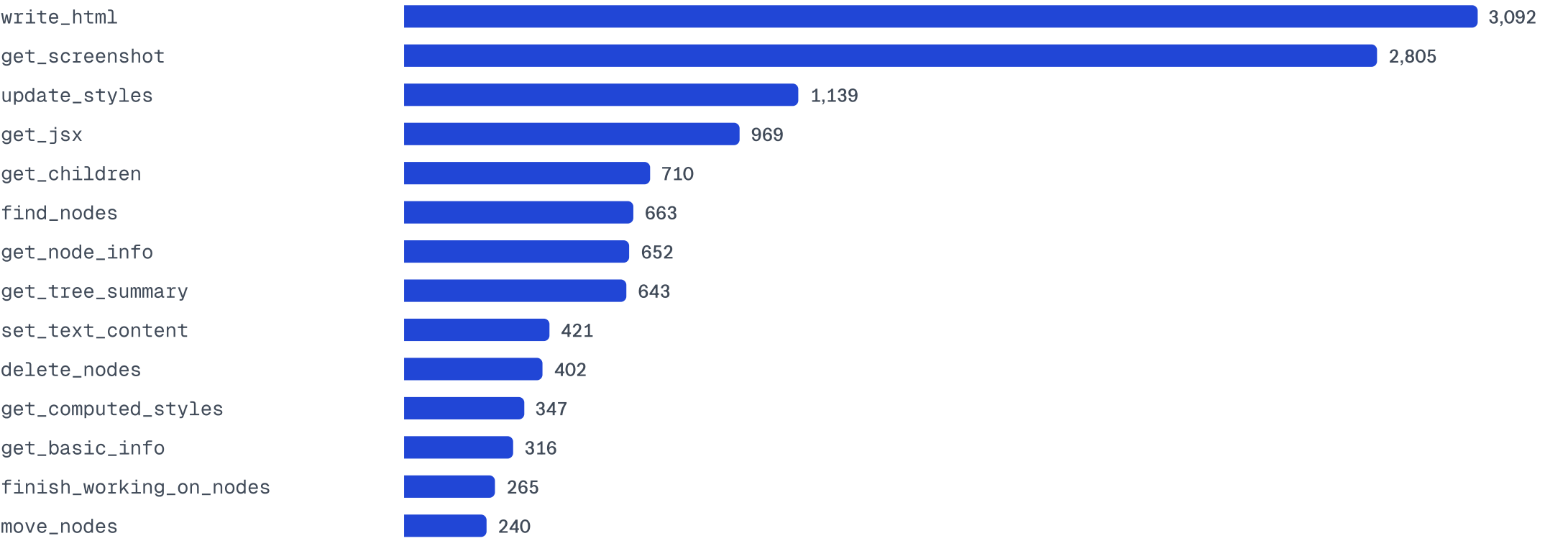
- **One hour.** Read *Ten changes for you*, then watch the first three videos in the plan.
- **The full course.** Follow the four-week plan: under an hour of video a week, plus practice on your own files.
- **An answer now.** Search the video library, or look up the term in the glossary, which points to the chapter that teaches it.

Your Paper, measured

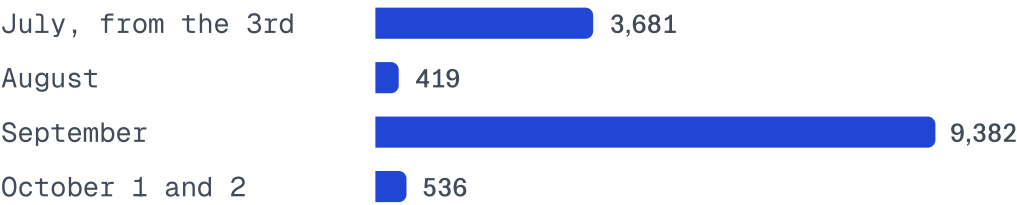
These figures come from your Claude Code session files: every Paper MCP call recorded between 3 July and 2 October 2026, including the calls made by your subagents and workflows. Your Paper files go back to 21 May 2026, but the oldest session that still holds a Paper call is from 3 July, so earlier work is not counted.



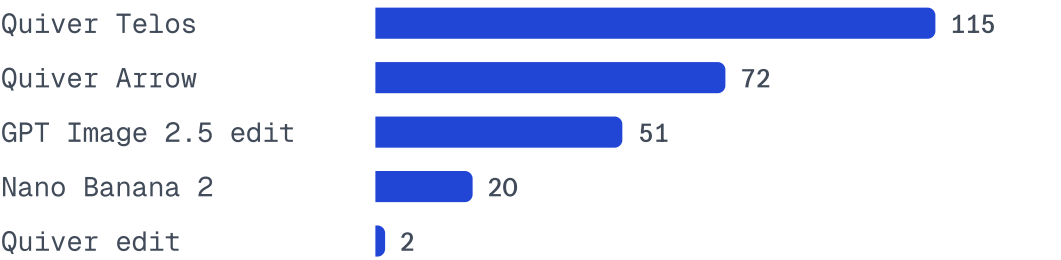
Calls by tool, the top 14



Calls by month



Generations started inside Paper, by model



Where the calls went

Month	Calls	Mostly
July (from the 3rd)	3,681	Water (2,446 calls over 99 transcripts, the month of the multi-agent design panels), then RevOpt and other projects (1,235)
August	419	Ballottage, first pass
September	9,382	Ballottage (5,842), Water and the budget tool (1,821), Poseidon (1,719)
October 1 to 2	536	Water and Altitude (158), a print layout and smaller sessions

What the numbers say

You design through agents, in a tight loop. `write_html` (3,092 calls) and `get_screenshot` (2,805) run almost one for one: write a group, look at it, write the next. That is the rhythm Paper's own MCP guide asks agents to follow. Most writes are small (median 1,399 characters), but one in ten goes past 6,000 characters and the largest reached 36,437. Your notes tie the "file disconnected from the server" failures to bursts of large writes.

Your agents search instead of being shown. `find_nodes`, `get_tree_summary`, `get_children` and `get_node_info` add up to 2,668 calls, about one call in five. `get_selection` was called once. Each lookup is an agent working out which element you meant. Selecting the element in Paper and telling the agent to work on the selection skips that search. See [Agents in Paper](#).

Comments are unused as a channel to agents. Seven calls listed comment threads; none read a thread or resolved one. You give precise visual feedback in chat (alignment, spacing, words that read oddly). A comment pinned on the element carries the same feedback with its location attached, and the agent can resolve it when done.

Tokens are thin compared with your rules. 22 `create_tokens` and 16 `set_tokens` calls across 14 files, while your designs repeat the same values everywhere: cobalt, the glass recipe (white at 55%, top sheen, inset highlight), card radii of 12, 20 and 24, the Inter Tight 900 title format. See [Tokens, color and type](#).

Exports are routine; PDF is not. 202 export calls and 3 combined PDFs. Paper's export writes into Downloads, which you don't want, and the native PDF export failed on the report pages of the budget tool, so those went out as PNG.

Errors are rare, and half of them were not Paper's. 64 calls failed out of 14,018 (0.5%). 31 were Claude's auto-mode safety check being unavailable for a moment. Of the Paper-side failures, 16 were "MCP server not connected" because Paper was closed when the session started, 6 were timeouts, 4 were invalid export or screenshot arguments, 3 were "Open a Paper file to use this tool", and 1 was the file disconnecting from Paper's server.

Generation leans on Quiver. 260 generations started inside Paper: 115 Quiver Arrow Telos (the 3D icons and supports), 72 Quiver Arrow, 2 Quiver Arrow edits, 51 GPT Image 2.5 edits (the Ballottage figurines) and 20 Nano Banana 2.

Your files sit in one folder. All 14 files are in "Projects", including two "Sauvegarde" copies made on 6 September as manual backups before refactors.

Ten changes for you

Ordered by how much time they would save you, starting with habits you repeat in every session. Each one says what your sessions show, what to change, and where this guide and the videos teach it.

1. Point at the element instead of describing it

What your sessions show. In 14,018 Paper calls, your agents read your selection once. They spent 2,668 calls (`find_nodes` , `get_tree_summary` , `get_children` , `get_node_info`) working out which element a description meant, such as "the second card on the Tarifs board".

Change. Select the element in Paper and write "my selection" in the prompt; the agent reads it with `get_selection` . For a file that is not in front, copy its link with ⌘L and paste it into the prompt, and the agent opens that file before doing anything else.

Learn it

Agents in Paper

▶ Teams with agents 0:40

▶ Tabs for agents 1:04

2. Write feedback as comments and let the agent clear them

What your sessions show. Your feedback arrives in chat, mostly in French and very precise: a misaligned lane, a word that reads oddly, a pill that has to go. Comment tools were called 7 times, and no thread was ever read or resolved.

Change. Press C and pin each note on the exact layer, then send one line asking the agent to work through the comments on that board and resolve each one. It lists the threads, fixes them as small edits and marks them resolved. The MCP has no tool to write a comment, so the split is fixed: you write, the agent resolves. Ask for a closing table of comment, change and status.

Learn it

Agents in Paper, recipe 3

▶ Comments for agents 0:08

▶ Comment and resolve

3. Keep one Paper connection, over stdio

What your sessions show. 16 of your failed calls were "MCP server not connected" because Paper was closed when the session started, and your Water notes keep a hand-written HTTP client as the way back in. Since 24 September two Paper connections load in every session (the plugin over HTTP and the desktop extension over stdio): 71 tool definitions for one app.

Change. Move to the stdio connection through the Paper CLI (`~/ .paper/bin/paper mcp`), which Paper's docs now recommend because the agent sees Paper's tools even while the app is closed. Either update the paper-desktop plugin to v0.2.1 or use the Connect your agent button in Paper's right panel, then remove the duplicate. Updating drops the plugin's two skills, so copy `code-to-design` and `design-to-code` into `~/ .claude/skills/` first if you use them.

Learn it

Your setup

Connecting an agent

▶ New in Paper, Sep 0:54

4. Put tokens in before the boards

What your sessions show. 22 `create_tokens` and 16 `set_tokens` calls across 14 files, while your rules repeat the same values everywhere: cobalt, white surfaces on a neutral ground, the glass recipe (white at 55%, top sheen, inset highlight), radii of 12, 20 and 24, Inter Tight 900 titles. Values an agent writes as literals do not follow later changes; token-bound values do.

Change. For each product with code (Altitude, Water, Poseidon), have the agent create tokens from the CSS variables before any board work, then rebind the existing boards section by section without changing how they look. Add opacity tokens, new in September, for the glass recipe. Copy a theme only between files of the same product (Paper logo menu, Theme, Copy theme then Paste theme), and copy again after changes, since a pasted theme does not stay linked.

Learn it

[Tokens, color and type](#), recipes 1 and 3

► [CSS variables to tokens 0:30](#)

► [MCP tokens 7:52](#)

► [New in Paper, Sep 1:52](#)

5. Start redesigns from what is live

What your sessions show. Your rule is "improve, don't regress", and your sharpest rejections came when a redesign started from a generator instead of from the existing design. Altitude, Poseidon and the Water dashboard are deployed, so the newest version of each lives in production, not in its Paper file.

Change. Capture the live page with Paper Snapshot, a Chrome extension (⇧⌘P, ⌘Enter for the whole page, then ⌘V in Paper), and iterate from the capture with the agent. For individual components, ask the agent to bring them from code onto the canvas. Replaced boards go to the archive page as usual.

Learn it

[Snapshot, collaboration and releases](#)

► [Snapshot](#)

► [Iterate on Snapshot](#)

► [Components from code](#)

6. Let flex and constraints hold your grids

What your sessions show. Your boards follow strict grids (4 × 352 with 16 and 24 gutters, or 12 × 64 with 32) and you check them at several widths. One recurring `write_html` problem, replace mode dropping absolutely positioned children, only bites freeform frames.

Change. Build dashboards with flex all the way down: rows and columns with gap and padding, Fixed widths for grid columns, Fit for heights. Use Absolute position only for overlays (glass bars, badges, figurines on a floor), each pinned to the side it belongs to. Brief agents with that structure rather than coordinates.

Learn it

[Canvas, layout and files](#)

► [Flex layout 10:43](#)

► [Constraints 1:05](#)

7. One folder per product,

Learn it

[Canvas, layout and files](#)

no hand-made backup files

What your sessions show. All 14 files sit in one folder, including two Sauvegarde copies made on 6 September before refactors, and your notes record two big tidy-ups of files that had become a mess.

Change. Create one folder per product (Water and the budget tool, Ballottage, Altitude, Poseidon, the social posts, the chart system) plus Backups, and nest folders where it helps. Keep your numbered pages inside each file. When you want a checkpoint, duplicate the file from the dashboard and move the copy into Backups. Open the files you are working on as tabs.

► Folders 0:24

► New in Paper, Aug 4:56

8. Get PDFs out of Paper again

What your sessions show. 202 exports and 3 combined PDFs. The native PDF export failed on the budget tool's report boards, so they went out as PNG, and every export lands in Downloads, where you don't want files.

Change. Paper renders PDFs from its HTML, so text stays searchable. Lay the report boards out in reading order (a combined PDF follows canvas position, top to bottom then left to right), select them all and choose Export combined PDF. If it fails, export the boards one at a time to find the one that breaks, then simplify that board's effects. Have the agent move each export out of Downloads right away.

Learn it

Canvas, layout and files, recipe R6

► Searchable PDFs 1:04

9. Ship Paper Shaders in your code

What your sessions show. You ask for dashboards that feel like the top of the technology, deck-grade rather than flat, and you already accepted three.js for the Water scene. Paper Shaders are the same WebGL effects in Paper and in code.

Change. Design the shader in Paper (S opens the shader menu), then run the same one in production: `@paper-design/shaders-react` in your React apps, `@paper-design/shaders` with `ShaderMount` in vanilla dashboards such as Altitude. The packages are Apache-2.0 with no dependencies. Pin the exact version, because 0.0.x releases rename parameters. A static shader (speed 0) costs nothing after its first frame, and rendering pauses off-screen.

Learn it

Shaders and motion

Paper Shaders in code

► Paper Shaders

► Cloudflare shader 0:32

10. Spend generation units where they show

What your sessions show. 260 generations: 115 Quiver Arrow Telos, 72 Quiver Arrow, 51 GPT Image 2.5 edit, 20 Nano Banana 2 and 2 Quiver edits. At the prices in Paper's MCP guide that is about 804 usage units, 460 of them on Telos, the slowest and most expensive SVG model (4 units an image, against 3 for Arrow).

Change. Draft objects with Arrow and keep Telos for finals. Branch each variation from the original, as a new node that uses it as a reference, instead of regenerating from scratch. Cut out subjects with right-click, Remove

Learn it

Vectors, images and AI generation

Image models

► Quiver SVG 2:26

► Remove background 0:06

background, rather than asking a model for transparency. Clean Quiver output by hand (one stroke width, stray paths deleted) before deciding to regenerate.

Also worth trying

- **Review from your phone.** A terminal Claude Code session with `/rc` lets you steer Paper from the Claude mobile app and review the screenshots there; keep the Mac awake. In July this needed a terminal session; check whether your Claude desktop app now offers it directly. ► [Remote Control 0:11](#)
- **Breadth-first variants.** For design panels, have the agent create every artboard first, then build each layer across all of them, so every direction reaches the same finish. ► [Opus 5.5 in 5 min 0:28](#)
- **Exact weights.** Inter Tight is variable, so title weights can sit between the named steps. ► [Variable weights 0:28](#)
- **Proven patterns for flows.** Pull Mobbin references onto the canvas before asking for a new Ballottage flow. ► [Mobbin flows](#)

Four-week plan

All 52 videos, in the order that pays off fastest for the way you work. Week 1 fixes the agent loop you use every day; week 2 gives you the canvas skills to judge and correct what agents build; week 3 ties Paper to your code; week 4 is visual craft. Each week is under an hour of video plus practice on your own files. Ticks are saved in this browser.

Week 1· Your agent loop (37 min of video)

Watch

- ☐ [Paper: the connected canvas for teams shipping with agents](#) · 0:35
- ☐ [Paper is the design tool for teams building with agents](#) · 1:12
- ☐ [Agent set up: connect Claude Code to Paper](#) · 4:50
- ☐ [New in Paper: faster performance, agent reliability, opacity tokens](#) · 5:40
- ☐ [Leave comments for your agent in Paper](#) · 0:27
- ☐ [Comment, reply, and resolve in Paper](#) · 0:54
- ☐ [Tabs, now in Paper](#) · 0:33
- ☐ [Using tabs to run agents across multiple Paper files at once](#) · 2:57
- ☐ [Use an AI agent to explore different design directions](#) · 6:15
- ☐ [Claude's Fable 5 vs Opus 4.8: which model designs better in Paper?](#) · 4:22
- ☐ [What can Opus 5.5 design in five minutes?](#) · 0:55
- ☐ [Use Claude's Remote Control to design in Paper from your phone](#) · 3:00
- ☐ [Design with real content: pull Notion data into Paper with Claude Code](#) · 3:29
- ☐ [Agent set up: connect Cursor to Paper](#) · 1:39 (optional, you don't use Cursor)

Do

- ☐ Fold your two Paper connections into one stdio connection (see [Your setup](#) and change 3 in [Ten changes](#)).
- ☐ Start your next Paper session with the pre-flight prompt (recipe 1 in [Agents in Paper](#)).
- ☐ Leave your next round of Ballottage feedback as comments on the screens, then run "Work through my comments" (recipe 3).
- ☐ Select one Water board and ask for variations of the selection instead of naming it.
- ☐ Open the budget tool and Poseidon as tabs and pin one agent to each with its ⌘L link (recipe 9).

You're done when an agent has resolved a comment thread you wrote, and a session started on the right file without a lookup.

Week 2· Canvas, layout and files (35 min)

Watch

☐ [8 Paper tips and tricks every designer should know](#) · 3:46

☐ [We updated hover selection in Paper](#) · 1:45

☐ [Flex layout in Paper](#) · 12:29

☐ [Constraints in Paper](#) · 3:31

☐ [Organize your Paper files with folders and nested folders](#) · 1:54

☐ [Create high quality, searchable PDFs in Paper](#) · 2:49

☐ [New in Paper: tabs, comments, free editors, and more](#) · 9:23

Do

☐ Give each project its own folder and move the two Sauvegarde files into a Backups folder.

☐ Rebuild the Poseidon nav rail by hand: vertical flex, Space between, brand and nav wrapped with $\curvearrowright$ A.

☐ On one Ballottage screen, put the glass bar and badges on Absolute position with pinned constraints, then resize the artboard.

☐ Export the budget tool's report boards as one combined PDF; if it fails, export boards one at a time to find the one that breaks.

☐ Set Nudge settings to 8 and turn on Layout guides ($\curvearrowright$ G).

You're done when a board survives a width change without your help, and the report exists as one searchable PDF.

Week 3 · Tokens and code (61 min)

Watch

☐ [New in Paper: tokens, vector editing, Figma import, and more](#) · 8:18

☐ [From CSS variables in your codebase to Paper design tokens with MCP](#) · 4:29

☐ [Create design tokens from your code base](#) · 0:52

☐ [Explore Paper's MCP-generated tokens with the Paper team](#) · 10:16

☐ [Copy and paste tokens between Paper files](#) · 0:25

☐ [If your font supports variable weights, your design tool should too](#) · 1:27

☐ [Create magic swatches in Paper](#) · 0:34

☐ [Sync designs with your codebase using Paper MCP, Claude Code and Cursor](#) · 7:19

☐ [Bring your components from code onto the Paper canvas](#) · 6:09

☐ [Push designs in Paper back to code](#) · 4:16

☐ [From design to code in Paper](#) · 0:14

☐ [Paper and Conductor: design to code with parallel AI agents](#) · 13:53

☐ [How the Skydive team ships with their agents in Paper](#) · 2:09

☐ [Design first, prompt the animations after](#) · 0:50

Do

☐ Seed the Altitude Sports file with tokens from its live code (recipe 1 in [Tokens, color and type](#)).

- ☐ Rebind the Water dashboard KPI row to tokens without changing its look (recipe 3 there).
- ☐ Turn the Ballottage glass recipe into tokens: white at 55%, sheen and highlight values as opacity tokens.
- ☐ Bring two Altitude components from code onto the canvas, change one in Paper, and push the change back (see [Design to code, and back](#)).

You're done when one board is fully bound to tokens and one change has gone from Paper to code and back.

Week 4 · Vectors, images, shaders and Snapshot (58 min)

Watch

- ☐ [SVG editing just launched in Paper](#) · 0:48
- ☐ [SVG editing in Paper: pen tool, path editing, and what's coming next](#) · 9:17
- ☐ [Generate and refine SVG illustrations with Quiver AI inside Paper](#) · 4:36
- ☐ [Generate images in Paper using the xAI model](#) · 3:37
- ☐ [Remove image backgrounds in Paper](#) · 0:26
- ☐ [Copy designs from Figma and paste them into Paper](#) · 4:01
- ☐ [How to use Paper Shaders to elevate your designs](#) · 4:48
- ☐ [How to use logo animation shaders in Paper](#) · 9:05
- ☐ [Understanding contour in Paper's logo animation shaders](#) · 0:56
- ☐ [Apply a halftone shader to text](#) · 0:55
- ☐ [Paper shader of the month: Lens Distortion](#) · 0:35
- ☐ [Recreate Cloudflare's loading shader from scratch with Paper Shaders](#) · 2:11
- ☐ [Design using your live site as a starting point with Paper Snapshot](#) · 2:30
- ☐ [Use an AI agent to iterate on a design captured with Paper Snapshot](#) · 3:30
- ☐ [Remix Resend's action button with Paper Snapshot and Paper Shaders](#) · 4:30
- ☐ [Use Mobbin and Paper to design flows from proven patterns](#) · 0:42
- ☐ [Using Paper and Resend to turn a design into a ready-to-send email template](#) · 9:15

Do

- ☐ Snapshot the live Altitude dashboard into its Paper file and improve one section from there instead of from the old boards.
- ☐ Put one shader on a Poseidon or Altitude hero, then ship the same shader in code with `@paper-design/shaders-react`.
- ☐ Clean one Quiver Telos object: one stroke width, stray paths deleted, container sized to your grid.
- ☐ Remove the background of one figurine image in Paper instead of regenerating it.

You're done when a live page has gone through Snapshot and back, and a shader runs in your code.

Canvas, layout and files

This chapter covers how Paper places things (flex layout and constraints), how you move around and select on the canvas, how images and colors are handled, how files are organized, and how boards become PDFs. It draws on six videos published between 2 June and 20 July 2026, so every feature is described as of those dates; later releases may have changed details. Facts taken from the Paper MCP tool descriptions rather than from a video are labelled as such.

1. The mental model

Frames are divs. A frame is a parent container; Paper says outright that a frame is what HTML calls a div. Layers inside a frame are its children, and children of the same frame are siblings ► [Flex 0:29](#). Moving a frame moves all of its children ► [Flex 0:51](#).

Two kinds of frame. A freeform frame lets children sit at any coordinates. A flex frame places its children from the parent's settings and the siblings around them, so they snap together and adjust when you add or remove layers ► [Flex 1:06](#). The layers panel tells them apart: a two-column icon for a horizontal flex frame, a stacked-rows icon for a vertical one ► [Flex 7:44](#), and dashed corners for a freeform frame ► [8 tips 0:51](#).

Paper speaks CSS. Paper calls the feature flex, not auto layout, because it is built on HTML and CSS and both agents and developers use the CSS terms ► [Flex 0:13](#). For you this means the HTML your agents write with `write_html` and the Flex section in the panel describe the same thing. The table below is my mapping, not something the videos spell out; check it with `get_computed_styles` on a node you have set in the panel.

Panel setting	CSS your agents write (my mapping)
Flex with → / ↓	display: flex; flex-direction: row / column
Gap between elements	gap
Horizontal / vertical padding	padding-left and right / padding-top and bottom
Nine-point grid	justify-content and align-items (axes swap with direction)
Space between	justify-content: space-between
Align to baseline	align-items: baseline
W/H Fixed	a px value
W/H Fit	fit-content (the MCP's own instructions ask agents for height "fit-content" when content clips)
W/H Fill	flex: 1 on the main axis, align-self: stretch on the cross axis
Clip content	overflow: hidden
Absolute position + constraints	position: absolute with the pinned offsets (top, right, bottom, left)

Videos in this chapter

6 videos, 26 minutes

How to create high quality, searchable PDFs in Paper
2 Jun 2026, 2:49

We updated hover selection in Paper based on your feedback
10 Jun 2026, 1:45

8 Paper tips and tricks every designer should know
15 Jun 2026, 3:46

How to organize your Paper files with folders and nested folders
17 Jun 2026, 1:54

Flex layout in Paper
20 Jul 2026, 12:29

Constraints in Paper
20 Jul 2026, 3:31

Two positioning systems. Flex children follow the flex rules. Everything else, meaning children of freeform frames and children with Absolute position ticked inside a flex frame, follows constraints, which decide what happens when the parent is resized ► [Constraints 0:02](#).

Sizing modes. The W dropdown lists Fixed, Fit, Fill and Relative ► [Flex 9:37](#); H is seen set to fixed values, Fit and Fill across the demos. Fixed holds a value whatever happens around it, and Fit sizes a frame from its contents ► [Flex 8:37](#); Fill takes the unused space in the parent ► [Flex 9:09](#). Relative is never explained; the closest thing shown is a Stretch constraint turning the width into a percentage (36% on screen) ► [Constraints 2:44](#). The size badge under a selection names the mode per axis, for example "Fit 116 × Fit 32" or "Fill 267 × Fit 317"; a bare number means Fixed ► [Flex 1:44](#).

2. The Layout and Flex panel

Control	Where	What it does
Wrap in flex ⌘A	Layout section, loose layers selected	Puts the selection in a new flex frame ► Flex 1:22
Add flex ⌘A	Layout section, freeform frame selected	Turns that frame into a flex frame ► 8 tips 0:51
→ / ↓	Flex section	Row or column; child order is kept ► Flex 3:57
Gap field	Flex section, tooltip "Gap between elements"	Type, arrow keys, drag the field icon, or drag the pink band on canvas ► Flex 2:00
Padding fields	Flex section, tooltip "Horizontal padding"	Per axis; the square icon at the row's end splits into four sides ► Flex 3:17
Nine-point grid	Flex section	Alignment; double-click it for Space between ► Flex 5:45
Sliders icon	Flex section	Menu with "Align to baseline" and "Space between" ► Flex 5:45
–	Flex section header	Removes flex
Clip content ⌘C	Under the Flex or Layout controls	Checkbox (not discussed in the videos)
Absolute position	Layout section, child of a flex frame	Takes the child out of the flow ► Flex 11:28
Constraint dropdowns and pin diagram	Layout section, non-flex items	Horizontal: Left, Right, Left + Right, Center, Stretch ► Constraints 0:38
Align buttons	Layout header, non-flex items	Left, horizontal center, right, top, vertical center, bottom, plus a "..." menu; centering sets Center constraints ► Constraints 1:42

Gestures worth memorizing: double-click a frame's top or bottom edge to set Fit height ► [Flex 5:06](#); Option + double-click an edge to set Fill ► [Flex 9:37](#); hold Option to read distances to the parent's edges ► [Flex 12:00](#); in numeric fields the arrow keys step by 1, and with Shift by the nudge value (8 in the narration) ► [Flex 2:00](#). With a child of a flex frame selected, arrow keys move it earlier or later in the order ► [Flex 2:55](#). Option-drag duplicates a layer ► [Flex 0:39](#).

For orientation, the right panel of a selected frame stacks: Layout, Flex, Radius, Blending, Fill, Outline, Border, Shadow, Inner shadow, Filters, Selection colors, Guides, Video, Export ► [Flex 1:44](#). Border has a side dropdown; the demo sidebar uses a 1 px border on the "Right" side only ► [Flex 7:52](#).

3. Procedures

3.1 Make a flex frame

1. Select the loose layers (Shift-click to add) and press ⌘A, or click "Wrap in flex ⌘A" ► [Flex 1:22](#).
2. To convert an existing freeform frame instead, select the frame and press ⌘A; the button reads "Add flex ⌘A" in that case ► [8 tips 0:57](#).
3. To group layers without flex, press ⌘F ("Frame selection") ► [8 tips 0:42](#).
4. Rename the frame by double-clicking its name on the canvas or in the layers panel ► [Flex 1:44](#).

After ⌘A the gap was 8 in one demo and 12 in another (a list that was already stacked), so check the value rather than assume it ► [Flex 1:44](#) ► [8 tips 1:06](#).

3.2 Set spacing

1. With the parent selected, set the gap: type it, use arrow keys (Shift for bigger steps), drag the field icon, or drag the pink band between children on the canvas ► [Flex 2:00](#).
2. Set horizontal and vertical padding; click the square icon at the end of the padding row for individual sides ► [Flex 3:17](#).
3. Fills accept tokens: in Fill, pick a token (the demo uses "muted"); the row shows the token name and a detach icon ► [Flex 3:06](#).

When you drag a new layer into a flex frame, a blue outline shows the receiving frame and a blue line shows where it will land ► [Flex 2:28](#).

3.3 Direction, alignment, space between

1. Click → for a row or ↓ for a column ► [Flex 3:57](#).
2. Click one of the nine points to align children inside the frame ► [Flex 4:28](#).
3. For Space between, open the sliders icon and choose "Space between", or double-click the nine-point grid ► [Flex 5:45](#).

3.4 Split alignment with nesting

Space between spreads every child, so with three children the middle one floats to the center ► [Flex 6:08](#). Two patterns fix it:

- **Left group, right item (a row).** Shift-select the items that belong together, ⌘A, set their gap. The parent now has two children and Space between pushes them to the edges ► [Flex 6:18](#). Stacked text inside the group (name over email) is one more ⌘A with direction ↓ and gap 0 ► [Flex 7:11](#).
- **Top group, bottom item (a sidebar).** A vertical frame aligned top-left stacks a new child under the others ► [Flex 7:52](#). Set the sidebar to Space between, wrap the brand and nav with ⌘A (gap 24); the group stays at the top and the user button at the bottom, whatever the height ► [Flex 8:08](#).

3.5 Sizing

1. Fixed: resize on the canvas, type W or H, or use arrow keys ► [Flex 8:37](#).

2. Fit: pick Fit in the dropdown or double-click the top or bottom edge
▶ [Flex 5:06](#).
3. Fill: pick Fill in the W dropdown, or Option + double-click the left or right edge ▶ [Flex 9:37](#).
4. Parent driven by a child: set the parent's W to Fit, then change the child's width; the parent follows, and siblings set to Fill follow the parent
▶ [Flex 10:03](#).

A whole split-view dashboard built this way resizes without breaking, which the video frames as looking right on both a 14-inch and a 16-inch laptop
▶ [Flex 10:43](#).

3.6 Overlays: Absolute position plus constraints

1. Select the overlay child (a close button in the demo) and tick "Absolute position"; it floats above the content and ignores the flex rules ▶ [Flex 11:28](#).
2. Hold Option to read distances and place it (16 from the top and right in the demo) ▶ [Flex 12:00](#).
3. Paper starts it at Left / Top, so it drifts when the parent grows
▶ [Constraints 0:27](#). Pin the right side in the diagram (tooltips read like "Pin top") or choose "Right" in the horizontal dropdown ▶ [Constraints 1:05](#).
4. Resize the parent to test.

3.7 Center, Left + Right, Stretch

1. A modal inside a freeform frame stays Left / Top even if you drag it to the middle ▶ [Constraints 1:18](#).
2. Use the Layout header's align buttons (horizontal centers, then vertical centers); Paper sets both constraints to Center for you ▶ [Constraints 1:42](#).
3. Choose the horizontal behavior ▶ [Constraints 2:06](#):

Horizontal constraint	On parent resize
Center	Width fixed, side spaces change
Left + Right	Side spaces fixed (500 px in the demo), width changes ▶ Constraints 2:30
Stretch	Width and side spaces change in proportion; W shows a percentage ▶ Constraints 2:44

3.8 Images and color

1. Crop: hold ⌘ while dragging an image's edges ▶ [8 tips 1:37](#).
2. Reset: double-click an edge or corner, or click the pixel-size chip under the file name in Fill > Image ▶ [8 tips 2:05](#) ▶ [8 tips 2:17](#). A 1600×1200 file came back at 800 × 600; the halving is not explained.
3. Palette from a photo: right-click > "Extract colors". Paper adds a freeform frame "Magic Swatches" holding rectangles named by hex, such as "Color #A65C0D" ▶ [8 tips 2:22](#) ▶ [8 tips 2:36](#).
4. Sample a color: select a layer and press I for the eyedropper loupe
▶ [8 tips 3:05](#). The full color picker has "sRGB" and "Display P3" tabs and shows LCH, HSL and RGB values, each with a copy button ▶ [8 tips 3:00](#).

The full right-click menu on an image reads, top to bottom: Copy ⌘C, Copy link ⌘L, Copy as, Paste ⌘V, Paste on top ⇧⌘V, Paste to replace ⇧⌘R, Duplicate ⌘D, Copy styles ⇧⌘C, Paste styles ⇧⌘V, Frame selection ⇧F, Wrap in flex ⇧A, Show / hide ⇧⌘H, Lock / unlock ⇧⌘L, Arrange, Select

parent (Escape), Select children (Enter), Vectorize image, Extract colors, Remove background, Open image in new tab. Only Extract colors is demonstrated ► [8 tips 2:30](#); magic swatches and background removal each get a later short ([LKmKPuN8bnU](#), [zG5DixnVTC4](#)).

3.9 Selecting and moving around

- Default hover targets the top-level parent under the cursor (Figma's behavior); double-click to go deeper, or hold ⌘ to deep-select ► [Hover 0:13](#).
- To make hover pick the closest frame instead, click the file name at the top of the left panel and tick "Use deep selection" ► [Hover 0:36](#). That menu also holds Back to dashboard ⇧⌘D, Show pixel grid ⌘', Snap to pixel ⇧⌘', Layout guides ⇧G, Multiplayer cursors ⇧M, Open links in desktop app and Nudge settings... ► [Hover 0:38](#).
- ⌘ + right-click lists every layer under the cursor, from the artboard down to the smallest shape ► [Hover 0:57](#).
- Double-click a layer's icon in the layers list to zoom to and center it ► [8 tips 0:27](#).
- ⌘. hides all UI; ⇧. collapses it into floating pills (file name, toolbar, Copy link and zoom) ► [8 tips 0:12](#) ► [8 tips 0:21](#).

3.10 Files and folders

1. In the dashboard, open Files and click "+ New folder"; the name field opens ready to type, prefilled "Untitled project" ► [Folders 0:08](#).
2. Right-click a file > Move to > folder; the current location is greyed out ► [Folders 0:24](#).
3. Open a folder and repeat "+ New folder" to nest, as deep as needed ► [Folders 0:39](#).
4. Folders move the same way, confirmed by a toast ► [Folders 1:03](#).
5. Navigate with the breadcrumb; rename with right-click > Rename or by double-clicking the current folder's name in the breadcrumb ► [Folders 1:20](#).

The file menu also has Open in new tab, Copy link and Archive, and the header has "Search files ⌘F" and a grid or list view ► [Folders 0:24](#). The dashboard sidebar holds Recents, Learn, then the team's Files, Archive and Settings ► [Folders 0:08](#); from inside a file, Back to dashboard is ⇧⌘D ► [Hover 0:38](#).

3.11 Export PDFs

1. Select a frame, scroll to "Export" at the bottom of the right panel, set the format to PDF ► [PDF 0:20](#).
2. Optional: the sliders icon opens Settings with File suffix, Color space (sRGB), Quality (High) and Image resampling (Detailed) ► [PDF 0:36](#); the demo deck lists the alternatives as Medium or Low quality and Basic resampling for lighter files ► [PDF 1:36](#).
3. Click "Export ⇧⌘E". The file goes to Downloads ► [PDF 0:42](#), named after the frame ("Steve Jobs resume.pdf") ► [PDF 1:48](#).
4. For several frames, select them all: "Export N elements" writes one PDF per frame, "Export combined PDF" writes one multi-page file ► [PDF 1:04](#).

Paper prints PDFs through the browser because it renders with HTML and CSS, so text stays real text ► [PDF 0:03](#). The combined four-slide deck (1980 × 1080 slides) came out as "Combined.pdf" at 554 KB ► [PDF 1:48](#); ⌘F in Preview finds words ► [PDF 1:22](#), and Claude (desktop app, model menu showing Opus

4.8) summarized the deck accurately from the file ► [PDF 1:43](#). In the combined file the title slide came first although it was last in the layers list, so order follows the canvas, not the layers ► [PDF 1:22](#). The MCP's export_combined_pdf description states the rule: top-to-bottom, then left-to-right, using the highest PDF quality and resampling set on any of the nodes (MCP tool description).

4. Advanced techniques

- **Build dashboards flex all the way down.** The split-view demo has no freeform positioning, which is why it survives resizing ► [Flex 10:43](#). Freeform frames are fine for illustrations and swatch boards; Paper's own Extract colors output is freeform ► [8 tips 2:36](#).
- **Flow plus overlay.** Keep the card in flex and put only the badge or close button on Absolute position with pinned sides ► [Flex 11:28](#) ► [Constraints 1:05](#).
- **Gutters that never change.** Left + Right keeps fixed margins while the element grows, the right choice for full-width banners inside fixed gutters ► [Constraints 2:30](#).
- **Size the parent from its most important child.** A Fit parent with Fill siblings lets one component (a user menu, a chart legend) set the width of a whole column ► [Flex 10:03](#).
- **Order pages by placing boards.** Since combined PDFs follow canvas position, lay report boards out in reading order before exporting ► [PDF 1:22](#).
- **Styles as a clipboard.** Copy styles ⌘C and Paste styles ⌘V sit in the right-click menu, useful to align a regenerated element with an approved one (listed, not demonstrated) ► [8 tips 2:30](#).

5. Pitfalls and limits

Pitfall	What to do
Absolute children start at Left / Top and drift on resize ► Constraints 0:27	Pin the sides you mean
Dragging to the visual center does not center constraints ► Constraints 1:18	Use the align buttons
Space between spreads every child ► Flex 6:08	Group with ⇧A first
A dropped child stacks under the others in a top-aligned frame ► Flex 7:52	Space between plus a nested group
⇧A does two different things ► 8 tips 0:57	Check whether a frame or loose layers are selected
Exports land in Downloads ► PDF 0:42 ; the MCP export tools take no output folder (MCP tool description)	Move files right after exporting
"Relative" sizing is unexplained ► Flex 9:37	Avoid it until documented

Gaps that the Paper team acknowledged in viewer comments (dates approximate): no keyboard shortcut for Fit or Fill yet, with a pointer to the shortcut list at [paper.design/docs/support#keyboard-shortcuts](#) (early September 2026, on [8 tips](#)); scaling a whole design with locked proportions, like Figma's K, is coming (mid-September 2026, on [Flex](#)); resizing an image together with its frame, and resizing with locked aspect ratio from the W/H fields, are coming, and meanwhile the team suggests asking an agent to do the scaling (early July 2026, on [8 tips](#)).

6. Apply it to your files

1. **Poseidon, nav rail.** Rebuild the sidebar as a vertical flex frame on Space between, with brand and nav wrapped by ⌈A at gap 24 and the user block as the second child ▶ [Flex 8:08](#). Drag the board taller and shorter; the user block must stay at the bottom.
2. **Altitude Sports and Water Dashboard, KPI rows.** On a 1536 board, four 352 cards with 24 px gaps make a 1,480 px row (28 px side padding); with 16 px gaps the row is 1,456 px (40 px padding). Build the row as horizontal flex with Fixed 352 children, then on a duplicate switch the cards to Fill and narrow the board to 1440 to see how they reflow ▶ [Flex 9:37](#). Keep the duplicate on your 99 Archive page if you want it.
3. **Ballottage, overlays.** Put floating CTAs, the liquid-glass bottom bar and corner badges on Absolute position. Set bars to Left + Right and pin their bottom handle in the diagram, set badges to Right and Top, and pin figurines to the bottom so they keep sitting on their surface ▶ [Constraints 0:38](#) (the video only shows Top and Center in the vertical dropdown, so use the diagram). Resize the artboard to another size from the Phone presets ▶ [Flex 0:38](#) and check nothing detaches.
4. **Budget tool, report PDF.** Place the 1400 × 990 boards in reading order in one row, name each frame as you want its file named, select them, use "Export combined PDF", then search a figure in Preview with ⌘F ▶ [PDF 1:04](#) ▶ [PDF 1:22](#). If it fails like your earlier attempt, export one board alone to find the page that breaks.
5. **The Projects folder.** Create Clients, Products, Systems, Social and Backups, move your 14 files into them, and put both Sauvegarde files in Backups ▶ [Folders 0:24](#).
6. **Social posts, palette.** Run Extract colors on the hero photo of a post, keep the swatch frame next to the post, and use I to sample from it for accents ▶ [8 tips 2:22](#) ▶ [8 tips 3:05](#). Before reviewing any of these, check that Nudge settings... is 8, the step the flex video describes for Shift+arrow ▶ [Flex 2:00](#), and turn on Layout guides ⌈G ▶ [Hover 0:38](#).

7. Prompt recipes

Paste the preamble once per session, then any recipe. Tool behaviors cited in the recipes come from the Paper MCP tool descriptions, not from the videos.

```
Paper is open on the file <FILE>. Load the Paper MCP guide first.
Rules: edit existing boards incrementally (one visual group per write),
get_screenshot after each group, pause about 500 ms between writes,
never use write_html in replace mode on a frame that has absolutely
positioned children, and call finish_working_on_nodes at the end.
Never delete boards or pages; move replaced boards to the 99 Archive page.
```

R1. Flex audit, read-only (Altitude Sports).

```
On page "01 Dashboard", list every frame with two or more children that is
not display:flex (freeform), plus every child using position:absolute.
Use get_tree_summary and get_computed_styles. For each, say whether it
should become a row or a column, the gap and padding it would need on our
8 px rhythm (gaps 16 or 24), and which children should be Fixed, Fit or
Fill. Do not write anything; give me the table first.
```

R2. Convert one group to flex (Water Dashboard KPI row).

Convert the KPI row on the main board to a horizontal flex frame: gap 24px, cards width 352px fixed, row centered on the 1536 board. Use `update_styles` on the existing nodes, not `write_html`. If `update_styles` returns `ignoredStyles`, stop and tell me why. Then read the row back with `get_computed_styles`, take a screenshot, and report the row width (expected 1480px).

R3. Sidebar with a pinned footer (Poseidon).

Make the left nav a vertical flex frame with `justify-content space-between`, padding 24px top and bottom, 16px left and right. Wrap the brand block and the nav list in one child frame (vertical, gap 24px) so there are exactly two children: that group and the user block. Verify by setting the nav height 200px taller, screenshot, then restore the original height.

R4. Overlay pinned to a corner (Ballottage).

In the card `<CARD>` on `<SCREEN>`, make the badge `<BADGE>` `position:absolute` with top 16px and right 16px (pinned to the right, not a computed left). Keep the card itself in flex. If `update_styles` returns `ignoredStyles`, the node is not absolute yet: fix that first. Test: widen the card by 120px, screenshot, check the badge stayed 16px from the top-right corner, then restore the width.

R5. Responsive stress test (Altitude Sports).

Duplicate the "01 Dashboard" board twice next to the original. Set the copies to width 1440px and 1280px. Screenshot all three and list every element that clips, overlaps or leaves the grid. Do not fix anything yet. Then ask me whether to delete the two copies you created or move them to the 99 Archive page; touch nothing else.

R6. Combined report PDF (budget tool).

Export the report boards on page "04 – Rapport PDF" (1400x990 each) as one PDF with `export_combined_pdf`. Pages are ordered top-to-bottom, then left-to-right by canvas position: first list the boards in the order they will print and wait for my OK. Keep the default PDF settings. After export, move the newest PDF from `~/Downloads` to `./exports/<date>-rapport.pdf`, extract its text (`pdftotext` or Python), and confirm that every KPI value on the boards appears in the text. If the export fails, export each board alone with the export tool (`format pdf`) and tell me which one breaks.

R7. Magic Swatches to tokens (social posts).

I ran Extract colors on the hero photo; Paper created a frame named "Magic Swatches" with rectangles named "Color #HEX". Read them with `get_children` and `get_computed_styles`, propose at most two accent tokens that work on our neutral white grounds (no cream or sepia), and create them with `create_tokens` only after I approve the names and values.

R8. Folder plan (dashboard).

List my Paper files and propose a folder tree (Clients, Products, Systems, Social, Backups) with each file assigned. Flag files that look like backups or duplicates. I will do the moves myself in the dashboard.

Tokens, color and type

This chapter covers Paper's token system, how color gets into a file, and the typography controls that matter for agent-built designs. It is drawn from seven videos published between 9 June and 21 July 2026, with pointers to the August and September changelogs where a feature announced in June later shipped. Each claim is dated by its video: a June video describes Paper as it was in June.

The mental model

A token is a named value stored in the file. Paper keeps a file's tokens together as its "theme": the menu that copies them between files is called Theme

► [CSS vars to tokens 3:24](#). When a property uses a token, the panel shows a chip instead of a raw value. Detaching turns it back into a plain value you can edit freely ► [CSS vars to tokens 1:17](#) ► [Tokens with the team 3:24](#).

The token types follow Tailwind. Paper supports color, font family, font weight, font size, line height, letter spacing, spacing, breakpoints, containers and radius, and the presenter describes the system as mapping closely to Tailwind ► [June changelog 0:54](#). Opacity tokens arrived later and can be paired with a color token on the same fill ► [September changelog 1:52](#) ► [September changelog 2:05](#).

Names keep their CSS form; pickers shorten them. An agent creating tokens from code or from a design uses CSS-variable names such as `--color-accent`, `--text-7xl` or `--container-page` ► [Tokens with the team 2:40](#). The color picker lists them without the namespace ("accent", "blue-3") ► [Tokens with the team 2:24](#) ► [CSS vars to tokens 1:12](#). The size dropdown shows "lg 13", and hovering the field shows "text-lg" ► [CSS vars to tokens 2:50](#) ► [CSS vars to tokens 3:00](#).

Agents bind tokens through `var()`. Paper's canvas is HTML and CSS, and agents build boards with Write Html calls ► [CSS vars to tokens 2:01](#). Their HTML references tokens as `var(--name)`. The agent described its style guide as "wired" to the tokens that way ► [Tokens with the team 2:40](#). When a token later changed value and name, every bound element followed, style guide included ► [Tokens with the team 7:52](#) ► [Tokens with the team 8:02](#). This is the main reason to create tokens before an agent writes HTML: literal values do not follow later token changes ► [Tokens with the team 8:00](#).

Aliases are allowed. Semantic tokens can point at palette tokens through `var()`; the import kept aliases like `--color-popup` pointing at their base token ► [CSS vars to tokens 0:56](#). The palette must exist before the aliases that reference it ► [June changelog 0:32](#).

Tokens are per file, and there were no modes in June. Moving tokens to another file is a copy and paste through the Theme menu, while shared libraries are still in progress ► [CSS vars to tokens 3:08](#). Light and dark pairs written with `light-dark()` were imported with their light value only ► [CSS vars to tokens 0:56](#). Themes and modes were "in progress" at the end of June ► [June changelog 1:06](#). The team described modes as light, dark and possibly contrast, and theme classes as bundled styles: a heading style such as H1, or a card style combining shadows, borders and radius ► [Tokens with the team 9:11](#) ► [Tokens with the team 9:22](#) ► [Tokens with the team 9:32](#). The token docs

Videos in this chapter

7 videos, 26 minutes

If your font supports variable weights, your design tool should too!

9 Jun 2026, 1:27

Explore Paper's MCP-generated tokens with the Paper team

22 Jun 2026, 10:16

From CSS variables in your codebase to Paper design tokens with MCP

24 Jun 2026, 4:29

Create Design Tokens From Your Code Base

26 Jun 2026, 0:52

Copy and paste tokens between Paper files

26 Jun 2026, 0:25

New in Paper: tokens, vector editing, Figma import, and more

30 Jun 2026, 8:18

Create magic swatches in Paper

21 Jul 2026, 0:34

(paper.design/docs/tokens, linked from the video) hold the roadmap

► [CSS vars to tokens 3:58](#).

Where tokens appear in the panels

Property	Control	What you see	Source
Fill color	Four-circle button right of the color field (tooltip "Color tokens")	List of color tokens; chip plus a detach icon once applied	► Tokens from code 0:32 ► CSS vars to tokens 1:17
Border color	Border section	Chip such as "border"	► Tokens with the team 7:44
Font size	Dropdown on the size field	Detach, then steps with px values (xs 12 ... 7xl 82)	► Tokens with the team 4:08
Line height	Dropdown on the line-height field	Detach, Normal, then percentage steps (none 100%, tight 105%, snug 126%, normal 150%, relaxed)	► Tokens with the team 4:32
Letter spacing	Letter-spacing field	Tracking tokens	► Tokens with the team 4:46
Width and height	W / H dropdown	Fixed, Fit, Fill, Relative, then a Spacing group and a Container group (prose 560, content 1180, page 1440)	► Tokens with the team 5:20
Gap and padding	Layout fields	Spacing tokens	► Tokens with the team 5:11
Radius	Radius field	Radius tokens	► Tokens with the team 5:26
Audit	Selection colors (right panel)	Every color in the selection, as token names or hex, with usage counts; "Show all N colors"	► Tokens with the team 7:44 ► June changelog 4:24

Four ways to get tokens into a file

Route	Best when	Tool	Shown in
A. From your codebase	Code already defines the system	Agent with the Paper MCP	► CSS vars to tokens 0:18
B. From an existing design	The design is ahead of the code	Agent with the Paper MCP	► Tokens with the team 0:48
C. By hand	A few tokens, quick edits	Theme tab (August builds onward)	► August changelog 2:26
D. From another Paper file	Sharing one system across files	Paper logo menu > Theme	► Copy theme 0:10

Route A: one prompt from your codebase

1. Open the target file in the Paper desktop app. The agent works on whichever file is open, so the prompt does not need a file name
► [CSS vars to tokens 0:35](#).
2. In the agent, inside the repo, send: "Create tokens in the Open Paper file based on the CSS variables in this codebase." ► [CSS vars to tokens 0:30](#)
3. Expect this call order: Get Guide, Get Basic Info, Get Tokens, then several Create Tokens calls, batched by category ► [CSS vars to tokens 0:48](#). Each Create

Tokens call carries a `tokens` array of objects with a `type` (for example `"color"`) and a `name` in CSS-variable form (same frame). Cursor stopped for approval with Allowlist MCP Tool and Run buttons.

4. Read the summary. The demo reported 96 tokens: 65 colors, 16 typography, 10 radius and 5 spacing and layout, each group traced to its source file, with notes on how `light-dark()` and aliases were handled
▶ [CSS vars to tokens 0:56](#).
5. Spot-check on the canvas: draw a rectangle, click the four-circle button beside the Fill color, pick a token ▶ [CSS vars to tokens 1:07](#).
6. Ask for a style guide board that shows the tokens ▶ [CSS vars to tokens 1:27](#).
While the agent writes, dots move along the artboard's top edge and an agent avatar sits beside yours ▶ [CSS vars to tokens 1:43](#). The resulting board noted that its values resolve through `var()` aliases ▶ [CSS vars to tokens 2:06](#).
7. Select a text layer on the board: size and line height sit in small bordered boxes, which is how a typography token looks ▶ [CSS vars to tokens 2:50](#).

Route B: from a design you already have

1. Ask the agent for a style guide built from a named frame, and list the categories: font size, line height, letter spacing, colors, spacing, radius, breakpoints. Say what to do where the design is silent; the demo asked for Tailwind defaults for breakpoints ▶ [Tokens with the team 0:48](#).
2. Expect tokens even if you did not ask. The agent (Opus 4.8 in the Claude app) created 60 tokens in Tailwind v4 naming and bound the style guide to them ▶ [Tokens with the team 1:50](#) ▶ [Tokens with the team 2:40](#).
3. On the following request the agent first ran get tokens, found the set already in the file and did not create duplicates ▶ [Tokens with the team 2:40](#).
Make that check a rule in your prompts.
4. Build new screens on the system with a plain instruction such as "using our existing token system" ▶ [Tokens with the team 2:45](#). Ask for a mapping report; the demo agent listed which tokens went where (display font for headlines, mono for labels, radius steps per component) ▶ [Tokens with the team 7:20](#).

Route C: by hand (shipped after these videos)

In June, tokens could only be created through the MCP ▶ [CSS vars to tokens 0:02](#) ▶ [June changelog 0:25](#). The August changelog adds a Theme tab beside the Design tab, where you can search tokens, create one of any type and edit values ▶ [August changelog 2:26](#). The type list there reads Color, Radius, Spacing, Container, Breakpoint, Font family, Font weight, Font size, Line height and Letter spacing ▶ [August changelog 2:32](#). Editing a color updates everything that uses it, and you can compare against the previous color ▶ [August changelog 3:08](#) ▶ [August changelog 3:23](#). In September, a fill color could become a token from the fill's token button with "+" ▶ [September changelog 2:19](#), and the color token menu gained small-grid, large-grid and list views ▶ [September changelog 2:36](#). Those videos have their own chapters.

Route D: copy a theme from another file

1. In the source file, click the Paper logo at the top left, hover Theme, choose Copy theme ▶ [CSS vars to tokens 3:24](#) ▶ [Copy theme 0:10](#).
2. Open or create the target file; the demo used ⌘⇧N on the dashboard ▶ [Copy theme 0:12](#).
3. Paper logo > Theme > Paste theme. A "Theme pasted" toast confirms, and the token list fills ▶ [CSS vars to tokens 3:41](#).

The same submenu has Clear theme, which none of the videos demonstrates
▶ [Copy theme 0:02](#). A Paper team reply places this command in "the Edit Menu", but every video shows it under the Paper logo menu
▶ [Tokens with the team, comments](#). The narration also says a theme can be copied into a codebase, without showing the format ▶ [CSS vars to tokens 3:08](#).

Applying, swapping, detaching

1. Select the layer, click the four-circle button beside the Fill color, pick a token. The field becomes a chip and the button becomes a broken-link icon
▶ [CSS vars to tokens 1:17](#).
2. To swap, reopen the list and pick another token ▶ [Tokens with the team 3:13](#).
3. To detach a color, click the broken-link icon; the hex becomes editable
▶ [Tokens with the team 3:24](#). For type and size values, Detach is the first item of the field's dropdown ▶ [CSS vars to tokens 3:00](#).
4. The color picker has sRGB and Display P3 tabs, an eyedropper, L C H, H S L and R G B rows with copy buttons, a hex and opacity field, and Previous / New swatches ▶ [Tokens with the team 3:28](#). When you drag the hue slider, the selection point in the color field moves to follow the new hue instead of staying in place, which the hosts attribute to OKLCH
▶ [Tokens with the team 6:26](#).
5. The "I" key opens the eyedropper directly ▶ [8 tips 3:05](#).

Changing a token so the change sticks

The team demonstrated a full loop: explore by hand, then make the agent write the result back into the tokens.

1. Detach the color on one or two elements and try values in the picker. The hosts recolored a button and an eyebrow label teal ▶ [Tokens with the team 6:06](#).
2. Ask the agent to read the new colors from that frame, update the matching tokens to those values, and rename the tokens to fit ▶ [Tokens with the team 7:31](#).
3. The agent found `set_tokens`, which can change values and names in one call, renamed the accent and gold tokens, and the change reached every screen and the style guide ▶ [Tokens with the team 7:52](#).
4. Re-link the elements you detached. The agent warned that those two elements now held literal values and would not follow future token edits unless reconnected ▶ [Tokens with the team 8:00](#).
5. Check names and meanings. A related token kept a "gold" name after gold became teal, and the new teal sat close to the success green
▶ [Tokens with the team 8:00](#) ▶ [Tokens with the team 8:02](#).

Type: tokens and variable fonts

Type tokens. A type scale arrives as named steps with values, for example xs 12, sm 14, base 16, lg 19, 2xl 28, 4xl 40, 5xl 52, 7xl 82 ▶ [Tokens with the team 4:08](#). Line-height tokens were stored as percentages, which keeps them proportional at any size ▶ [Tokens with the team 4:37](#). The agent flagged this as a departure from px values in the brief and offered px line-height tokens per size instead ▶ [Tokens with the team 2:40](#). Font family and font weight are token

types too ► [June changelog 0:54](#). One viewer reports that weights applied through the MCP showed as "unavailable" in the text layer's weight control ► [CSS vars to tokens, comments](#), so check weights after an agent pass. Theme classes, which would bundle family, weight, size, line height and letter spacing into one style such as H1, were announced but not shipped in June ► [Tokens with the team 4:46](#).

Variable font weights and optical size.

1. Select a text layer and click the more button (sliders icon) at the top right of the Text section ► [Variable fonts 0:12](#).
2. In the popover, open the Variable font tab (the others are Formatting and Features) ► [Variable fonts 0:14](#).
3. Drag Weight or type a value: 400 is Regular, 100 Thin, 800 ExtraBold, and values between the dots such as 364 or 560 work too ► [Variable fonts 0:28](#) ► [Variable fonts 0:36](#). The weight dropdown's label changes with the value; the demo frame cuts it off ("Thi..." at 100, "Extr..." at 800, "Wei..." at 364 and 560).
4. Leave "Automatic optical sizing" on unless you need a fixed optical size; the Optical size slider stays greyed while it is on ► [Variable fonts 0:14](#).

The presenter's reasons to bother: exact weights rather than the nearest named one, smooth weight animation, and one font file instead of one per weight ► [Variable fonts 0:46](#) ► [Variable fonts 1:07](#). The demo used DM Sans; behavior with static fonts is not shown.

Color from images: magic swatches

1. Copy an image in the browser and paste it into Paper with ⌘V ► [Magic swatches 0:14](#).
2. Right-click the image and choose Extract colors. The same menu offers Vectorize image and Remove background ► [Magic swatches 0:16](#).
3. A frame named "Magic Swatches" appears under the image, with one layer per color named "Color #RRGGBB" (six in the demo) ► [Magic swatches 0:22](#).
4. Apply a swatch with the eyedropper; "I" opens it ► [8 tips 2:29](#) ► [8 tips 3:05](#).
5. Swatches are plain layers, not tokens. To keep a color, make it a token: with the MCP, or (September builds) from the fill's token button with "+" ► [September changelog 2:19](#).

Agent and MCP notes

- **Call order.** The two code imports both read the guide, the file info and the existing tokens before creating anything ► [CSS vars to tokens 0:48](#) ► [June changelog 0:32](#).
- **Base before alias.** The agent's first plan put semantic colors first. It reversed that because palette colors must exist before aliases point at them ► [June changelog 0:32](#).
- **Name rewriting.** Paper turned Tailwind's double-dash pattern `--text-xs--line-height` into `--text-xs-line-height` ► [June changelog 0:32](#). Keep a mapping when porting back to code.
- **Updates and renames.** `set_tokens` changes values and names; bound nodes update ► [Tokens with the team 7:52](#).

- **Semantic names are your call.** Asked about ambiguous names like color-info-5, the team's answer was to instruct the agent to use semantic names [► June changelog, comments](#).
- **Dark variants without modes.** A later video asks the agent to mock up dark mode on a duplicated frame using the codebase's dark-mode variables rather than invented colors [► Remote Control 1:55](#).
- **Artboard placement.** Paper placed the agent's new artboards automatically to avoid overlap, so they may not line up [► Tokens with the team 7:20](#).
- **CSS without a panel control.** Properties the panel has no field for are listed under "Other styles" with a checkbox each (max-width and white-space-collapse in the demo) [► Tokens with the team 4:08](#). Look there when agent HTML carries CSS you did not expect.
- **Working across files.** Since late June an agent can create a file and open files, instead of only working on the file open in the app [► June changelog 6:28](#). That is what makes an MCP-based token copy between files possible (recipe 5 below; this exact flow is not shown in the videos).

Pitfalls and limits

Pitfall	Effect	What to do	Source
Detaching to experiment	The element keeps a literal value; token updates skip it	Re-link after the token update	► Tokens with the team 8:00
<code>light-dark()</code> in code	Only light values imported	Separate dark tokens, or a dark variant on a duplicated frame	► CSS vars to tokens 0:56 ► Remote Control 1:55
Percentage line heights	No guaranteed px rhythm	Ask for px line-height tokens per size	► Tokens with the team 2:40
Aliases created first	References to missing tokens	Palette first, then semantic tokens	► June changelog 0:32
Double-dash names	Paper names differ from code	Keep a mapping	► June changelog 0:32
Agent creates tokens unasked	Token sprawl, overlapping sets	State in the prompt whether tokens may be created	► Tokens with the team 1:50
Renaming one token	Related names go stale	Review the family after renames	► Tokens with the team 8:00
Per-file tokens	Copies drift apart	Re-copy from one master file; libraries pending	► CSS vars to tokens 3:08
Font weight via MCP	Weight control may read "unavailable" (viewer report)	Verify weights after agent writes	► CSS vars to tokens, comments
Magic swatches	Not tokens	Promote chosen colors to tokens	► Magic swatches 0:22

Untested in these videos: pasting a theme into a file that already has tokens, and what Clear theme removes.

Apply it to your files

1. **Altitude Sports: seed from code.** Run Route A from the repo of the live Railway app, then ask for a token board on a new numbered page ("NN Tokens", using the next free number before the archive page) without touching the existing dashboard pages. Accept the import only if the

summary lists counts per group, the source files, and how `light-dark()` and aliases were handled ▶ [CSS vars to tokens 0:56](#).

2. **One theme per product.** Keep each product's tokens in its own file. Copy a theme only between files of the same product: the two Water files (Water Dashboard and the budget tool) share one system, so copy from one to the other ▶ [Copy theme 0:10](#), then have an agent compare `get_tokens` counts across both. Nothing shows a pasted theme staying linked, so copy again after every change to the source file.
3. **Water Dashboard: color audit and rebind.** Select the main dashboard board and read Selection colors with "Show all N colors" ▶ [June changelog 4:24](#). Any hex that is not a token, and any red or orange, is a finding. Rebind one group at a time (KPI cards, then tables, then charts) with a screenshot after each, and re-link anything you detached ▶ [Tokens with the team 8:00](#).
4. **Chart system: type scale with px rhythm.** Create the chart label and title scale as tokens, decide between percentage and px line heights ▶ [Tokens with the team 2:40](#), and add series color tokens. Check that each label's line height lands on an 8 px multiple.
5. **Ballottage: weights and a palette from the renders.** Try exact variable weights on the score numerals ▶ [Variable fonts 0:36](#), and run Extract colors on a figurine render to test the 3D objects against the screen palette ▶ [Magic swatches 0:18](#).
6. **Social posts: palette per photo.** Extract colors from each post's hero photo, keep two or three swatches, and save them as role-named tokens (post-accent, post-ink) so the series stays consistent ▶ [Magic swatches 0:22](#).

Prompt recipes

Each recipe works as a single message to Claude Code with the Paper MCP. Replace anything in angle brackets with the real name from your file. Write them in French if you prefer; the tool names stay the same.

1. Seed tokens from a Tailwind v4 repo

```
Paper file "Altitude Sports" is open in the desktop app. Load the Paper MCP guide, then get_basic_info and get_tokens.
Create Paper tokens from the CSS variables of this repo's Tailwind v4 theme (the CSS file(s) with @theme and :root).
Rules:
- Base palette tokens first, then semantic tokens that alias them with var().
- Keep the Tailwind v4 namespaces (--color-*, --text-*, --leading-*, --tracking-*, --font-*, --font-weight-*, --spacing*, -
-radius-*, --breakpoint-*, --container-*).
- If a token already exists, skip it and list it. Do not rename or delete anything.
- light-dark() values: create the light value, and list every dark value you skipped.
- Batch create_tokens by category. If a call fails because the file disconnected, wait, then retry the same batch only.
Do not write or modify any board. Finish with a table: group, count, source file, notes on aliases and Paper's renamed
double-dash names.
```

2. Token board on its own page

```
In "Altitude Sports", create a page "<NN> Tokens" (next free number, before the archive page) and one 1440-wide artboard on
it.
Build the token board with write_html, one group per call: colors (palette, then semantic), type scale, line heights,
letter spacing, spacing, radius, containers.
Every value must reference its token with var(--name); no literal hex or px where a token exists.
Use the 12 x 64 grid with 32 gutters and the 8 px rhythm, white surface on a neutral light ground, no loose captions.
Call get_font_family_info before styling text. Take a screenshot after each group and fix clipping with height fit-content.
Call finish_working_on_nodes at the end.
```

3. Rebind an existing board without redesigning it

In "Water Dashboard", page "<dashboard page>": improve, don't regress. No layout, copy or spacing changes.

1. `get_tokens`, then read the board with `get_tree_summary` and `get_computed_styles`.
2. List every color, font size, line height and radius that has an exact token match, and every value with none.
3. Rebind exact matches to their tokens, one section at a time (KPI cards, table, charts), the way the Paper guide says tokens are applied. Try one node first and confirm the token chip shows in Paper.
4. Screenshot after each section. Do not touch values without a match: report them in a table instead. Flag any red or orange you find.

4. Push my manual color edits into the tokens

On the board "<board name>" I detached and recolored <element 1> and <element 2> by hand.

Run `get_tokens` first. Read their current fills, update the tokens they came from (<token 1>, <token 2>) to those values with `set_tokens`, and rename them only if the old name no longer describes the color.

Then re-link both elements to the updated tokens, list any other nodes that still hold literal colors, and check that the new accent does not sit too close to the success color.

Screenshot the board and the token board afterwards.

5. Copy the Water tokens into the budget tool over the MCP

Read all tokens from "Water Dashboard" with `get_tokens` (find its `fileId` with the file listing tool).

Open "Budget tool" and `get_tokens` there too. Create only the tokens it is missing, palette before aliases, and update any token whose value differs from the Water Dashboard file.

Do not touch boards. Report three lists: created, updated (old → new), and tokens that exist only in the budget tool, left as they are.

6. Dark variant without modes

Duplicate the frame "<frame name>" next to the original and name it "<frame name> / dark".

Recolor the duplicate using only the dark-mode values of this repo's CSS variables (`light-dark()` second values or the `.dark` selector), not invented colors.

Create any missing dark tokens with a `-dark` suffix first, then bind the duplicate to them. Leave the light frame unchanged.

Screenshot both side by side.

Vectors, images and AI generation

Paper treats vector art, generated images and pasted Figma work as ordinary layers on the canvas, so you and your agents edit them like anything else. This chapter covers SVG editing and the pen tool, generating SVGs with Quiver and images with Grok and other models, the image actions in the right-click menu, and Figma copy/paste. It draws on six videos published between 26 May and 29 August 2026, plus three cross-references to other channel videos. Paper ships fast, so every feature is described as of its video's date.

The mental model

An SVG is a container holding paths. In the layers list an SVG layer contains one or more Path layers; the container behaves like a frame and the paths are what you edit ► [SVG editing 1:31](#). Each path moves, resizes and rotates on its own ► [SVG editing 1:43](#) and carries its own Fill and Stroke ► [SVG editing 2:25](#). Nodes live inside paths and only appear in vector editing mode ► [SVG editing 2:04](#).

Double-click goes one level deeper. Select the SVG, double-click to select a path, double-click again to see its nodes ► [SVG short 0:10](#) ► [SVG short 0:16](#). Escape, or a double-click on empty canvas, takes you back out ► [SVG editing 2:04](#).

The container is the crop. New containers measure 256 x 256 ► [SVG editing 6:08](#) and the container panel shows "Clip content `\C`" ticked ► [SVG editing 1:28](#). A node dragged past the edge is cut off unless you enlarge the container ► [SVG short 0:24](#). Generated SVGs follow the same model: the Quiver result in the demo is a 256 x 256 container with Clip content ticked ► [Quiver SVG 2:24](#).

Generation is two toolbar buttons and one kind of prompt panel. "Create image" (⇧⌘I) ► [xAI images 0:02](#) and "Create SVG" (⇧⌘J) ► [Quiver SVG 0:11](#) each open a panel at the bottom of the canvas with a model dropdown and a create button. While a job runs, a placeholder layer named "Generating with ..." sits in the layers list ► [xAI images 0:54](#) ► [Quiver SVG 1:36](#). The result is a normal layer named from the prompt, such as "Monstera plant planter" or "Wobbly black hand" ► [xAI images 1:24](#) ► [Quiver SVG 2:24](#).

Selecting a result turns the panel into an editor. With a generated image selected, the panel shows it as context and the next prompt modifies it ► [xAI images 1:21](#). Each new version lands beside the source, which stays untouched ► [xAI images 3:16](#).

Image utilities live in the right-click menu: "Vectorize image", "Extract colors", "Remove background" and "Open image in new tab" ► [Remove BG 0:06](#).

Figma work pastes in as editable layers, with colours, fonts, shapes and structure intact ► [Figma paste 0:59](#); a later roundup adds that images and SVGs are fetched too ► [June roundup 5:20](#).

Which tool for which job

Videos in this chapter

6 videos, 23 minutes

Generate images in Paper using the xAI model

26 May 2026, 3:37

SVG editing in Paper: pen tool, path editing, and what's coming next

3 Jun 2026, 9:17

Copy designs from Figma and paste them into Paper

3 Jun 2026, 4:01

SVG editing just launched in Paper

5 Jun 2026, 0:48

Generate and refine SVG illustrations with Quiver AI inside Paper

18 Jun 2026, 4:36

Remove image backgrounds in Paper

29 Aug 2026, 0:26

Job	Tool	Where	Source
Tweak an icon or figure	Vector editing mode	double-click a selected path	► SVG editing 2:04
Draw a custom shape	Pen tool	toolbar, or P	► SVG editing 3:05
Illustration or icon from a prompt	Create SVG (Quiver)	toolbar, or ⇧⌘J	► Quiver SVG 0:11
Photo or render from a prompt	Create image	toolbar, or ⇧⌘I	► xAI images 0:02
Variations of an image	select it, prompt again	prompt panel	► xAI images 1:21
Cut out a subject	Remove background	right-click the image	► Remove BG 0:06
Raster to SVG	Vectorize image	right-click the image	► Shaders 3:47
Reuse Figma work	Cmd+C in Figma, Cmd+V in Paper	canvas	► Figma paste 0:39
Hand a section to code	Copy as React CSS or Tailwind	right-click > Copy as	► Figma paste 2:06

Edit an existing SVG

1. Select the SVG, then double-click to select one of its paths, or pick the Path in the layers list [► SVG short 0:10](#) [► SVG editing 1:43](#).
2. Move, resize or rotate the path with its handles; the other paths stay put [► SVG short 0:12](#).
3. Double-click the selected path to enter vector editing mode, then drag nodes [► SVG editing 2:04](#).
4. Leave with Escape or a double-click on empty canvas [► SVG editing 2:04](#).
5. A selected path shows only Fill and Stroke in the right panel. The fill swatch opens a picker with "sRGB" and "Display P3" tabs, an eyedropper, L C H, H S L and R G B fields, and a hex / opacity field [► SVG editing 2:32](#).
6. Click "+" next to Stroke to add one, then set width, colour and position [► SVG editing 2:32](#) [► SVG short 0:34](#).

Stroke position

The position dropdown values shown are "Above fill" and "Below fill". Above fill centres the stroke on the edge, half inside and half outside the fill's bounding box. Below fill puts it under the fill, so only the outer half shows and it reads as an outside outline [► SVG editing 2:40](#) [► SVG editing 2:54](#).

Draw with the pen tool

1. Press P or click the pen button in the toolbar [► SVG editing 3:05](#).
2. Click to place points. They snap to half pixels unless you disable it (the video does not show where); snapping to other objects was not available yet [► SVG editing 3:15](#) [► SVG editing 3:27](#).
3. Hold Shift to place points at 45° angles, and click the first point to close the shape [► SVG editing 3:27](#).

4. A new pen path starts with no fill and a 1 px black stroke, Above fill
- ▶ [SVG editing 3:20](#). Add a fill and remove the stroke if you want a solid shape
 - ▶ [SVG editing 3:43](#).

Goal	Gesture	Source
Pull curve handles from a node	hold Command and drag; you can release Command once it curves	▶ SVG editing 4:00
Move one handle on its own	hold Option	▶ SVG editing 4:15
Mirror the handles again	hold Command again	▶ SVG editing 4:28
Bow a straight segment	Command-drag from the middle of the edge	▶ SVG editing 4:40
Insert a point	pen tool, hover an edge, click	▶ SVG editing 4:53
Drag nodes without adding any	enter vector editing with the move tool, not the pen	▶ SVG editing 3:43
Remove a section of a path	select its nodes in vector editing, press Delete	▶ Quiver SVG 3:35
Undo	Command+Z	▶ Quiver SVG 3:50

Paths and containers

- Drawing while a path is in editing mode extends that same Path layer
 - ▶ [SVG editing 5:03](#).
- For a separate Path in the same SVG, select the SVG or one of its paths without being in vector editing mode, then draw; the new Path has its own fill and stroke ▶ [SVG editing 5:14](#) ▶ [SVG editing 5:32](#).
- Drawing outside the container's box creates a new SVG. To merge it, Cmd+C the path and Cmd+V it into the original ▶ [SVG editing 5:52](#).
- True crop, scale and resize of SVGs was still on the roadmap. Workaround: select a path (or be in vector editing mode) and drag the container's edges to scale it, crop it or tighten it around the artwork ▶ [SVG editing 6:08](#).

Generate an SVG with Quiver

1. Click "Create SVG" in the toolbar or press ⌘ ⌘J ▶ [Quiver SVG 0:11](#).
2. Choose "QuiverAI Arrow 1.1" (the default, subtitled "Balanced SVG quality and speed") or "QuiverAI Arrow 1.1 Max" ("For detailed SVGs, takes more time") ▶ [Quiver SVG 0:24](#). Those were the only two on 18 June; the Arrow Telos model your agent sessions call through paper-gen:// was not in that list.
3. Paste the prompt and click "Create SVG" ▶ [Quiver SVG 0:37](#) ▶ [Quiver SVG 1:31](#).
4. The SVG builds itself live on the canvas; Max needs extra time
 - ▶ [Quiver SVG 1:42](#) ▶ [Quiver SVG 1:58](#). With the result selected, the panel shows it in a history row: thumbnail, prompt and three unlabeled icons
 - ▶ [Quiver SVG 2:24](#).
5. Run the cleanup pass below.

By October 2026 the same two choices are called Arrow 2 and Arrow 2 Telos. Through the MCP they are the `quiver-arrow` and `quiver-arrow-telos` models, at 3 and 4 usage units an image ([Image models](#)). Telos is the one you use most (115 runs), so draft with Arrow and keep Telos for finals.

Prompt anatomy

Part	Advice	Demo, paraphrased	Source
Subject	say what you want	one hand making an OK sign	► Quiver SVG 0:56
Style	describe the line, and say illustration, icon or logo	one continuous wobbly brush line, loose modern line art	► Quiver SVG 1:06
Colours	name specific colours	black, cream, lavender, flat three-tone	► Quiver SVG 1:19
Restraint	ask for minimal, lots of negative space	two offset paper rectangles, plenty of empty space	► Quiver SVG 1:19

The cleanup pass

1. Expand the SVG in the layers list. The demo came back as 3 Polygon layers and about 20 Path layers [► Quiver SVG 2:26](#).
2. Hide unwanted parts with the eye toggle before deciding to delete them [► Quiver SVG 2:26](#).
3. Compare stroke widths path by path. Lines meant to match came back at 1.28 and 0.99, and she set both to 1 [► Quiver SVG 2:38](#).
4. Delete stray strokes: each one is its own path, so select it and delete it [► Quiver SVG 3:01](#).
5. For a path that is partly wrong, enter vector editing, move the doubtful piece aside to see what it is, then delete its nodes; Command+Z restores mistakes [► Quiver SVG 3:35](#) [► Quiver SVG 3:50](#).
6. Budget time: the presenter skipped about five minutes of editing [► Quiver SVG 3:59](#).

Generate and iterate images

1. Click "Create image" or press ⌘ I [► xAI images 0:02](#).
2. Pick a model. The default "Variety pack" sends the same prompt to four models and returns four images [► xAI images 0:14](#).
3. Pick an aspect ratio; the default is 1:1 [► xAI images 0:26](#).
4. Write the prompt and click "Create image" [► xAI images 0:41](#). Her prompt names the medium (photorealistic), the subject, the surface it stands on, the light, the mood and a photographic register (an interior design photo); she adds that more detail gives better results [► xAI images 0:57](#).
5. To iterate, select the result. The panel shows a context row with the image and its prompt; click away to drop the context (she also mentions an X on that row, not visible in the recording) [► xAI images 1:21](#).
6. Ask for one change per prompt, keep the same model, and reselect the original before each new change [► xAI images 1:36](#) [► xAI images 2:24](#).

Models on 26 May 2026 [► xAI images 0:17](#):

Model	Subtitle in the menu
Variety pack	OpenAI Image 2, Grok, Imagen 4, Seedream 4.5
OpenAI Image 2	Great with text, can take longer
Imagen 4	Google's advanced model

Model	Subtitle in the menu
Flux 2 Pro	Fast and versatile
Grok	xAI image generation
Recraft V3	Good at rendering text
Ideogram v3	Dreamy and stunning
Seedream 4.5	Precise, good with text
Nano Banana Pro	2K resolution

The roster moves: by late September an agent-built variety-pack board was labelled OpenAI Image 2.5, Nano Banana 2, Flux 2 Pro and Recraft V4.1
▶ [Sept roundup 1:44](#).

Aspect ratios: Square 1:1; Landscape 3:2, 4:3, 16:9, 2:1; Portrait 2:3, 3:4, 9:16, 1:2 ▶ [xAI images 0:30](#). While an image is selected as context, the ratio control is hidden ▶ [xAI images 1:24](#).

What comes back. Grok returned the first image in about two seconds
▶ [xAI images 1:08](#). The layer is placed at 592 x 444 with an Image fill labelled "Grok 1184×888", so at half its pixel size ▶ [xAI images 1:24](#). Edits kept the planter, plant and fallen leaves in place ▶ [xAI images 1:58](#), though one edit erased a side wall ▶ [xAI images 2:11](#). Versions were auto-named "Sunset time", "Terracotta pot" and "Falling plant leaves" and laid out around the original ▶ [xAI images 3:16](#). A banner then warned that the rate limit was close, that new images arrive each Saturday, and offered "Upgrade" ▶ [xAI images 3:06](#).

Remove a background

- Right-click the image and choose "Remove background" ▶ [Remove BG 0:06](#).
- After a couple of seconds the subject is on a transparent background, still inside the original frame size (1731 × 2181 in the demo) ▶ [Remove BG 0:08](#)
▶ [Remove BG 0:10](#).
- Drop the cutout into the target frame, which crops it at its edges
▶ [Remove BG 0:16](#) ▶ [Remove BG 0:18](#). The short also shows the cutout over other backgrounds, but not how those were made ▶ [Remove BG 0:12](#).

The same menu holds "Vectorize image". In a March demo, right-click > vectorize ran an AI model that turned a flat image into an SVG that scales without losing quality ▶ [Shaders 3:47](#) ▶ [Shaders 4:05](#). "Extract colors" is listed but not demonstrated in any of these videos ▶ [Remove BG 0:06](#).

Right-click shortcuts worth knowing

Item	Shortcut	Source
Copy as PNG / Copy as Tailwind / Copy as React CSS	⇧⌘C / ⌘T / ⌘R	▶ Figma paste 2:06
Paste on top / Paste to replace	⇧⌘V / ⇧⌘R	▶ Remove BG 0:06
Duplicate	⌘D	▶ Remove BG 0:06
Copy styles / Paste styles	⌘C / ⌘V	▶ Remove BG 0:06
Frame selection / Wrap in flex	⇧F / ⇧A	▶ Remove BG 0:06

Item	Shortcut	Source
Remove flex / Ungroup	⇧⌘A / ⇧⌘G	► Figma paste 2:06
Bring to front / Send to back	] / [	► Remove BG 0:06
Show / hide, Lock / unlock	⇧⌘H, ⇧⌘L	► Figma paste 2:06
Select parent / Select children	Escape / Enter	► Figma paste 2:06

Bring designs in from Figma

1. In Figma, select frames or layers and press Cmd+C [► Figma paste 0:39](#).
2. In Paper, press Cmd+V; a toast says images will be fetched [► Figma paste 0:49](#).
3. The first time, an "Images from Figma" dialog asks for access. Choose "Connect and continue" (or "Continue without images"), approve in the browser, and return to Paper [► Figma paste 0:58](#) [► Figma paste 1:00](#).
4. Check names: the frames arrive with their Figma names, here "Property 1=Variant4" and "Property 1=Variant5" [► Figma paste 0:48](#).
5. For code, right-click > Copy as > "Copy as React CSS" or "Copy as Tailwind". The React output opens with a "from Paper" comment, the file link and a date, then a component with inline styles [► Figma paste 2:06](#) [► Figma paste 2:18](#). By September the same options also sat in the export panel behind a copy icon [► Sept roundup 3:38](#).

Figma components paste in detached, because Paper had neither components nor tokens on 3 June [► Figma paste 3:15](#).

Where agents fit

Only the Figma video uses an agent. The setup: Claude desktop app, a local Claude Code session, "Accept edits", model "Sonnet 4.6 · Medium", picked because the task was simple [► Figma paste 1:30](#) [► Figma paste 1:48](#). She targeted the frame by pasting its name into the prompt, not by selection: In Paper, duplicate "Property 1=Variant4" frame and translate the copy to spanish [► Figma paste 1:48](#). The log shows the order of work: Paper guide, find the frame, duplicate, set text content on every text node of the copy, screenshot to verify, finish working on nodes [► Figma paste 2:36](#). The duplicate kept the original's name [► Figma paste 2:36](#).

Image generation also works through the MCP: in the September roundup an agent uses the variety pack and labels each image with the model that made it [► Sept roundup 1:31](#).

Not covered by these videos: driving vector editing, Remove background or Vectorize through the MCP. None of the MCP tools you call is a background-removal or vectorize command, so treat those as manual canvas steps and let the agent handle composition around them.

Advanced techniques

- **Outside outlines:** set Stroke to Below fill so a thick outline never eats into a thin shape [► SVG editing 2:54](#).

- **Arcs without extra points:** Command-drag the middle of a straight segment, then Option-drag one handle for an asymmetric curve
▶ [SVG editing 4:40](#) ▶ [SVG editing 4:15](#).
- **Merge strays:** a shape drawn outside the container becomes its own SVG; Cmd+C, Cmd+V puts it back ▶ [SVG editing 5:52](#).
- **Model audition, then lock:** use the variety pack for the first pass ▶ [xAI images 0:14](#), then keep one model for every edit; she checks Grok is still selected before each follow-up ▶ [xAI images 1:36](#).
- **Always branch from the source:** reselecting the original before each change keeps edits from compounding ▶ [xAI images 2:24](#).
- **Hide, then delete:** on generated SVGs, hide parts first and delete only once the drawing holds up ▶ [Quiver SVG 2:26](#).
- **Recolour a whole SVG (untested):** the container panel lists "Selection colors" with a use count per colour (070707 used 21 times on the Quiver output); editing there may recolour every path at once, but no video shows it ▶ [Quiver SVG 2:24](#).
- **The canvas as a copy test:** a translated duplicate exposes new line breaks before code does ▶ [Figma paste 2:45](#).

Pitfalls and limits

Limit (as of the video date)	What to do	Source
SVG editing is beta	expect rough edges, report issues	▶ SVG editing 0:02
No snapping to other objects	half-pixel snapping, Shift for 45°, zoom in	▶ SVG editing 3:15
No booleans (unite, subtract)	build shapes from separate paths	▶ SVG editing 1:02
No true SVG crop or resize	drag the container edges with a path selected	▶ SVG editing 6:08
Nodes past the container are clipped	enlarge the container	▶ SVG short 0:24
Deleting nodes removes that section; no delete-and-keep-the-path, no click to remove a point	move the piece aside first, Command+Z; under the SVG editing video Paper replied that node deletion without breaking the path is on the roadmap, and viewers also asked for Option or Control click removal and vertex removal	▶ Quiver SVG 3:35 ▶ SVG editing 0:00
Generated SVGs are fragmented, strokes uneven	cleanup pass	▶ Quiver SVG 2:38
A prompted background colour may not survive	the lavender ground showed while building, then was absent; add your own background layer	▶ Quiver SVG 2:06 ▶ Quiver SVG 2:12
Image edits can change unrequested details	review every version	▶ xAI images 2:11
Weekly image quota	batch generation; "Upgrade"; a viewer's question whether models run on your own API keys went unanswered	▶ xAI images 3:06
Fixed aspect list, no 4:5	generate 3:4, crop with a frame	▶ xAI images 0:30
Cutouts keep the full original frame	trim the transparent margin before placing	▶ Remove BG 0:10
Figma paste is beta	report problems to team@paper.design or the Slack and Discord communities	▶ Figma paste 3:27
Figma images need a Figma connection	connect once, or continue without images	▶ Figma paste 0:58
One way only, no whole-file import	copy frames in batches; under the Figma paste video Paper replied that Paper to Figma is "not right now" and full import is only being considered	▶ Figma paste 0:00

Limit (as of the video date)	What to do	Source
Figma components arrive detached	rebuild them in Paper	► Figma paste 3:15
Agent duplicates keep the same name	rename after duplicating	► Figma paste 2:36

Announced as coming

On 3 June the roadmap board listed four things as available now: edit paths, adjust fill and stroke, import and generate SVGs, and move, resize and rotate layers [► SVG editing 0:36](#). Everything below was not yet shipped on that date.

Item	Horizon on 3 June 2026	Detail	Source
Boolean operations, improved snapping, path editing guides, convert shapes to paths, crop / scale / resize SVGs	near-term	none shown	► SVG editing 1:02
Mirroring	long-term	prototype panel "Mirrors" with a "Vertical" entry; horizontal also planned	► SVG editing 6:49
Repeating	long-term	line, grid and radial; the narration calls the demo a line repeater but its panel reads "Radial repeater" with count and angle fields; edits to the source update every copy	► SVG editing 7:30 ► SVG editing 7:51
Anchoring	long-term	slide an attached object along a path without editing the path	► SVG editing 8:21
Anneal brush, shape builder, vector networks	long-term	board labels; vector networks visible in the clips	► SVG editing 0:36 ► SVG editing 8:07
Components and tokens	"coming soon"	by 30 June tokens existed (created through the MCP); components were still in progress	► Figma paste 3:15 ► June roundup 0:12 ► June roundup 7:47

On 30 June the team said more SVG editing functionality was in the works [► June roundup 7:47](#); none of this chapter's videos confirms that the near-term items have shipped. Worth watching for a chart system: a radial repeater would turn gauge ticks or radial guides into one source path plus a modifier, as in the prototype clip [► SVG editing 7:39](#).

Apply it to your files

- 1. Chart figures: fix one vector figure by hand.** Duplicate the figure (⌘D) and move the original to your archive page [► Remove BG 0:06](#). Double-click down to a leader-line path and its nodes [► SVG short 0:16](#), then drag the elbow node with Shift held for a clean 45° leader [► SVG editing 3:27](#). Give the highlighted region a Below fill outline [► SVG editing 2:54](#). If a marker is clipped, drag the container edge with a path selected rather than regenerating [► SVG editing 6:08](#).
- 2. Design system: three Quiver icons that match.** Draft prompts with recipe F, generate with Arrow 1.1 for speed [► Quiver SVG 0:24](#), then run the cleanup pass: hide extras, one stroke width for every path, delete strays [► Quiver SVG 2:38](#). Size each container to your icon grid with a path selected [► SVG editing 6:08](#). Rejects go to the archive page.
- 3. Ballottage: iterate a figurine, then cut it out.** Select the figurine image so it becomes context [► xAI images 1:21](#) and change one attribute per prompt (light,

material, expression), branching from the original each time ► [xAI images 2:24](#). On the winner, right-click > "Remove background" ► [Remove BG 0:06](#), trim the margin, then use recipe E to stand it on the ring with a contact shadow.

4. Social posts: a 4:5 post from a 3:4 generation. There is no 4:5 ratio, so generate at 3:4 ► [xAI images 0:30](#) and let a 1080 x 1350 frame crop it, as the thumbnail frame crops the cutout ► [Remove BG 0:18](#). Check the Image fill label for the source pixel size before scaling up ► [xAI images 1:24](#). Share it with "Copy as PNG" (⌘⌘C) ► [Figma paste 2:06](#): as a copy command it should go to the clipboard rather than to ~/Downloads (the videos list it but do not demonstrate it).

5. Budget tool: an FR to EN wrap test. Run recipe C on the main budget board, then read the list of text whose line count changed, the same effect the Spanish demo surfaced ► [Figma paste 2:45](#). Decide the fixes yourself, then move the duplicate to the archive page.

6. Poseidon: one route arc by hand. Pen (P), click origin, click destination ► [SVG editing 3:05](#); Escape, re-enter with the move tool, Command-drag the segment's middle into an arc ► [SVG editing 4:40](#), Option-drag one handle to skew it ► [SVG editing 4:15](#), and add a stopover point with the pen ► [SVG editing 4:53](#).

Prompt recipes

Ready to paste into Claude Code with the Paper MCP connected. Replace the angle brackets. Each one targets layers by name, works on duplicates and asks for screenshots, the same pattern as the agent run in the Figma video ► [Figma paste 2:36](#).

A. Read-only audit of a generated SVG

Paper file "Chart figures". Load the Paper guide first. Find the SVG layer named "<figure name>" with find_nodes; do not rely on my selection. Read only, change nothing. List its children (type, name, size) and, for each path, the fill, stroke colour, stroke width and whether it extends past the container, using what get_children, get_node_info and get_computed_styles return. Give me one table, flag stroke widths that differ from the most common value, and say plainly if the tools do not expose path-level stroke data.

B. Normalise strokes on a duplicate

Same SVG. Duplicate it, move the original to the "99 Archive" page, and work only on the copy. Read the cobalt colour from get_tokens. Set every path's stroke width to <1> and stroke colour to that cobalt, one update_styles call per group of about five paths, with a get_screenshot after each group. If update_styles cannot change strokes on SVG paths, stop and tell me which property failed; I will fix it by hand in the Stroke field. Call finish_working_on_nodes at the end.

C. Translation wrap test

In Paper, file "Budget tool", page "01 Dashboard": duplicate the frame "<frame name>" and translate its copy from French to English. Rename the duplicate "<frame name> EN" and place it 64 px to the right of the original, top-aligned. Change text content only, never styles or sizes. Take a screenshot of both frames, then list every text node whose line count changed and any text that now overflows its card. Do not fix anything.

D. Variety-pack audition for a prop

Paper file "Ballottage". Create a page "06 Prop tests" and work only there. In the Paper canvas, use the variety pack of models to generate <a boxing-ring corner post with ropes, 3D, soft studio light, standing on a light grey floor, neutral light background>. Lay the four results in one row on the 8 px grid, each inside a white card that holds the name of the model that made it; no captions floating outside the cards. Screenshot when done.

E. Stand a cutout on the ring

Paper file "Ballottage". I removed the background of the image layer "<cutout name>" by hand. Move it into the frame "<ring scene>", scale it to <N> px tall and place it so its feet touch the ring floor at y=<value>. Add a separate contact-shadow layer under the feet: a dark radial-gradient ellipse at 20 to 30 percent opacity, no backdrop-filter. Keep everything on the 8 px grid, take a screenshot after each step, and stop if the figure looks like it floats.

F. Quiver prompts for design-system icons

Read the design system's colour tokens with get_tokens. Then write three prompts for Paper's Create SVG panel (QuiverAI Arrow 1.1) for these icons: <calendar>, <alert>, <export>. Each prompt names the subject, the style (single-weight outline icon, rounded joins, no fill), the exact colour as one cobalt hex from the tokens, and the constraints (centred, generous padding, no background shape, no text). Under 40 words each, as a numbered list I can paste.

Shaders and motion

This chapter is built from seven Paper videos published between March and August 2026, with three changelog cross-references up to 28 September 2026. Each feature is described as of the video that shows it. The interface moved during that period (the shader button changed from a cube icon to a diamond icon, and the toolbar became a floating bar) ► [Shaders tour 0:24](#) ► [Logo shaders 0:40](#), so expect small visual differences in your build; the names of panels and parameters stayed stable.

Two facts frame everything below. First, in these videos all motion inside Paper comes from shaders; animation of UI elements is added later, in code ► [Design first 0:18](#). Second, none of the seven videos shows an agent creating or editing a shader through the Paper MCP. Shaders are dialed by hand in the panel, and agents enter the picture for layout around them and for porting them to code.

The mental model

- **A shader is a layer.** It sits in the layers list under its type name (Warp, Fluted Glass, Pulsing Border) with its own icon, and it keeps the normal layer properties: Layout, Radius, Blending, Fill, Outline, Border, Shadow ► [Shaders tour 0:22](#). Later builds also list Inner shadow, Filters, Video and Export under it ► [Logo shaders 7:20](#). You size it, round it, nest it and reorder it like a frame.
- **On top of that sits a shader block:** Presets, sometimes an input (Image or Shape), Parameters, and one or more color lists (Foreground, Colors, Tint, Inner) ► [Logo shaders 0:56](#).
- **Three families**, matching the picker's three sections. Image filters transform an image you feed them ► [Shaders tour 1:32](#). Logo animations react to the geometry of a shape you supply ► [Logo shaders 0:44](#). Effects generate patterns from parameters alone, Warp and Pulsing Border among them ► [Resend button 1:06](#).
- **Everything moves by default.** Speed drives the animation; at 0% the shader becomes a still texture ► [Shaders tour 1:00](#).
- **Presets are starting points.** Picking one rewrites all parameter values; you tune from there ► [Shaders tour 0:50](#).
- **The eyedropper is the connector.** In an Image or Shape input it captures any canvas layer, including text, logos and other shaders ► [Shaders tour 2:09](#) ► [Halftone text 0:10](#) ► [Logo shaders 1:08](#). In a color list it picks colors from another element ► [Resend button 1:32](#). Captured inputs are stored as .webp files named after the source layer, such as "Paper Logo.webp" or "Fluted Glass.webp" ► [Logo shaders 1:20](#) ► [Shaders tour 3:30](#).
- **The code is the shader.** Right-click, Copy as React CSS gives a component from the `@paper-design/shaders-react` package with every parameter as a prop, and Paper says it is the same code that renders the shader on the canvas ► [Shaders tour 1:12](#). Panel percentages become decimals in the props (Speed 350% is written 3.5, Scale 247% is 2.47) ► [Shaders tour 1:24](#). Since July 2026 Paper Shaders are open source and free to use in sites, apps, templates and plugins ► [Logo shaders 0:02](#).

Videos in this chapter 7 videos, 23 minutes
How to use Paper Shaders to elevate your designs 5 Mar 2026, 4:48
Recreate Cloudflare's loading shader from scratch with Paper Shaders 2 May 2026, 2:11
Remix Resend's action button with Paper Snapshot and Paper Shaders 28 May 2026, 4:30
How to use logo animation shaders in Paper 7 Jul 2026, 9:05
Understanding contour in Paper's logo animation shaders 9 Jul 2026, 0:56
Apply a Halftone Shader to Text 20 Jul 2026, 0:55
Paper shader of the month: Lens Distortion 21 Aug 2026, 0:35

The catalogue

Picker section	Shaders	Input	Where seen
Image filters	Paper Texture, Fluted Glass, Water, Image Dithering, Halftone Dots, Halftone CMYK	Image (a sample "flowers.webp" until you replace it)	March 2026 ► Shaders tour 1:36
Image filters	Lens Distortion	Image	August 2026 ► Lens Distortion 0:04
Logo animations	Heatmap, Liquid Metal	Shape	March 2026 ► Shaders tour 1:36
Logo animations	Gem Smoke	Shape	July 2026 ► Logo shaders 0:40
Effects	Mesh Gradient, Static Mesh Gradient, Static Radial Gradient, Dithering, Grain Gradient, Dot Orbit, Dot Grid, Warp, Spiral, Swirl, Waves, Neuro Noise, Perlin Noise, Simplex Noise, Voronoi, Pulsing Border, Metaballs, Color Panels, Smoke Ring, God Rays	none, parameters only	May 2026 ► Resend button 1:06

Lens Distortion launched as August's "shader of the month" [► Aug changelog 8:14](#). In September 2026 Paper rebuilt the Paper Texture shader from scratch, with better rendering, more controls and new patterns such as wrinkles, folds and crumples [► Sep changelog 4:06](#). Paper says more shaders are coming [► Logo shaders 0:29](#).

Core procedures

1. Add and tune any shader

1. Press **S**, or click the shader button in the toolbar (tooltip "Shaders S")
[► Halftone text 0:04](#) [► Cloudflare loader 0:32](#).
2. Pick a shader in the **Shaders** picker. It lands as an 800 x 600 layer; resize it to your slot [► Cloudflare loader 0:40](#).
3. Click through **Presets** and keep the closest one [► Cloudflare loader 0:45](#).
4. Set colors. Click a swatch to open the color picker (sRGB / Display P3 tabs, an eyedropper, OKLCH, HSL, RGB and hex fields), or use the color list's own eyedropper to pull colors from another element; remove extra colors with the minus [► Cloudflare loader 1:12](#) [► Resend button 1:32](#).
5. Tune **Parameters**. Harcourt's habit: drag a slider to its maximum to see the extreme, then choose a value [► Logo shaders 5:01](#).
6. Set **Speed** last: lower for calm motion, 0% for a still texture
[► Shaders tour 1:00](#).

2. Feed an image filter, then chain filters

1. Add an image filter such as Fluted Glass. Its **Image** block shows the sample image with an eyedropper and a trash icon [► Shaders tour 2:09](#).
2. Click the eyedropper (tooltip "Capture image from another element") and click any layer: a photo, a generated image, a text layer or another shader [► Shaders tour 2:12](#). Uploading from disk is also possible; the narration calls the control "edit" without showing it. Stephen feeds it images from Paper's own generator, whose Variety pack option runs four models at once (Flux 2

Pro, Imagen 4, Ideogram v3 and Seedream 4.5 in March 2026)

► [Shaders tour 1:44](#).

3. Try the filter's presets (Fluted Glass: Default, Abstract, Waves, Folds) and tone the effect down if you want it subtle ► [Shaders tour 2:49](#).
4. To chain, add a second filter and eyedrop the first shader. Stephen puts Halftone CMYK with the Vintage preset on top of the Fluted Glass output, which arrives as "Fluted Glass.webp" ► [Shaders tour 3:12](#).

3. Animate a logo or any shape

1. Press S, go to **Logo animations**, pick Heatmap, Liquid Metal or Gem Smoke ► [Logo shaders 0:37](#).
2. In **Shape**, keep a preset shape (the default is a diamond; Liquid Metal also offers None, Circle, Daisy, Metaballs), click **Choose file** to upload, or use the eyedropper on a canvas layer such as your logo ► [Logo shaders 0:58](#) ► [Logo shaders 3:11](#).
3. Set the ground with the shader's **Fill**; all three demos end on black ► [Logo shaders 4:38](#).
4. Set **Contour** (Heatmap and Liquid Metal), then glows, noise, angle, speed and scale ► [Logo shaders 3:32](#).

4. Match a reference in minutes

The Cloudflare rebuild is a reusable method ► [Cloudflare loader 0:16](#):

1. Screenshot the reference and paste it next to your work. Paper gave the pasted screenshot a descriptive layer name ("orange and white background") on its own ► [Cloudflare loader 0:33](#).
2. Add the closest family (Warp for fluid loaders) and match the aspect ratio, square here ► [Cloudflare loader 0:42](#).
3. Keep the nearest preset (Cauldron Pot) ► [Cloudflare loader 0:54](#).
4. Match colors first with the picker's eyedropper, and delete colors the reference does not have ► [Cloudflare loader 1:03](#).
5. Then motion: Speed, Scale, Softness, swirl. High Softness blends the colors and hides the lightest one ► [Cloudflare loader 1:38](#). The "more swirl" step shows on screen as Swirl iterations going from 7 to 10 while Swirl stays at 86% ► [Cloudflare loader 1:48](#).

5. Put a shader inside a component

The Resend button remix is the reference build for a shader living inside UI:

1. Duplicate the component to keep a before and after ► [Resend button 0:55](#).
2. Set the page background to the real product's background, so you judge the glow in context ► [Resend button 1:14](#).
3. Add the shader (Pulsing Border), remove the shader's own Fill, and pick its colors from the original with the Foreground eyedropper (tooltip "Pick colors from another element") ► [Resend button 1:26](#).
4. Match the component's shape: Scale 100%, Roundness 100% for a pill, Radius Full ► [Resend button 1:54](#).
5. Remove the component's own fill and drag the shader into the outer frame, not into the content frame ► [Resend button 2:44](#).
6. In Layout, check **Absolute position** (it sits under Wrap in flex; once checked, Left and Top constraint dropdowns appear), set W and H to **Fill**, set X and Y to 0 ► [Resend button 2:56](#).

7. For a hard inner edge, put the shader behind the opaque content frame: in the demo, dragging it above Content in the layers list did that

► [Resend button 3:31](#).

6. Test it in context with Paper Snapshot

1. In Chrome, enable Paper Snapshot, hover the section, fine-tune with the up and down arrows, then click or press Return; a toast confirms the copy
► [Resend button 0:10](#) ► [Logo shaders 6:06](#).
2. In Paper, Cmd+V, then Shift+A to wrap the capture in a flex frame; add a matching fill and padding ► [Logo shaders 6:06](#). Snapshot keeps the structure: for the Resend button, an outer frame, a rectangle holding the hover state (hide it if you do not need it) and a content frame with the text and SVG
► [Resend button 0:33](#).
3. Drag the shader in, remove its fill and make it absolute; Harcourt's footer shader shows Absolute position with Center and Top constraints, a step the narration skips ► [Logo shaders 6:48](#).
4. Turn on **Clip content** (⌘C) on the frame so nothing spills ► [Logo shaders 6:55](#).

7. Shader effects on text

1. Add Halftone Dots or Lens Distortion and eyedrop a text layer into its Image block ► [Halftone text 0:10](#) ► [Lens Distortion 0:18](#).
2. For Halftone Dots: Type Goody, Scale up until the glyph fits, then Size, Radius and Contrast until it reads, then Foreground and Fill colors
► [Halftone text 0:14](#).
3. For Lens Distortion: start from a preset (Default, Fisheye, Duotone, Smoke) and adjust Lens bulge, Scale and Position; one headline can carry several variants side by side ► [Lens Distortion 0:20](#).

8. Get it out of Paper

- **Code:** right-click, Copy as React CSS (⌘R) or Copy as Tailwind (⌘T)
► [Shaders tour 1:12](#). The pasted code carries a header comment with the package version and a comment linking back to the Paper file and node
► [Shaders tour 1:24](#). Since September 2026 the copy options also sit behind a copy icon in the export panel ► [Sep changelog 3:50](#).
- **Still image:** Copy as PNG in the same menu ► [Shaders tour 1:12](#), or the Export section.
- **Video:** shader panels carry a Video section ► [Cloudflare loader 0:40](#), and the Paper team says video export is on the Pro plan (► [Logo shaders, comments](#)). Since September, image and video exports run on HTML in Canvas, which Paper says is faster and higher fidelity ► [Sep changelog 3:50](#).

Parameter reference

Presets and parameters are listed in panel order. "Values seen" are the settings a presenter landed on, useful as starting points.

Shader	Presets	Parameters	Values seen
Warp	Default, Cauldron Pot, Live Ink, Kelp, Nectar, Passion	Shape (Checks / Stripes / Edge), Speed, Scale, Softness, Proportion, Swirl, Swirl iterations, Distortion, Shape scale; Foreground	Defaults: Checks, Speed 100%, Scale 100%, Softness 100%, Proportion 45%, Swirl 80%, Swirl iterations 10, Distortion 25%, Shape scale 10% ► Cloudflare loader 0:40 . Cauldron Pot: Edge, Speed 1000%, Scale 90%, Softness 150%, Proportion 64%, Swirl 86%, Swirl iterations 7, Distortion 20%, Shape scale 60% ► Cloudflare loader 1:06 . Cloudflare match: Edge, Speed 420%, Scale 69%, Softness 64%, Proportion 64%, Swirl 86%, Swirl iterations 10, Distortion 20%, Shape scale 60% ► Cloudflare loader 1:48
Pulsing Border	Default, Circle, Northern lights, Solid line	Aspect ratio (Auto / Square), Speed, Scale, Roundness, Softness, Rotation, Thickness, Intensity, Bloom, Spot count, Spot size, Pulse, Smoke, Smoke size; Margins; Foreground	Starting values: Scale 60%, Roundness 25%, Softness 75%, Thickness 10%, Intensity 20%, Bloom 25%, Spot count 5, Spot size 50%, Pulse 25%, Smoke 30%, Smoke size 60% ► Resend button 1:16 ► Resend button 1:26 . Pill button: Scale 100%, Roundness 100%, Softness 54%, Thickness about 40%, Intensity 33% ► Resend button 3:34
Heatmap	Default, Sepia	Shape; Angle, Noise, Inner glow, Outer glow, Contour, Speed, Scale; Foreground	Black and white logo: Angle 45°, Noise 27%, Inner glow 28%, Outer glow 27%, Contour 50%, Speed 49%, Scale 53% ► Logo shaders 2:56
Liquid Metal	Default, Noir, Backdrop, Stripes	Shape; Repetition, Contour, Distortion, Softness, Angle, Redshift, Blueshift, Speed, Scale; Tint	Repetition 200%, Contour 40%, Distortion 7%, Softness 47%, Angle 180°, Blueshift 0%, Tint 5E5E5E on black ► Logo shaders 5:46
Gem Smoke	Default, Fire, Fluorescent, Infrared	Shape; Size, Outer distortion, Inner distortion, Outer glow, Inner glow, Offset, Angle, Speed, Scale; Colors, Inner	Faint footer: Size 94%, Outer distortion 65%, Inner distortion 80%, Outer glow 0%, Inner glow 42%, Offset -20%, Angle 115° ► Logo shaders 8:20
Fluted Glass	Default, Abstract, Waves, Folds	Image; Shape (Lines, Irregular lines, Wave, among others), Distortion shape (Flat, Prism, Contour, among others), Fit (Contain / Cover), Size, Highlights, Shadows, Angle, Distortion, Shift, Stretch, Blur, Edges, Scale	Waves preset: Shape Wave, Distortion shape Contour, Size 90% ► Shaders tour 2:36
Halftone CMYK	Default, Drops, Newspaper, Vintage	Image; Type (Dots / Ink / Sharp), Fit, Size, Grid noise, Softness, Contrast, Scale; Colors; Gain	After picking Vintage: Type Sharp, Size 20%, Grid noise 45%, Softness 40%, Contrast 125% ► Shaders tour 3:18
Halftone Dots	not shown	Image; Type (Classic / Goody / Holes / Soft), Scale, Grid (Hex / Square), Size, Radius, Contrast; Foreground	Letter S: Goody, Hex, Size 88%, Radius 112%, Contrast 37% ► Halftone text 0:40
Lens Distortion	Default, Fisheye, Duotone, Smoke	Image; Fit, Spread, Bias, Focus center, Focus edges, Swirl, Noise, Noise frequency, Noise offset, Lens bulge, Lens circle, Scale, Position X, Position Y; Grain	Lens bulge around -70%, Scale 140% ► Lens Distortion 0:16

Advanced techniques

- Contour is the logo knob.** Every pixel of a logo animation knows its distance to the shape's edge, and Contour sets how far in pixels still react: 0% is a plain mask with no edge reaction, 18% hugs the rim, 100% floods the interior with edge-following bands [► Contour 0:20](#) [► Contour 0:28](#) [► Contour 0:38](#). Harcourt settles on 40% for Liquid Metal [► Logo shaders 4:25](#).
- Inner glow vs Outer glow.** Inner glow works inside the shape's bounds, Outer glow outside; lowering Scale reveals more of the outer glow [► Logo shaders 2:02](#). Next to text, set Outer glow to 0% so the copy stays legible [► Logo shaders 7:35](#).
- Grain and direction.** On Heatmap, Noise adds a grain texture (keep it low) and Angle turns the direction of the effect, 45° in the demo [► Logo shaders 1:18](#).

- **Still graphic shapes.** Speed 0% with Softness 0% turns Warp into hard-edged, poster-like shapes ► [Shaders tour 1:06](#).
- **Vectorize flat art.** Right-click a flat image and choose Vectorize; an AI model turns it into an SVG layer ("Vector") that scales to 3558 x 3558 without visible loss ► [Shaders tour 3:58](#) ► [Shaders tour 4:15](#). It is a companion to shaders for poster work, not a shader itself.
- **Chains.** Any shader can be the input of an image filter, so looks can be stacked: glass, then halftone ► [Shaders tour 3:12](#).
- **Hard edges by layer order.** Thickness alone only hardens the outer edge; an opaque content frame in front of the shader hardens the inner edge too ► [Resend button 3:20](#).
- **Clip content for smoke.** Smoke and glow shaders spill past their frame unless the parent clips ► [Logo shaders 6:55](#).
- **Name sources before capturing.** Captured inputs inherit the source layer's name ("Paper Logo.webp", "Vercel Logo.webp"), so clean names make chains readable later ► [Logo shaders 5:46](#).
- **Type exact colors.** The color picker takes OKLCH, HSL, RGB and hex, so token colors can be entered exactly instead of sampled ► [Cloudflare loader 1:12](#).
- **Code as documentation.** Pasting the copied React code next to the shader on the canvas, as Stephen does, keeps the exact values visible in the file ► [Shaders tour 1:24](#).

Motion outside shaders

Paper's brand designer built the company website in Paper, with shaders subtly in the background, and then prompted the animations on top while an agent turned the Paper file into a Next.js codebase ► [Design first 0:18](#). The Resend remix ends on the same idea for components: because Paper is HTML and CSS, agents translate the design back to code quickly ► [Resend button 3:55](#). A workable split for your projects: design and freeze shader motion in Paper (it exports as code), draw UI states such as hover or loading as separate frames in Paper, and have the agent write the transitions between them in code.

Pitfalls and limits

- **Flex parents.** A shader dropped into a flex frame is laid out next to the content until you check Absolute position ► [Resend button 2:56](#).
- **Spill.** Without Clip content on the parent, smoke escapes the frame ► [Logo shaders 6:55](#).
- **Presets overwrite.** Clicking a preset after tuning replaces your values ► [Shaders tour 0:50](#).
- **Overload.** Outer glow and Repetition overwhelm fast; Repetition at 1000% was "visually overwhelming" and went back to 200% ► [Logo shaders 4:46](#).
- **Legibility on type.** Text effects break the glyph at the extremes: halftone dots at maximum Size melt the letter into a blob ► [Halftone text 0:26](#), and strong lens settings smear a headline ► [Lens Distortion 0:20](#).
- **Captures are files.** Inputs are stored as .webp captures. Whether they refresh when the source changes is not shown, so re-capture after editing a source layer.

- **Disabled commands.** Vectorize and Remove background are greyed out on a shader's context menu ► [Shaders tour 1:12](#); Vectorize applies to images and works best on flat ones ► [Shaders tour 4:05](#).
- **React only.** The code export shown targets React, with the package version pinned in the header comment ► [Shaders tour 1:24](#). No vanilla JS export appears in these videos. The npm package also works without React: see [Paper Shaders, in Paper and in code](#).
- **No custom shaders.** You work with Paper's catalogue; a viewer's question about writing custom shader code got no answer (► [Shaders tour, comments](#)).
- **Plan gating.** Video export is a Pro plan feature (► [Logo shaders, comments](#)).
- **Resizing.** There is no aspect ratio lock; hold Shift while dragging to keep proportions (Paper team reply, ► [Resend button, comments](#)). A Scale tool was promised as coming soon in a team reply under the logo shaders video and is listed as coming next in September ► [Sep changelog 4:49](#).
- **Noise.** Among logo animations only Heatmap shows a Noise parameter; a viewer's request to add it to the others got an emoji, not a commitment (► [Logo shaders, comments](#)).
- **Layer order.** In the Resend demo, placing the shader above the content frame in the layers list rendered it behind; the list seems to follow DOM order, the reverse of Figma's habit (inference from one example). Check the canvas after reordering.
- **Agents.** None of these videos show the Paper MCP reading or writing shader parameters. Plan for manual shader tuning and verify what your agent can see before relying on it (see the first prompt recipe).

Apply it to your files

1. Ballottage: a pulsing call to action

1. On the screen with the main call to action, duplicate the CTA frame (⌘D) and rename the copy "CTA / pulse"; the original stays for comparison and later moves to your archive page.
2. Press S, add Pulsing Border, remove its Fill, and use the Foreground eyedropper on your liquid-glass accent; keep two or three palette colors.
3. Scale 100%, Roundness 100% if the CTA is a pill, Radius equal to the CTA radius (Full for a pill).
4. Remove the CTA's own fill, drag the shader into the CTA's outer frame, check Absolute position, set W and H to Fill, X and Y to 0.
5. Give the content frame an opaque fill and place the shader behind it for hard inner and outer edges.
6. Lower Speed until the pulse feels calm. Done when: the label contrast holds, edges are clean at 200% zoom, and both versions sit side by side for your call.

2. Ballottage: a living mark for the splash screen

1. Rename the logo layer "Ballottage mark" so the captured input is readable.
2. Press S, Logo animations, Liquid Metal; eyedrop the mark; set the Fill to the splash ground.
3. Duplicate the shader three times and set Contour to 0%, 18%, 40% and 100%, laid out in a row on your grid; keep one, archive the rest.

4. Leave Repetition at 200%, push Distortion to the maximum and back, move Tint toward your palette.
5. Done when: the mark is readable at app-icon size and the motion stays inside the shape.

3. Water Dashboard or Altitude Sports: a shader loader

1. On your "02 Grid and components" page, add a loader card on the grid.
2. Press S, Warp; use the Cloudflare order: nearest preset, then colors, then motion. Type your cobalt and white token values in the picker; for Water, no red or orange.
3. Duplicate it with Speed 0% as the static and reduced-motion state.
4. Copy as React CSS and paste the code as a text layer (IBM Plex Mono) next to the card; port it with the first code recipe below.

4. Social posts: textures and type effects

1. On a new page, create three post artboards at your post size.
2. Background: an Effects shader at Speed 0% recolored to the brand, or the rebuilt Paper Texture.
3. Headline: Halftone Dots on one large letter, or Lens Distortion with small values; every text sits on the grid, no captions floating.
4. Export PNGs into the project folder, not ~/Downloads.

5. A motion and shaders page in each product file

1. In each product file that uses shaders (Ballottage, Altitude, the social posts), add a numbered page for motion and shaders next to its components page.
2. One card per approved shader: a live instance, its purpose, the preset it started from, every parameter and color value, and the pasted React code.
3. Done when: an agent can rebuild any shader in code from the card's text alone, without a screenshot.

6. Altitude Sports: test in context before porting

1. Capture the live portfolio's hero or footer with Paper Snapshot and paste it on a test page; Shift+A.
2. Drop a faint Gem Smoke or a slow Warp behind the content: Absolute position, Clip content on the frame, Outer glow 0% near KPI text.
3. Compare with the untouched capture; keep or archive.

Prompt recipes

These are written for Claude Code with the Paper MCP and assume your usual rules: incremental edits, a screenshot after each group, short pauses between writes. Shader tuning stays manual unless the first recipe shows your MCP can see shader parameters.

0. Probe what the MCP sees in a shader (run once)

Paper file: Ballottage. I added one Pulsing Border shader on the Archive page, layer name "probe / pulsing border".

Read only. Call `get_node_info`, `get_jsx` and `get_computed_styles` on that layer and report:

1. the node type the MCP returns for a shader,
2. whether any shader parameters (speed, scale, colors, presets) appear, and under which keys,
3. whether position, size, radius and overflow of the shader and its parent are visible.

Do not write anything. End with a short table: "can read / cannot read".

1. Port a Paper shader into a React app

Project: <your React app>. I copied a shader from Paper with right-click > Copy as React CSS:

<paste the copied code here>

1. Read the header comment and install exactly that version of `@paper-design/shaders-react`.
 2. Create `components/shaders/<ShaderName>.tsx` as a client component rendering this shader with exactly these props. Do not round, rename or "improve" values.
 3. Paper is the source of truth for placement: with the Paper MCP, find the shader layer (use the node link in the code comment, or `find_nodes` by its layer name on the page I name) and read `get_computed_styles` for the shader and its parent. Reproduce the parent's size, radius and overflow, and the shader's absolute positioning (inset 0, full width and height) if that is what Paper shows.
 4. Add a `prefers-reduced-motion` branch that renders the same props with speed 0.
 5. Compare `get_screenshot` of the Paper frame with a screenshot of the running page; list differences in size, radius, colors or layering, then fix only those.
- Touch no other component. Report the files changed.

For a vanilla JS dashboard such as Altitude, ask for the same steps with `@paper-design/shaders` and `ShaderMount` instead of the React component; [Paper Shaders, in Paper and in code](#) lists the constructor and the uniforms.

2. Prepare the Ballottage CTA for a shader, then verify my manual step

Paper file: Ballottage, page with the main CTA. Incremental edits, `get_screenshot` after each step, pause 500 ms between writes.

1. `duplicate_nodes` the CTA frame; place the copy 48 px to the right on the same row; rename it "CTA / pulse".
2. On the copy, keep text and icon inside one content frame with an opaque fill equal to the current CTA background. The outer frame keeps the radius, gets no fill and clips its content.
3. Never use `write_html` in replace mode on these frames (it drops absolute positioning). `update_styles` only.
4. Screenshot both CTAs and stop with "ready for the shader".

When I answer "done" (I will have added a Pulsing Border by hand):

5. `get_children` and `get_node_info` on "CTA / pulse": confirm the shader is a direct child of the outer frame, absolute at 0,0 with full width and height, and rendered behind the content frame. Report each mismatch; fix it only if it is a plain CSS layout property, otherwise tell me which panel field to change.
6. Final screenshot of both versions side by side.

3. Build a shader spec page in each product file

Files: Ballottage, Altitude Sports, Social posts. In each one, write only on a new page "Motion and shaders".

1. In each read-only file, use `get_tree_summary` and `find_nodes` to list layers that are Paper shaders (names such as Warp, Pulsing Border, Heatmap, Liquid Metal, Gem Smoke, Fluted Glass, Water, Paper Texture, Image Dithering, Halftone Dots, Halftone CMYK, Lens Distortion, Mesh Gradient, Grain Gradient, or a node type that says shader).
2. For each: `get_screenshot` and `get_node_info`; record file, page, parent path, size, radius, and any shader parameters the MCP exposes. If parameters are not exposed, mark the card "paste Copy as React CSS here" and list those shaders for me at the end.
3. In each file, create the next numbered page "Motion and shaders". One card per shader on the 4 x 352 grid, 16 px gutters, 8 px rhythm: screenshot, name, where used, size, parameters, colors. Labels in Inter, values in IBM Plex Mono, every label inside its card.
4. Screenshot after each row of cards.

4. Shader post set

Paper file: Social posts. Create a new page "Shader posts"; do not modify existing pages.

1. create_artboard three times at our post size, named "Post / halftone", "Post / lens", "Post / texture". Neutral light ground, our palette.
2. In each, lay out the headline and the logo on the grid and leave an empty frame named "SHADER SLOT" where the effect goes. No captions, no floating text.
3. Screenshot and stop. I will add the shaders by hand (Halftone Dots, Lens Distortion, a still texture at Speed 0).
4. When I say "export": export each artboard as PNG at 2x. Files must end up in the project folder under exports/, never in ~/Downloads; if the export tool cannot choose a folder, move the files right after and confirm ~/Downloads holds no copy.

5. Motion direction panel for the Ballottage splash

Run a panel: three designer subagents and one judge (all Opus), then you as lead. No Paper writes until the lead step. Each designer writes a motion spec for the Ballottage splash mark: which Paper shader (Liquid Metal, Heatmap, Gem Smoke, Warp or Fluted Glass) and why it fits Ballottage (3D figurines, liquid glass, boxing ring); the Shape input; the starting preset; every parameter value including Contour and Speed; colors from our palette; where it sits (behind which frame, clipped or not).

The judge ranks the specs on: mark legibility at app-icon size, consistency with the liquid glass, calm motion, no random colors.

Lead: put the winning spec on a card on a new numbered "Motion" page in Ballottage (screenshot after), then list the exact manual steps I must do in the shader panel, in order.

Agents in Paper

This chapter draws on the eleven agent videos on the Paper channel, from the first Notion demo (March 2026) to the Opus 5.5 test (late September 2026). Each feature is described as of the date of the video that shows it. You already run Paper almost entirely through agents, so the goal is not to introduce the MCP but to fill the gaps the videos expose: what the server needs, how the Paper team prompts, comments, tabs, remote sessions, model choice, and the failure modes behind your "not connected" and "file disconnected" errors.

The mental model

The canvas is HTML and CSS. Paper uses HTML and CSS as its rendering engine, which agents already write fluently. Tools with their own rendering engine cost agents more tokens and time and give lower accuracy

► [Connected canvas 0:14](#). This is why `write_html` is the natural way in. It also explains where writes lose fidelity: CSS with no equivalent in Paper's node model. Your own sessions list those edges (backdrop-filter, repeating-linear-gradient, transform: scale, styled inline spans, empty divs).

Agents read and write. Reading means the canvas is visual context next to your prompt: reference images, a pasted draft, your manual edits

► [Teams with agents 0:16](#). Writing means drafts, many variations to choose from, and boilerplate such as applying styles or renaming layers

► [Teams with agents 0:38](#). The designer keeps the decisions.

The server lives in the desktop app. The videos show a local server named `paper` at `http://127.0.0.1:29979/mcp` ► [Cursor setup 1:00](#). It works only while Paper Desktop is running with a design file open ► [Cursor setup 1:08](#). A Paper file in the browser does not count, and neither does the desktop dashboard alone ► [Claude Code setup 1:36](#). It is a local port, so there is no account token involved (a viewer's answer under ► [Cursor setup](#)), which is also why your scripts can call the same endpoint directly.

The plugin is more than the server. As shown in May 2026, the `paper-desktop` plugin bundled the server, two skills (`code-to-design` and `design-to-code`, callable with `/` or used automatically) and an auto-applied rule, `ensure-paper-started`, that reminds the agent to keep Paper open

► [Cursor setup 1:00](#). Your installed v0.1.0 still has them. [Your setup](#) records that upstream v0.2.1 moved to stdio through the Paper CLI and dropped the skills and rule, and that two Paper registrations are active in your sessions with slightly different tool sets.

The standard agent loop. Every demo follows the same order. Knowing it lets you spot a run going off track.

Step	What the agent does	Seen in
1	Loads the Paper guide and reads the file (file info, selection, structure)	► Fable 5 vs Opus 4.8 1:06 , ► Notion data 1:24
2	Checks which fonts exist (<code>get_font_family_info</code>) before any typography	► Notion data 1:54

Videos in this chapter

11 videos, 30 minutes

Design with real content: How to pull Notion data into Paper with Claude Code

5 Mar 2026, 3:29

Agent set up: How to connect Claude Code to Paper using the Paper MCP server

2 May 2026, 4:50

Use an AI agent to explore different design directions in Paper

2 May 2026, 6:15

Agent set up: How to connect Cursor to Paper using the Paper MCP server

11 May 2026, 1:39

Claude's Fable 5 vs Opus 4.8: which model designs better in Paper?

10 Jun 2026, 4:22

Use Claude's Remote Control to design in Paper from your phone

31 Jul 2026, 3:00

Using tabs to run agents across multiple Paper files at once

10 Aug 2026, 2:57

Leave comments for your agent in Paper

19 Aug 2026, 0:27

Paper: the connected canvas for teams shipping with agents

26 Aug 2026, 0:35

Paper is the design tool for teams building with agents

18 Sep 2026, 1:12

What can Opus 5.5 design in five minutes?

24 Sep 2026, 0:55

Step	What the agent does	Seen in
3	Writes a design brief: palette with hex values, type scale, spacing, direction	► Claude Code setup 2:22
4	Creates the artboard and builds it in chunks with <code>write_html</code>	► Claude Code setup 2:42
5	Screenshots its own work and runs a review checklist	► Claude Code setup 3:42
6	Summarizes, then releases the nodes with <code>finish_working_on_nodes</code>	► Fable 5 vs Opus 4.8 3:29

The review checklist in step 5 covers spacing, typography, contrast, alignment, clipping and repetition ► [Claude Code setup 3:42](#); Opus 4.8 added fixed icon and label lanes, artboard fit and color restraint ► [Fable 5 vs Opus 4.8 3:29](#).

What you see while it works. A row of animated dots above the artboard being read or edited ► [Claude Code setup 2:42](#). Since tabs (August 2026), dots also appear on the tab of the file the agent is in, plus a robot avatar beside yours at the top right ► [Tabs and agents 0:52](#).

What it does not see. The agent does not track your manual edits. Paper's own advice, printed in the email designed in the Remote Control video, is to have it re-read the canvas after you rework something ► [Remote Control 1:48](#). The same email advises asking for small, specific sections at a time rather than large blocks ► [Remote Control 1:48](#).

Connect an agent

Claude Code in a terminal (as of May 2026)

1. Install Claude Code: `curl -fsSL https://claude.ai/install.sh | bash` ► [Claude Code setup 0:24](#).
2. Run `claude`, accept the folder trust check, sign in if a browser window asks ► [Claude Code setup 0:43](#).
3. Add the server: `claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user`. It is written to the user config, `~/.claude.json` ► [Claude Code setup 1:05](#).
4. Restart Claude Code. A server added inside a running session does not load its tools; Claude says so and suggests `claude mcp list` to verify ► [Claude Code setup 1:52](#).
5. Open Paper Desktop on a design file, start `claude` again, and run `/mcp` to see `paper` connected ► [Claude Code setup 1:05](#).
6. Prompt with the words "in Paper" so the agent targets the canvas ► [Claude Code setup 2:22](#).

Claude desktop app (your setup)

- Two videos run Paper from the Code tab of the Claude desktop app: a local folder, "Accept edits" mode, and model and effort chips in the composer ► [Design directions 0:12](#), ► [Fable 5 vs Opus 4.8 0:17](#).
- To a viewer struggling with Claude Desktop, the Paper team answered: use Claude Code inside the Claude desktop app, and email team@paper.design for help (► [Claude Code setup, comments](#)). They also confirmed Claude needs a paid plan (Pro) for this.
- The late-September release adds a one-click agent setup from inside Paper ► [New in Paper, Sep 28 0:54](#); [Snapshot, collaboration and releases](#) and

[Connecting an agent](#) cover it.

Cursor

- 1. The Paper docs give one command for Cursor: `/add-plugin paper-desktop` ► [Cursor setup 0:12](#).
- 2. Either open Cursor Settings (Command+Shift+J), go to the plugin marketplace, search "Paper" and click Get ► [Cursor setup 0:30](#); or paste the command into a New Agent chat, choose "Add for Myself" (or "Add to Project") and confirm "Add Plugin" ► [Cursor setup 0:48](#).
- 3. Check Cursor Settings > Tools & MCP. If the agent cannot see the tools, reload the window or toggle the server off and on ► [Cursor setup 0:12](#).

Pre-flight before any long run

Check	How	What it tells you
Is the MCP live, and on which file?	Ask "Are we connected to the Paper MCP?"	file, page, artboard and size, node count, fonts ► Design directions 0:24
Server status in a terminal	<code>/mcp</code>	status, auth, URL, tool count; actions View tools, Re-authenticate, Clear authentication, Reconnect, Disable ► Notion data 0:54
Server list from the shell	<code>claude mcp list</code>	whether the server is registered ► Claude Code setup 1:52
Context cost of each server	<code>/context</code> (named in Notion's docs on screen)	tokens used per enabled MCP server ► Notion data 1:00

Prompting habits the videos rely on

- **Name the target.** Say "in Paper" ► [Claude Code setup 2:22](#). With tabs, also name the file, paste its link, or say "the active Paper file" ► [Tabs and agents 0:42](#).
- **Point at the selection.** "Explore 10 variations of my selection" works because the agent can read what you have selected ► [Teams with agents 0:40](#), ► [Teams with agents 0:16](#).
- **Give the system up front.** Left alone, agents invent a palette and fonts: a terracotta accent ► [Claude Code setup 2:22](#), an orange scheme ► [Notion data 2:20](#), DM Sans with an indigo accent ► [Design directions 1:28](#). The Paper team's answer to "do I lose creative control?" is the same: give styles in the prompt and keep editing on the canvas (► [Design directions, comments](#)).
- **Ask for names.** Agents that name their outputs make follow-ups precise: "Ink" and "Slate" ► [Design directions 1:28](#), "01 Tidal" to "06 Mineral" ► [Opus 5.5 in 5 min 0:30](#), semantic section names ► [Notion data 3:05](#).
- **Expect a plan for anything structural.** When asked to merge two variants, the agent stated its plan with hex codes before building ► [Design directions 4:44](#).
- **Dictate when it is faster.** Two videos use the composer's microphone for long prompts ► [Opus 5.5 in 5 min 0:08](#), ► [Remote Control 1:02](#).

Workflows

Explore directions, then combine

1. Confirm the connection and the open file ► [Design directions 0:24](#).
2. Ask for N iterations of a named frame, varying colors and layout
► [Design directions 0:52](#).
3. The agent reads the frame's structure, proposes named directions, and builds each as a new artboard next to the untouched original
► [Design directions 1:28](#).
4. It screenshots each variant to check it is distinct and on brief, then moves on ► [Design directions 2:46](#).
5. Review against the original's feature list: one variant silently lost its add button ► [Design directions 3:36](#).
6. Combine by name: navigation from one, palette and side panel from another, plus a new accent ► [Design directions 4:17](#). The result landed in the empty slot of a 2 x 2 grid ► [Design directions 5:36](#), with top-level layers named Nav, Feed Column and Right Panel ► [Design directions 5:52](#).
7. Fix small things yourself. A too-dark color was changed by hand rather than with another prompt ► [Design directions 5:43](#).

Real content from other tools

The agent can combine the Paper MCP with any other source. In the Notion demo it queried Notion and Paper in parallel (`notion-search` and `get_basic_info`), fetched the page, checked fonts, wrote a brief and built a "Core Values" page whose sections it named Header, Core Values Section, Tactics Section, Reading Section and Footer ► [Notion data 1:24](#), ► [Notion data 3:05](#). Stephen notes it works with any MCP data source or a web search ► [Notion data 0:04](#), and for data visualization ► [Notion data 3:05](#). Other examples: copy from a private Slack canvas with images matched by layer name ► [Remote Control 1:02](#); and, in an animated mock, live playlist data fetched from an API and written one row per `write_html` call before deriving three 390×844 mobile screens ► [Connected canvas 0:06](#), ► [Connected canvas 0:10](#).

Comments as the agent's to-do list

Pin comments on the exact layers, then send one line: you left comments on this frame, check them and implement them ► [Comments for agents 0:08](#). In the short, a comment on a card asked for another card; the agent added an "Ops Agent" card in the same style and, per the narration, handled the other comments in the same pass ► [Comments for agents 0:16](#). The short does not show whether the threads get resolved, so ask for it. Your MCP can list and read threads and change a thread's status, but it has no tool to create a comment ([Your setup](#)): you write the comments, the agent works through them.

Several files at once (tabs)

1. Open the files as tabs. Each agent prompt names its file ► [Tabs and agents 0:42](#).
2. Instead of the name, paste the file's link (Copy link, ⌘L) ► [Tabs and agents 1:04](#). The agent then loads the guide and calls `open_file` before anything else ► [Tabs and agents 1:24](#).

- 3. "The active Paper file" means the tab in front when the agent starts. Stay on it until the agent is working ► [Tabs and agents 1:32](#).
- 4. An agent can read one page and write another in the same file: it read the light and dark tokens on a Tokens page and applied them on the Design page ► [Tabs and agents 1:55](#).
- 5. Tab menu (right-click): Reload, Copy Link ⌘L, Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right, Close All Tabs; double-click renames ► [Tabs and agents 2:20](#).

Designing from your phone (Remote Control)

- 1. In a terminal Claude Code session (not the Claude desktop app, as of July 2026), type `/remote-control` or `/rc` ► [Remote Control 0:11](#). The Claude desktop app now has a Remote Control option for its own sessions, so check there before opening a terminal.
- 2. A notification confirms the session can be controlled from the phone ► [Remote Control 0:29](#).
- 3. Keep the Mac awake: System Settings > Lock Screen, set both "Turn display off ... when inactive" options to "For 3 hours" ► [Remote Control 0:38](#).
- 4. Claude mobile app > Code tab > Sessions: open the session marked Connected ► [Remote Control 0:46](#). The whole desktop conversation is there, with model and permission-mode chips and a dictation mic ► [Remote Control 0:54](#).
- 5. Prompt as usual; review through the screenshots the agent returns in the chat ► [Remote Control 1:48](#).
- 6. Example follow-up: a dark mode on a duplicated frame, built from the codebase's CSS variables instead of invented colors ► [Remote Control 2:03](#). The result kept the light frame untouched and matched the code colors ► [Remote Control 2:18](#).

There is no Paper mobile app; the phone only drives the desktop session, which behaves like any agent session at the desk ► [Remote Control 2:23](#).

Choosing a model

Date	Test	What happened
June 2026	Fable 5 vs Opus 4.8, same prompt, one expense-entry screen, both at High effort, same file	Opus 4.8 finished clearly first despite starting later; the presenter calls Fable 5 slower and significantly more expensive, possibly stronger on long independent work ► Fable 5 vs Opus 4.8 2:35 . Fable 5 described its result in more detail; Opus 4.8 ended by proposing a next variant ► Fable 5 vs Opus 4.8 3:48 . No winner was declared.
July 2026	Remote session	Opus 5 with 1M context on Claude Max ► Remote Control 0:18
September 2026	Opus 5.5 at "Extra" effort, as many good meditation-app home screens as possible in five minutes	Six named directions on 402×874 artboards, all finished in just under six minutes; 6 of 18 tool calls failed in the parallel phase ► Opus 5.5 in 5 min 0:28 , ► Opus 5.5 in 5 min 0:36

The June model menu listed Fable 5, Opus 4.8, Sonnet 4.6, Haiku 4.5, Opus 4.7 (Legacy) and Opus 4.6 (Legacy), with a fast-mode toggle ► [Fable 5 vs Opus 4.8 0:17](#). Slightly more commenters preferred the Opus screen than the Fable one, and several called a one-screen test too narrow (► [Fable 5 vs Opus 4.8, comments](#)). For you, on Opus only: these tests suggest the current Opus handles single screens and multi-variant runs well and fast, so extra effort pays off more in constraints and review than in a pricier model.

Advanced techniques

- **Breadth-first variant builds.** Create every artboard, then build each layer across all of them (headers everywhere, then heroes everywhere)
▶ [Opus 5.5 in 5 min 0:28](#). All variants reach the same finish at the same time, which makes judging fair.
- **Duplicate, then transform.** Duplicate the board first, rename the copy, transform only the copy ▶ [Tabs and agents 1:18](#), ▶ [Remote Control 2:03](#). A viewer notes the manual duplicate is Option-drag on Mac (▶ [Tabs and agents, comments](#)).
- **Code as the source of truth for color.** Ask for the codebase's CSS variables, not invented values ▶ [Remote Control 2:03](#). Paper's email adds that one prompt can turn your CSS variables into Paper tokens ▶ [Remote Control 1:48](#), and that the agent can update the tokens after code changes, or the other way round ▶ [Remote Control 2:24](#).
- **Propagate a manual fix.** Edit one screen by hand, then ask the agent to apply your changes to the rest of the flow ▶ [Teams with agents 0:28](#). Start with a re-read, since the agent does not see your edits otherwise
▶ [Remote Control 1:48](#).
- **References first.** Put references on the canvas and ask the agent to list the best practices it sees before designing ▶ [Teams with agents 0:24](#); or point it at a frame you pasted from Figma ▶ [Teams with agents 0:26](#).
- **Verify color searches.** While recoloring, an agent found that `find_nodes` matches substrings (a search for `#000000` hit zero-valued properties) and that SVG icon strokes were missing from its color searches; it then read the sidebar and top-bar subtrees so icons would not vanish on dark
▶ [Tabs and agents 2:12](#). Make that check explicit in recolor prompts.
- **Parallel reads at the start.** Fetch the data source and the Paper file info in the same step, then plan ▶ [Notion data 1:24](#).
- **Build once, duplicate the rest.** Fable 5 built the first keypad row, then duplicated it ▶ [Fable 5 vs Opus 4.8 3:09](#); the mock agent wrote one table row per call ▶ [Connected canvas 0:06](#).
- **A/B harness.** Two sessions, same prompt, artboard titles that carry the variant name, one shared file ▶ [Fable 5 vs Opus 4.8 0:48](#). Use it to compare prompts or effort levels, not only models.
- **Layer clean-up as boilerplate.** Renaming layers into a semantic tree is one of the jobs the Paper team hands to agents ▶ [Teams with agents 0:46](#).

Pitfalls and limits

Symptom	Cause	Fix	Source
Tools fail or "not connected"	Paper Desktop closed, or no design file open	Open Desktop on the target file before the session; check <code>/mcp</code>	▶ Cursor setup 1:08
Paper tools missing right after setup	Server added inside a running session	Restart the agent, then retry	▶ Claude Code setup 1:52
Agent works in the wrong file	"Active file" resolved at start, or no file named	Paste the file link (<code>⌘L</code>) into the prompt	▶ Tabs and agents 1:32
Invented palette and fonts	No brief or system given	Put palette, fonts and grid in the prompt	▶ Claude Code setup 2:22

Symptom	Cause	Fix	Source
A variant lost a feature	Regeneration drops parts	Compare each variant to the original's feature list	► Design directions 3:36
Failed tool calls in parallel builds	Many writes at once (cause not shown)	Ask which artboards are incomplete; finish them one at a time	► Opus 5.5 in 5 min 0:28
Recolor misses icons or hits wrong nodes	<code>find_nodes</code> substring matches; SVG strokes	Confirm hits with <code>get_computed_styles</code> ; read SVG subtrees	► Tabs and agents 2:12
Agent overwrites your hand edits	It does not track manual changes	Re-read the board before each new pass	► Remote Control 1:48
Screenshot times out	Whole canvas or a very large artboard (viewer report)	Ask for a specific frame; the Paper team says that limits the capture	► Table 5 vs Opus 4.8, comments
Agent cannot read frames pasted from Figma	Viewer report, no fix given	Check those frames render on the canvas first	► Claude Code setup, comments
Remote session unavailable or dropped	Started from the Claude desktop app (terminal only, July 2026); Mac asleep	Start <code>/rc</code> in a terminal session; set the Lock Screen timers	► Remote Control 0:11 , ► Remote Control 0:38

From your own sessions (not shown in the videos, keep applying): pause 250 to 700 ms between rapid writes to avoid "file is currently disconnected"; `write_html` in replace mode drops absolute positioning of children; styled inline spans become one text frame per word; glass must be faked because backdrop-filter bleeds; rotation pivots on the top-left corner.

Apply it to your files

1. **Comments pass on Water Dashboard.** Pin your review notes, in French, on the exact KPI cards, table and chart labels of the 01 Dashboard board. Run recipe 3. Check that each thread is resolved, that nothing outside the commented layers moved, and that no red or orange crept in.
2. **Six directions for Ballottage.** Select the main prediction screen. Run recipe 4 on a new page "03 Explorations": six named directions, built breadth-first, constrained to your figurines (sitting on a surface), tasteful liquid glass and your fonts. Then run recipe 5 to combine the two best by name. Move the losers to the archive page.
3. **Dark variant of Altitude Sports from code.** Run recipe 6 on the main dashboard board, pointing at the repo's Tailwind v4 theme file. If the repo has no dark variables yet, define them in code first so code stays the source of truth. Check every SVG stroke and chart color on the copy, and keep the light board untouched.
4. **Two files in parallel with tabs.** Open the budget tool and the Express Entry dashboard as tabs. Start one agent per file with recipe 9 (file link in the prompt), each filling a table with real data through recipe 7 (the budget lines of the budget tool; Express Entry draw data). Keep your write pauses in both sessions.
5. **Propagate one fix across your chart figures.** Fix one FT-style chart figure by hand (axis labels, source line, gridline weight). Run recipe 8 to apply the same changes to its siblings, one figure per write, with screenshots every three.
6. **Remote review of Poseidon.** Before leaving the desk: open Paper on the Poseidon file, start `claude` in a terminal in its project folder, run `/rc` , set the Lock Screen timers, and use recipe 10. From the phone, review the returned screenshots and send short fixes.

Prompt recipes

Paste into Claude Code with the Paper MCP. Replace the angle brackets.
French works as well as English for the notes you add.

1. Pre-flight

Load the Paper guide if you haven't this session. Are we connected to the Paper MCP? Without changing anything, tell me: the open files, the file and page I'm looking at, the artboards on that page with sizes, the node count and the fonts in use.

2. Scoped edit inside my system

In Paper, file "<file>", page "<01 Dashboard>". First re-read the board "<board>" (tree summary and a screenshot); I may have edited it by hand. Then make only these changes: <list>. Edit existing nodes (update_styles, set_text_content, duplicate_nodes); do not regenerate the board. Keep the <4 x 352 grid, 24 px gutters / 12 x 64 grid, 32 px gutters>, the 8 px rhythm, white surfaces on a neutral light ground, <cobalt> accent, my fonts (<Inter Tight / Inter>). No loose text, no red or orange. One visual group per write, about half a second between writes, a screenshot after each group, finish_working_on_nodes at the end.

3. Work through my comments

I left comments on this board: <paste Copy link>. List every open comment thread on it. Handle them one by one as small edits to the existing nodes, screenshot the affected section after each group, and mark each thread resolved once done. End with a table: comment, what you changed, resolved yes/no, anything you could not do and why.

4. Named directions, breadth-first

In Paper, file "<Ballottage>": create a page "03 Explorations" and work only there. Explore 6 directions for the screen I have selected; start each from a duplicate of it. Name them "Explore / 01 <direction>" to "Explore / 06 <direction>". Keep my fonts (), 3D figurines that sit on a surface, liquid glass used sparingly. Vary layout, hierarchy and color only. Build breadth-first: all headers, then all heroes, then the rest. If a tool call fails, tell me which artboard is incomplete and finish it on its own. Screenshot each and give one line per direction on how it differs from the original.

5. Combine by name

Make "Explore / 07 Combined" next to the others: the navigation from 02, the card layout from 04, the palette of the original. Show me the plan with hex codes first and wait for my OK before building.

6. Dark variant from code

In Paper, file "<Altitude Sports>", page "<01 Dashboard>": duplicate the board "<board>" and name the copy "<board> Dark". Do not touch the light board. Read the dark theme variables in <path to the Tailwind v4 theme / globals.css> and apply those exact values to the copy: backgrounds, surfaces, borders, text, chart colors and SVG icon strokes. Do not invent colors. find_nodes matches substrings, so confirm every color hit with get_computed_styles before changing it. Screenshot the copy section by section.

7. Real data into a table

In Paper, file "<Express Entry dashboard>": fill the table "<table>" with real data from <CSV path / repo file / URL>. No invented numbers. Keep the existing row component: duplicate the first row and change its text with set_text_content, one row per call. If a value is missing, leave the cell empty and list it. Screenshot the table at the end.

8. Propagate a manual fix

I fixed "<figure A>" by hand. Re-read it with get_jsx and compare it with "<figure B>", which I did not touch. List the differences, then apply the same differences to <figures C to H>, one figure per write with a short pause, a screenshot every three figures. Edit the existing nodes; do not rebuild them.

9. Pin an agent to one file

Work only in this Paper file: <paste Copy link>. Open it with the MCP even if another tab is in front, and do not act on any other open file. Task: <task>.

10. Remote session

I'm away from the desk and on my phone. After each change, send a screenshot of only the frame you changed. Ask before deleting or moving anything to another page. Keep each reply short.

11. A/B harness

In Paper, file "<file>", page "<page>": build <component spec> as a new artboard titled "A / <label>". At the end, list your design decisions and one variant you would try next.

Run the same prompt in a second session with "B / " and a different effort level or wording.

12. Layer clean-up before porting to code

On the board "<board>", rename layers to semantic names (for example Header, KPI Row, KPI Card / Revenue, Chart / Weekly Demand, Table / SKUs) without changing any style or structure. Use rename_nodes only, then show me the resulting tree.

Related videos in other chapters

[Sync with the codebase](#), [CSS variables to tokens](#) and its [short version](#), [MCP-generated tokens](#), [iterating on a Snapshot capture](#), [Conductor with parallel agents](#), [pushing designs back to code](#), [code components onto the canvas](#), [the August release](#) (tabs and comments) and [the September release](#) (one-click agent setup, image generation through the MCP).

Design to code, and back

This chapter covers the loop between your repositories and the Paper canvas: bringing tokens, components and live pages into Paper, designing with them, and sending finished frames back to code. It draws on seven official videos published between March and September 2026, so every feature is described as of its video's date. Where a statement comes from a viewer comment rather than from Paper, it says so.

The mental model

The canvas is already HTML and CSS. Paper renders designs with HTML and CSS, so a frame on the canvas is effectively code already, and an agent building it into your app has no translation layer to cross. Harcourt sums up the payoff as faster, more accurate and cheaper in tokens ► [Push to code 2:47](#). The same is true in the other direction: when an agent designs in Paper it writes real HTML and CSS through `write_html` ► [Sync codebase 2:45](#) ► [Conductor 4:26](#), which is why Paper calls this the agent's native language ► [Code components 4:28](#). In both demos that HTML uses inline styles and names each layer through a `layer-name` attribute ► [Sync codebase 2:32](#) ► [Conductor 5:04](#), so the names you see in the layer list are the ones the agent chose.

One agent session sees both sides. Paper's answer to "how do I get an existing repo onto the canvas" is to start the agent inside the repo with the Paper MCP connected and ask it to scan the styles and recreate components or screens (Paper team reply under ► [Sync codebase](#)). Every video in this chapter uses that single bridge: the agent reads files on one side and Paper nodes on the other.

Pieces that come from code go back as code. When a design is built from components that were pulled out of the codebase, the agent recognises them on the way back and reuses them, so behaviour comes for free: cards that already link to playlist pages keep doing so ► [Code components 5:26](#), and a reused search field filters results without new logic ► [Push to code 2:31](#). The Paper team confirms the search bar was recognised because it had been pulled from the codebase first (reply under ► [Push to code](#)).

Statics on the canvas, motion in code. Paper's own website was designed in Paper, shaders included, and its animations were prompted on top of the Paper file when an agent turned it into a Next.js codebase, about six months before the August 2026 clip ► [Animations after 0:18](#).

Ways into Paper

Route	What you get	Tokens	Layer names	Best for
Agent rebuilds from the repo	Component sheets or pages built from source, with spec notes ► Code components 0:59	Imported and bound automatically in the demo ► Code components 1:16	Semantic (Cover card, Track row)	Design systems, anything you will push back
Agent lays out a token file	A token board with variable names, values and usage notes ► Sync codebase 3:12	Values shown on the board	Named by the agent	Auditing and retuning colours

Videos in this chapter

7 videos, 35 minutes

How to sync designs with your codebase using Paper MCP, Claude Code and Cursor

5 Mar 2026, 7:19

Paper and Conductor: design to code with parallel AI agents

19 Aug 2026, 13:53

Design first, prompt the animations after

24 Aug 2026, 0:50

Push designs in Paper back to code

2 Sep 2026, 4:16

From design to code in Paper

3 Sep 2026, 0:14

How to bring your components from code onto the Paper canvas

18 Sep 2026, 6:09

How the Skydive team ships with their agents in Paper

21 Sep 2026, 2:09

Route	What you get	Tokens	Layer names	Best for
Paper Snapshot (Chrome extension)	Editable layers copied from the live site, no agent needed	None	Generic "Frame", raw CSS kept under Other styles	Fast references, context around a new section
	▶ Code components 1:33		▶ Code components 3:30	

Ways out of Paper

Route	Scope	How	Source
Agent builds a named frame	A page or section, with behaviour	Prompt with frame name, target file, behaviour, reuse rule	▶ Push to code 1:04
Copy as Tailwind / Copy as React CSS	One layer, ideally a component	Right-click, Copy as...	▶ Push to code 3:18
Developer joins the Paper team	Anything	They run the same process	▶ Push to code 3:46

Setup

These are the steps shown in the March 2026 video. By September 2026 the editor's right panel has an MCP section with a "Connect your agent" button [▶ Code components 0:40](#), and Paper's recommended setup moved on later in 2026 (outside these videos: see [Connecting an agent](#)). Your own connection already works, so read this mainly for Cursor or Codex, and for the habit in step 1.

- Open Paper Desktop and the target file first. The docs state that opening a file starts the MCP server in the background [▶ Sync codebase 4:40](#).
- Claude Code: in a normal terminal (not inside a running Claude session) run `claude mcp add --scope user --transport http paper http://127.0.0.1:29979/mcp` [▶ Sync codebase 0:39](#). It writes the server to the user config in `~/.claude.json` [▶ Sync codebase 0:54](#).
- Start Claude and type `/mcp`; Paper should be listed as connected [▶ Sync codebase 1:04](#).
- On the first call of each tool, choose "Yes, and don't ask again for paper - [tool] commands in [folder]". Stephen recommends it for read tools and also for write tools [▶ Sync codebase 1:27](#).
- Cursor: on the docs page, "Connect to Cursor", button "Add to Cursor", then Install in the "Install MCP Server?" dialog (Name `paper`, Type `streamableHttp`, same URL) [▶ Sync codebase 4:40](#). It lives under Cursor Settings > Tools & MCP; if the connection misbehaves, toggle the server off and on [▶ Sync codebase 5:03](#).
- Codex: the docs say Settings > MCP Servers, add a custom server on the "Streamable HTTP" tab with name `paper` and the same URL [▶ Sync codebase 1:04](#).
- Any agent: ask it which MCPs it has; Paper should appear with read, write, tokens, screenshots, comments and JSX/CSS export [▶ Conductor 1:04](#). Through a plugin, tool names look like `mcp__plugin_paper-desktop_paper__write_html` [▶ Conductor 4:58](#).

Procedure 1: tokens from code to canvas and back

1. In the repo, ask the agent to read your colour file and arrange every colour on the Paper canvas ► [Sync codebase 1:11](#).
2. Expect `get_basic_info`, a file read, `get_font_family_info` (checks fonts on your machine and Google Fonts) and then `write_html` into a new artboard ► [Sync codebase 2:00](#).
3. Check the board: Stephen's lists each colour's name, CSS variable, oklch value and the usage note taken from code comments, grouped Chromatic, Neutrals, Surfaces ► [Sync codebase 3:12](#).
4. The agent screenshots its own work, finds problems (here clipped bottom padding) and fixes them with `update_styles` ► [Sync codebase 3:45](#).
5. To retune, select the swatches and edit the values in the right panel's **Selection colors** list, which shows each colour as L%, chroma, hue° and alpha with a usage count ► [Sync codebase 5:13](#).
6. With the swatches still selected, ask the agent to bring the colours you modified in your selection back into the CSS file ► [Sync codebase 5:44](#). It reads Get Selection, Get Computed Styles and Get Node Info ► [Sync codebase 6:04](#).
7. Review the diff. The agent changed only the selected colours and reported two other variables sharing the old hue without touching them ► [Sync codebase 6:48](#).

Procedure 2: components from code onto the canvas

Native components on the canvas were "coming soon" as of September 2026 ► [Code components 0:00](#); the Paper team replied "Almost here!" under that video. Until then, a whole existing page can come in with one short prompt that names the source file and any extra context to include, as Harcourt did for the Learn tab and its sidebar ► [Push to code 0:24](#). For a component library:

1. Open an empty Paper file with the agent connected. Prompt: pull the code components relevant to the flow you plan (name a few) and build them in this file before designing ► [Code components 0:28](#).
2. Inspect the sheet. The agent produced numbered sections (01 Cover card, 02 Person, 03 Track row, 04 Section header), dark and light versions, semantic layer names and a one-line spec per component ► [Code components 0:59](#).
3. Inspect bindings. Select a text layer: fields show token chips such as `text` and `sans` instead of raw values ► [Code components 1:28](#).
4. Compose: ask for the new section built from those components, filled with real data from the repo's API ► [Code components 2:16](#). While it works, Paper shows the agent's avatar and which frames it is reading ► [Code components 2:42](#).
5. Variants: ask it to duplicate the frame and try the alternative ► [Code components 3:55](#). The original stays untouched and the copy is named after the change ("For you section · Grid 112") ► [Code components 4:22](#).

Procedure 3: capture the live site with Paper Snapshot

1. Enable the Paper Snapshot extension in Chrome; the first time on a site, click it in the Extensions menu to grant access ► [Code components 1:51](#).
2. Hover an element and click to copy it, or press Command+Return to take the whole page ► [Code components 3:15](#). Full pages take longer; a "Capturing selection..." toast shows progress ► [Conductor 1:34](#).
3. Paste into Paper with Command+V ► [Conductor 1:45](#). The result is editable layers ► [Code components 2:03](#).
4. For a dark variant, switch the macOS appearance to dark before capturing ► [Conductor 1:53](#).
5. To drop a captured or generated section into a captured page's layout, paste it inside the container and untick **Absolute position** so it flows in the flex column ► [Code components 3:30](#).
6. Clean up: snapshot layers arrive as "Frame" with raw CSS under Other styles ► [Code components 3:30](#). The Paper team suggests an agent pass after pasting to rename layers and apply tokens (reply under ► [Push to code](#)).

Procedure 4: build a frame into code with an agent

1. Finish the design as named frames, and draw behaviour as extra artboards: Harcourt drew a separate scrolled state next to the main redesign ► [Push to code 0:40](#).
2. Where possible, design with components that came from the codebase (her Search field) ► [Push to code 0:36](#).
3. Write the prompt with four parts: the frame name in the open Paper file, the file to update, the behaviour to keep and where it already exists in the app, and an instruction to reuse existing code components instead of creating new ones ► [Push to code 1:04](#). When there are several similar frames, copy the exact frame name from Paper into the prompt ► [Conductor 11:29](#).
4. Raise the effort for a real build: she switched Claude Opus 5 from Low to High and turned on Fast mode ► [Push to code 1:23](#).
5. Answer the agent's questions; she frames this as the designer making the decisions an engineer used to make alone ► [Push to code 1:37](#).
6. Read the summary before keeping changes: files changed, which existing patterns and components were reused, and how behaviour was implemented ► [Push to code 2:09](#).
7. Verify against the canvas. The Conductor agent read the frame with `get_screenshot` and `get_jsx` ► [Conductor 11:52](#), served the page locally, and compared a headless screenshot with the Paper frame ► [Conductor 13:10](#). The Kraft agent checked its Explore page with browser actions before reporting ► [Code components 5:16](#). Codex, in another demo, read `get_selection`, `get_screenshot`, `get_tree_summary` and `get_computed_styles` before writing a component ► [Animations after 0:20](#).

Procedure 5: copy one layer as code

1. Select the layer; hold ⌘ while clicking to reach a nested layer directly
▶ [Push to code 3:05](#).
2. Right-click, **Copy as...**, then **Copy as Tailwind** (⌘T) or **Copy as React CSS** (⌘R); **Copy as PNG** is ⌥ ⌘C ▶ [Push to code 3:18](#).
3. Prefer component-sized layers: she copied one sidebar tab because it is itself a code component, though a whole frame also works ▶ [Push to code 3:19](#).

Several agents at once

- Several agents can write to the same Paper file simultaneously; a second agent restyled frames while the first was still creating them ▶ [Conductor 7:15](#)
▶ [Conductor 8:12](#).
- The Paper MCP can create a new file (it opens as a new tab) and name it, and an agent can rename files ▶ [Conductor 9:04](#) ▶ [Conductor 9:30](#).
- With several files open, say in every prompt which file to work in
▶ [Conductor 9:58](#).
- In Conductor, a workspace is an isolated copy of the codebase, a tab shares the same copy; for Paper-only edits a tab is enough ▶ [Conductor 2:26](#)
▶ [Conductor 7:49](#). Enter steers a running agent, Command+Enter queues
▶ [Conductor 10:08](#), Control+O expands all tool calls ▶ [Conductor 5:12](#).
- Roles can be split between agents: Skydive's design agent drafts in Paper, then shares the design with a build agent that publishes it to Webflow, while designers supervise ▶ [Skydive 1:08](#) ▶ [Skydive 1:17](#).

Feedback loops

- **Comments in Paper:** enable comments, click on the canvas, type in "Leave a comment", Send, then tell the agent you left comments in Paper
▶ [Conductor 12:51](#).
- **Comments on the code:** in Conductor's diff, select lines and use "Add a comment for the AI" ▶ [Conductor 12:26](#).
- **Steer mid-run** instead of starting another agent when you spot an error early ▶ [Conductor 10:43](#).
- **Manual pass:** Skydive's designers fix the agent's draft by hand before the build agent takes it ▶ [Skydive 1:00](#); Harcourt switches a font herself in the Fonts popover ▶ [Conductor 6:58](#).

Advanced techniques

- **Ask for N directions with a reason each.** Asking for three iterations, one of them inspired by the changelog, gave three distinct layouts ▶ [Conductor 2:40](#); stating the count is deliberate ▶ [Conductor 3:16](#). Treat the first result as a start
▶ [Conductor 5:49](#).
- **Feed real content.** From the repo's API ▶ [Code components 2:16](#), from captured pages ▶ [Conductor 6:12](#), or from Notion, Slack or Jira through their MCPs

- ▶ [Conductor 6:21](#).
- **Train the agent on the system first.** Skydive gives its design agent the Paper link to the design system before asking for anything (Copy link ⌘L)
▶ [Skydive 0:30](#).
- **Rebuild in sections.** One prompt worked for a simple page, but she usually goes in smaller chunks ▶ [Push to code 0:29](#), and the Paper team recommends a section at a time for faithful rebuilds (reply under ▶ [Push to code](#)).
- **Prompt the motion in code.** Agu designed the static site in Paper and prompted animations and interactive extras during the Next.js build
▶ [Animations after 0:18](#) ▶ [Animations after 0:35](#).
- **Match model and effort to the job.** Stephen notes Opus is slower but better
▶ [Sync codebase 2:22](#). Harcourt raised Opus 5 to High for the page build
▶ [Push to code 1:23](#); in the Conductor session, High went to the three design iterations ▶ [Conductor 3:54](#) and Medium to the restyle and the static port
▶ [Conductor 7:24](#) ▶ [Conductor 11:52](#).
- **Name interaction states.** A viewer reports that sibling layers named "default" and "hover" inside a layer named after the component let Cursor infer the states through MCP; Paper did not confirm it (comment under ▶ [Sync codebase](#)).
- **Native apps.** Paper's team says it tried this loop on a native iOS app and it works well, with no video yet (reply under ▶ [Push to code](#)).

Pitfalls and limits

Problem	What happens	Fix	Source
Agent cannot reach Paper	No server running	Open the file in Paper Desktop before the session	▶ Sync codebase 4:40
Approval prompt on every call	One prompt per tool	"Yes, and don't ask again" per folder	▶ Sync codebase 1:27
Flaky connection in Cursor	MCP still has rough edges	Toggle the server off and on	▶ Sync codebase 5:03
Related tokens not updated	Write-back is limited to the selection	Ask explicitly for derived variables	▶ Sync codebase 6:48
Snapshot layers are anonymous	"Frame" names, raw CSS, no tokens	Agent rename and token pass	▶ Code components 3:30
Pasted section floats	Arrives with Absolute position	Untick Absolute position	▶ Code components 3:30
Wrong typeface	Agent defaulted to Matter	Name the font in the brief	▶ Conductor 6:58
Edits land in the wrong file	Several tabs open	Name the file every time	▶ Conductor 9:58
Plain HTML instead of a component	The target repo was empty	Port into a real project	▶ Conductor 11:52
Smaller variant clips text	Agent reported the tradeoff	Read the agent's notes before choosing	▶ Code components 3:55
No native components yet	Components rebuilt as frames	Agent rebuild or Snapshot until they ship	▶ Code components 0:00

A viewer claims Snapshot does not count toward a Paper MCP quota; Paper did not confirm it (comment under ▶ [Push to code](#)). In another reply under the same video, Paper says WebMCP is live and a remote MCP is "in the works" (September 2026).

Apply it to your files

1. **Altitude token round trip.** From the Altitude repo, have Claude lay out the color variables of its stylesheet on a "Tokens · from code" artboard in the Altitude Sports file, one group per write, with variable, value and usage note per swatch (Procedure 1). Then select the cobalt scale, shift it in Selection colors, and ask Claude to write back only the selected variables and list the dependent ones. Done when the diff touches exactly the selected tokens.
2. **Altitude Sports component sheet.** In the production repo, ask Claude to rebuild the dashboard's real KPI card, chart card, filter bar and table row on page "02 Grid and components", bound to tokens, with a spec line each (Procedure 2). Then compose a new section from them on a duplicate of "01 Dashboard" with real values from the repo, and push it back naming the frame and the target route (Procedure 4). Done when the coded section reuses the existing components.
3. **Water Dashboard from production.** Snapshot the live Water dashboard into a frame, run the rename-and-tokens pass, then redesign on a duplicate (Procedure 3 and 2). Leave your blunt notes as Paper comments instead of chat, and ask Claude to apply and resolve them one by one. Done when all threads are resolved and the original frame sits on the archive page.
4. **Budget tool, behaviour as artboards.** For the budget tool, draw the sticky or scrolled state of the side rail as its own artboard next to the main board, then port both with one prompt that names both frames and the component library to reuse (Procedure 4). Use Copy as Tailwind `\T` on one KPI card first to compare its classes with your tokens (Procedure 5).
5. **Poseidon panel, three directions.** Ask for three iterations of the network map in three named frames, each with a stated inspiration and the font fixed to Inter Tight with IBM Plex Mono labels; run a second agent on the same file for a restyle while you review (Several agents at once). Port the winner and require a screenshot comparison against the Paper frame.
6. **Ballottage motion after.** Freeze the ring and reveal screens in Paper as statics, then prompt the transitions in the app's code instead of attempting transforms on the canvas ► [Animations after 0:18](#).

Prompt recipes

These follow the patterns shown in the videos and add your own rules (incremental writes, a screenshot after each group, archive instead of delete, no exports to ~/Downloads). Write them in French if you prefer; the structure is what matters.

Outside these videos: your Claude Code also lists two skills from the installed Paper plugin, `paper-desktop:code-to-design` and `paper-desktop:design-to-code`, which package the two directions of this chapter. [The plugin's two skills](#) describes them and notes that updating the plugin removes them.

1. Tokens from code to a Paper board

Read <your CSS variables file: the @theme block of a Tailwind v4 stylesheet, or the :root block of a plain one>. In the Paper file "Altitude Sports", page "<components page>", create an artboard "Tokens · from code" (1440 wide, 12 x 64 grid, 32 gutters, 8 px rhythm). For each colour token draw a swatch with: display name, CSS variable, exact value, and the usage note from the code comment. Group as in the file. One write_html per group, get_screenshot after each, fix clipping with update_styles, then finish_working_on_nodes. Do not touch other boards.

2. Selected colours back to code

I edited colours in my current Paper selection (Water Dashboard, blue scale). Use get_selection, get_computed_styles and get_node_info to read the new values, map each one to its variable in <your CSS variables file>, and update only those variables. Show me the diff. List any other variables that still use the old values, without changing them.

3. Real components onto the canvas

In this repo, find the components used on the Altitude Sports dashboard (KPI card, chart card, filter bar, table row). In the Paper file "Altitude Sports", page "02 Grid and components", build a component sheet: numbered sections, light versions only, layers named after the components, a one-line spec under each (sizes, radius, type scale, token names). Import the repo's design tokens into Paper and bind them. One component per write_html, screenshot after each.

4. Snapshot cleanup

In the Paper file "Water Dashboard", the frame "Snapshot · live" is a paste from Paper Snapshot. Rename the generic "Frame" layers semantically (Header, KPI row, KPI card, Chart, Table...) and bind colours and type to the existing tokens. Do not change any visual value. Report anything that has no matching token.

5. Variant on a duplicate

Duplicate the frame "01 Dashboard · KPI row" and show it with the compact KPI card instead of the large one. Keep the original untouched, name the copy "01 Dashboard · KPI row · compact", keep it on the 4 x 352 grid with 24 gutters. Tell me what gets clipped or lost at the smaller size.

6. Three directions for a panel

In the Paper file "Poseidon", create three iterations of the route network section, each on its own artboard named "Network · A", "Network · B", "Network · C" (1536 wide). A: single trunk line. B: full network diagram. C: inspired by the existing "02 Grid and components" page. Fonts: Inter Tight for display and body, IBM Plex Mono for labels. Neutral light ground, white surfaces, cobalt accent, no loose captions. Screenshot each when done.

7. Frame to code

Build the frame "01 Dashboard · Final" from the open Paper file into <target page file>. Reuse existing components from <your component folders>; do not create new ones when one exists. Take exact values from get_jsx and get_computed_styles, never from screenshots, and map them to the tokens in <your CSS variables file>; list any value without a token. The sticky header behaviour is drawn in "01 Dashboard · Scrolled"; match the scroll pattern already used elsewhere in the app. Ask me your questions before writing code. When done, open the page in the Claude app's Browser pane at 1440 width, screenshot it, compare it with get_screenshot of the frame, and list the differences.

8. Comments as instructions

I left comments on the Paper file "Budget tool". Read all open comment threads, apply each one as a small edit on the board it points to, screenshot after each edit, then resolve the thread. Do not redesign anything that has no comment. Summarise what changed per thread.

Shortcuts and settings reference

Action	Shortcut or path (as displayed)	Source
Copy as PNG / Tailwind / React CSS	⌘C / ⌘T / ⌘R (right-click, Copy as...)	► Push to code 3:18
Select nested layer	Hold ⌘ and click	► Push to code 3:05
Select parent / children	Escape / Enter	► Push to code 3:18
Copy link to layer or file	⌘L	► Push to code 3:18
Copy styles / Paste styles	⌘C / ⌘V	► Push to code 3:18
Paste on top / Paste to replace	⌘V / ⌘R	► Push to code 3:18
Frame selection / Remove flex / Ungroup	⌘F / ⌘A / ⌘G	► Push to code 3:18
Show / hide, Lock / unlock	⌘H, ⌘L	► Push to code 3:18
Wrap in flex / Clip content	⌘A / ⌘C (right panel)	► Push to code 2:12 ► Code components 3:32
Snapshot whole page	Command+Return in Paper Snapshot	► Code components 3:15
File tab menu (right-click a Paper tab)	Reload, Copy Link ⌘L, Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right, Close All Tabs	► Conductor 7:20
Add Paper to Claude Code	<code>claude mcp add --scope user --transport http paper http://127.0.0.1:29979/mcp</code>	► Sync codebase 0:39
Conductor: steer / queue / expand tool calls	Enter / Command+Enter / Control+O	► Conductor 10:08 ► Conductor 5:12

Snapshot, collaboration and releases

This chapter covers eight videos published between May and September 2026: Paper Snapshot (copying a live web page into Paper as editable layers), iterating on a capture with an agent, reference-driven flows steered by comments, handing a frame to another agent (the Resend email workflow), tabs and parallel agents, comments, team roles, and the rest of the August and September changelogs. Each feature is described as of the date of the video that shows it.

Short name	Video	Date
Snapshot	Design using your live site as a starting point with Paper Snapshot	2026-05-02
Snapshot + agent	How to use an AI agent to iterate on a design you captured with Paper Snapshot	2026-05-02
Resend	Using Paper and Resend to turn a design into a ready-to-send email template	2026-06-05
Tabs short	Tabs, now in Paper	2026-08-11
Aug update	New in Paper: tabs, comments, free editors, and more	2026-08-12
Comments short	Comment, reply, and resolve in Paper	2026-08-20
Mobbin	Use Mobbin and Paper to design flows from proven patterns	2026-09-23
Sept update	New in Paper: faster performance, agent reliability, opacity tokens, and more	2026-09-28

The mental model

Paper's canvas is built on HTML and CSS, the same model as a website. Several features in this chapter follow from that. Snapshot rebuilds a live page as layers instead of pasting a picture, because nothing has to be translated

► [Snapshot 0:15](#). An agent extending a captured section edits it like code: it read the JSX, found a grid defined as repeat(2, 1fr) and changed it to three columns

► [Snapshot + agent 1:30](#). A frame can leave Paper as a PNG, Tailwind or React CSS

► [Sept update 3:46](#), or be rebuilt by an agent as email-safe HTML ► [Resend 6:24](#).

The work follows three loops:

1. Bring material onto the canvas: a capture of a live page ► [Snapshot 0:53](#), references fetched by another MCP server ► [Mobbin 0:06](#), or a brief parked beside the design, such as a style guide or a block of copy ► [Resend 1:07](#) ► [Snapshot + agent 0:18](#).

Videos in this chapter

8 videos, 32 minutes

Design using your live site as a starting point with Paper Snapshot

2 May 2026, 2:30

How to use an AI agent to iterate on a design you captured with Paper Snapshot

2 May 2026, 3:30

Using Paper and Resend to turn a design into a ready-to-send email template

5 Jun 2026, 9:15

Tabs, now in Paper

11 Aug 2026, 0:33

New in Paper: tabs, comments, free editors, and more

12 Aug 2026, 9:23

Comment, reply, and resolve in Paper

20 Aug 2026, 0:54

Use Mobbin and Paper to design flows from proven patterns

23 Sep 2026, 0:42

New in Paper: faster performance, agent reliability, opacity tokens, and more

28 Sep 2026, 5:40

- 2. Change it, with an agent through the Paper MCP and with your own hands, keeping the two in sync ► [Resend 4:22](#).
- 3. Hand it off: copy it as code, or paste a Paper link into another agent session ► [Resend 5:18](#).

Collaboration runs on three surfaces. Tabs let several agents work in several files while their presence shows on tabs and frames ► [Aug update 1:31](#). Comments carry instructions to teammates and to agents ► [Aug update 7:37](#). Roles decide who can edit and who pays ► [Aug update 5:43](#).

Paper Snapshot

Install

- 1. Open the Chrome Web Store page "Paper Snapshot" and click "Add to Chrome" ► [Snapshot 0:34](#).
- 2. Pin it: open Extensions, find Paper Snapshot, pin it to the toolbar. These steps are on the "Paper Snapshot quick start guide" page at [paper.design/snapshot-extension](#) ► [Snapshot 0:41](#).

Capture and paste

- 1. Open the live page and click the Paper Snapshot icon in Chrome's toolbar. A hint bar appears at the top of the page ► [Snapshot 0:53](#).
- 2. Hover. A blue bounding box marks the element under the cursor; everything inside it is the selection ► [Snapshot 0:53](#).
- 3. Widen or narrow the target with the arrow keys, then click or press Return ► [Snapshot 1:08](#).
- 4. Paper reports when the capture is complete. Switch to Paper (web or desktop) and press ⌘V on the canvas ► [Snapshot 1:21](#).

Key	Hint bar text
Click or ←	"to capture"
↑ ↓	"to fine-tune target"
Esc	"to cancel"
⌘V (in Paper)	paste the capture

Sources: ► [Resend 0:56](#), ► [Snapshot 1:21](#).

What lands on the canvas

The demo capture arrived as a 1280 × 900 frame called "Frame", with the heading and paragraph as text layers and the cards as nested frames ► [Snapshot 1:30](#). The card row is a real flex frame with a gap of 16, the cards keep their 1 px borders per side (at 18.3% alpha) and a 24 px radius, and "Selection colors" lists every captured color with its alpha ► [Snapshot 1:36](#). Selecting a card and pressing → moved it after its sibling and the row reflowed; double-clicking the heading made its words editable ► [Snapshot 1:42](#) ► [Snapshot 1:54](#).

Two consequences. The layers come in with generic names ("Frame", "Frame"), so rename them before anyone, person or agent, navigates by name ► [Snapshot 1:42](#). And the capture is a fixed-size copy of what the page rendered at that moment, not a live link to the site ► [Snapshot 1:30](#).

Iterating on a capture with an agent

Shape the prompt

Give the agent the context you would give a person: that the work is in Paper, which frame, and where the content lives ► [Snapshot + agent 0:13](#). Harcourt parked the new feature's copy in a small frame beside the capture and pointed the agent at it by its title instead of pasting the text ► [Snapshot + agent 0:36](#). For a reskin, state what must stay fixed as well as what changes: the Resend prompt kept every container and column width and forbade carrying over the original fonts and colors ► [Resend 1:37](#). That prompt was also saved on the canvas as a text layer, next to a "Style Guide" frame holding three swatches named by hex value, a font note and a logo ► [Resend 1:07](#).

What the agent did

Harcourt's run used the Claude desktop app's Code tab with Sonnet 4.6 and the "Auto accept edits" mode ► [Snapshot + agent 0:18](#).

1. Read the canvas: "Get Screenshot", "Get Tree Summary", then the JSX of an existing column ► [Snapshot + agent 1:06](#).
2. Planned at the structure level: switch the grid to three columns and duplicate one column ► [Snapshot + agent 1:30](#).
3. Wrote: "Duplicate Nodes", "Update Styles", then new text and icon in the duplicate ► [Snapshot + agent 1:44](#).
4. Reviewed itself: a "Checkpoint review" of spacing, typography, contrast, alignment, clipping and repetition ► [Snapshot + agent 2:06](#). In the Resend run the list also had "Artboard fit", followed by a summary of every value it changed ► [Resend 3:19](#).

Permissions

Harness	Options shown
Claude desktop app, Code tab	"Deny" (Esc), "Always allow for project (local)" (⌘↵), "Allow once" (↵) ► Snapshot + agent 1:32
Claude Code in a terminal	"Yes", "Yes, and don't ask again for commands in ", "No" ► Resend 5:51

Harcourt chooses always allow so the agent stops asking before every duplicate ► [Snapshot + agent 1:32](#).

Finish by hand, then resync the agent

1. Fill what the agent left out. The new column had no image; the options named were Paper's image generation or uploading a team asset ► [Snapshot + agent 2:31](#).
2. Fix heights after a structural change. Three narrower columns left the media frames too tall; double-clicking a frame's bottom edge switched it to fit height ► [Snapshot + agent 2:48](#).
3. Make your own edits. Harcourt set the footer text to white and the footer band to slate blue ► [Resend 3:35](#).
4. Before the agent works again, ask it to re-read the design and confirm it understands your changes. It took a screenshot, read computed styles and listed exact before and after values ► [Resend 4:22](#).

Reference-driven flows with comments

The Mobbin short describes a loop where comments do the steering

► [Mobbin 0:00](#):

1. An agent pulls reference flows through Mobbin's MCP onto the canvas: one board per product and flow, named like "Ref · Stripe - Add to cart", with numbered, captioned screens ► [Mobbin 0:06](#) ► [Mobbin 0:10](#).
2. Keep only the references that suit the brand ► [Mobbin 0:12](#).
3. Pin comments on the screens you like ► [Mobbin 0:16](#).
4. Prompt with two Paper links, one to the references and one to your design system boards, and ask for five variations. The demo ran Opus 5 at High in the Claude desktop app, and the design system lived on two boards, "01 · Foundations" and "02 · Components" ► [Mobbin 0:18](#) ► [Mobbin 0:22](#).
5. The agent writes each variation as a labelled row ("V1 · Drawer", "V2 · Toast", "V3 · Popover") ► [Mobbin 0:26](#).
6. Pick, comment, repeat a couple of times, then finish by hand ► [Mobbin 0:30](#).

The short does not show how Mobbin's MCP is connected or which tools ran.

Handing a design off

Copy as code or image

Option	Shortcut	Where
Copy as PNG	⇧⌘C	Right-click → "Copy as...", or the copy icon in the Export section
Copy as Tailwind	⇧⌘T	Same two places
Copy as React CSS	⇧⌘R	Same two places
Copy link	⌘L	Right-click, or "Copy link" at the top of the right panel

Sources: ► [Sept update 3:46](#), ► [Sept update 3:52](#).

Hand a frame to another agent

The Resend half of the collab is a pattern that works for any vendor plugin:

1. Copy the frame's link in Paper ► [Resend 5:18](#).
2. In the receiving Claude Code session, paste it with a one-line request ► [Resend 5:18](#). Chris's session ran Opus 4.8 with a 1M context, a different session and model from the one that did the design ► [Resend 6:08](#).
3. The agent first loads Paper's guide (get_guide, topic "paper-mcp-instructions") and reads the frame, then loads the vendor's skill, here "resend:resend" ► [Resend 5:51](#) ► [Resend 6:03](#).
4. It translates the design into the target format: table-based email HTML, with {{{RESET_URL}}} as a variable it added on its own ► [Resend 6:24](#).
5. Review in the target tool. In Resend the template starts as a Draft, can be tested with JSON variables, and goes live when published ► [Resend 7:01](#)

► [Resend 7:57](#).

Keep the agent's two warnings in mind: the logo pointed at a paper.design asset URL that should be hosted elsewhere before production, and the raw HTML could not be edited in Resend's visual editor until converted

► [Resend 6:49](#).

Tabs: several files, several agents

1. Click "+" in the tab bar to open a new or existing file. The first tab is the Dashboard ► [Aug update 0:29](#).
2. Run one agent per task, and name the file in every prompt so agents do not mix up tabs ► [Aug update 1:04](#).
3. For an exact target, select it, press ⌘L and paste the link into the prompt ► [Aug update 1:44](#).
4. Watch progress: the agent's avatar joins the avatar row, the tab shows animated dots, and the frame being edited shows a dotted marker ► [Aug update 1:31](#) ► [Tabs short 0:08](#) ► [Tabs short 0:20](#).

The demo ran two Cursor agents at once: one built color tokens from a swatch frame, the other scaled a frame to 3x, which Harcourt calls a workaround until the scale tool ships ► [Aug update 2:05](#) ► [Aug update 3:34](#). In September, opening and switching tabs became faster ► [Sept update 0:29](#).

The tabs promo adds one useful image of the workflow: a Claude Code panel holding a two-option spec for each dashboard region (navigation, KPI cards, charts, tables, filters) while wireframe cards fill in on the canvas, and later "Dashboard A" and "Dashboard B" side by side ► [Tabs short 0:10](#) ► [Tabs short 0:24](#).

Comments for people and agents

1. Press C or click the toolbar's comment button, then click anywhere and type ► [Comments short 0:06](#) ► [Comments short 0:10](#).
2. Open a pin to reply or add an emoji reaction ► [Comments short 0:16](#) ► [Comments short 0:22](#).
3. Resolve with the check icon; a "Comment resolved" toast offers "Undo" ► [Comments short 0:28](#) ► [Aug update 7:20](#).
4. Review everything in the right panel, which lists comments only while comment mode is on ► [Comments short 0:34](#).

Panel control	Options
"Search"	Text match ► Comments short 0:38
Sort	Newest first, Oldest first, By page ► Comments short 0:42
"Anyone"	Filter by author ► Comments short 0:46
"Any page"	Filter by page ► Comments short 0:46
Check toggle	Resolved or unresolved ► Comments short 0:50

For agents: leave the comments, then tell the agent you left comments on the frame and ask it to implement them. In the demo, a comment asking for one more card in a preview list produced the card, and the agent handled the

other comments as well ► [Aug update 7:37](#). On the MCP side this maps to `list_comment_threads`, `get_comment_thread` and `set_comment_thread_status` (tool names from your toolset; the videos do not show the calls).

Team, roles and files

Role	Can	Cost
Admin	Manage members, billing and team settings	not stated
Editor	Create and edit files	Free and unlimited on the free plan; a paid seat on Pro
Viewer	View files only	Free on every plan

Sources: ► [Aug update 5:43](#), ► [Aug update 6:28](#).

To invite: team settings → "Members" → "Invite members" → email and role → "Send invite". To change a role: the member's "..." menu → "Change role" → "Save role". Moving someone up from Viewer is billed pro rata ► [Aug update 6:09](#) ► [Aug update 6:28](#).

In the dashboard, ⌘-click selects several files and folders for "Copy links", "Move ... items to..." or "Archive" ► [Aug update 4:56](#). Right-click → "Duplicate" makes " (copy)" in the same folder ► [Aug update 5:15](#). Search (⌘F) looks across all folders ► [Sept update 2:48](#). File size limits on Pro were raised in August ► [Aug update 5:33](#).

Everything else that shipped

Tokens in the app

- The left panel's "Theme" tab lists tokens with a search and a "+" button. The August types: Color, Radius, Spacing, Container, Breakpoint, Font family, Font weight, Font size, Line height, Letter spacing ► [Aug update 2:22](#). September adds Opacity ► [Sept update 1:52](#).
- Font pickers have "Fonts", "Favorites" and "Tokens" tabs; starred fonts appear in Favorites ► [Aug update 2:34](#).
- Editing a token updates every use at once; the color picker has "sRGB" and "Display P3" tabs, its "Previous" and "New" swatches let you compare, and closing the editor saves ► [Aug update 2:58](#).
- The "Color Tokens - Swatches" board used in that demo was itself generated by asking an agent for a style guide, according to a Paper team reply (under ► [Aug update 0:00](#)); an agent then turned the board into tokens ► [Aug update 2:05](#).
- An opacity token (60% in the demo) can fill the percentage slot next to a color token, giving "ink" at 60% ► [Sept update 2:05](#).
- To turn a raw fill into a token, click the token icon beside the fill, press "+", then name the new token in the Theme tab ► [Sept update 2:19](#).
- The token picker shows a list, a small grid or a large grid ► [Sept update 2:36](#).

Layout and editing

- Flex wrap (August): turn off "Clip content" (⌘C) on the outer frame to see

- the overflow, select the direct parent, open the flex section's "more properties" menu ("Align to baseline", "Space between", "Wrap") and choose Wrap ► [Aug update 4:00](#). Item order decides where lines break: a long item that does not fit wraps at its own position ► [Aug update 4:16](#).
- Reworked flex panel (September): a wrap toggle beside the direction arrows, space-between next to the gap, separate horizontal and vertical gaps when wrapping, alignment and gap for grid layouts; flex layouts built by agents are now editable in the panel ► [Sept update 3:05](#).
 - Option-drag a copy, rotate it, then press ⌘D: each new duplicate repeats the offset and the rotation ► [Aug update 4:27](#).

Right-click shortcuts worth knowing ► [Sept update 3:46](#):

Command	Shortcut
Select parent / Select children	Escape / Enter
Paste on top / Paste to replace	⇧⌘V / ⇧⌘R
Copy styles / Paste styles	⌘C / ⌘V
Frame selection	⇧F
Remove flex	⌘⇧A
Ungroup	⇧⌘G
Bring to front / Send to back	] / [
Show / hide; Lock / unlock	⇧⌘H; ⇧⌘L

Agent setup and generation

- With nothing selected, the right panel's "MCP" section has a "Connect your agent" button. The dialog covers Cursor, Claude, Claude Code, GitHub Copilot, VS Code, OpenCode, Codex, Codex CLI, Antigravity and "Other agents", each with a one-click install, a manual option and three test prompts ► [Sept update 0:54](#). For GitHub Copilot the demo clicked "Add to GitHub Copilot", restarted Copilot, then switched "Paper" on under Customize → MCP ► [Sept update 1:09](#).
- Agents can generate images through the MCP. Asked for a "variety pack", an agent built one board with four images labelled OpenAI Image 2.5, Nano Banana 2, Flux 2 Pro and Recraft V4.1 ► [Sept update 1:31](#).

Rendering and shaders

- About half the memory use and smoother zoom and pan on large files ► [Sept update 0:29](#). Images and videos now render through HTML in Canvas, which makes exports faster and higher fidelity ► [Sept update 3:55](#).
- Lens Distortion is an image filter: add it, use the eyedropper to pull in the content to distort, then tune it. Presets Default, Fisheye, Duotone, Smoke; parameters Fit, Spread, Bias, Angle, Perspective, Count; works on text layers too ► [Aug update 8:11](#) ► [Aug update 8:33](#).
- Paper Texture, rebuilt: Fiber, Drops, Folds, Wrinkles and Crumples controls, plus paper and shadow colors ► [Sept update 4:06](#).

Coming soon (as of 28 September 2026)

Item	What was said	Source
Remote MCP	Use Paper from cloud agents without the desktop app open	► Sept update 4:32
Components on the canvas	Code components brought onto the canvas cannot yet become Paper components	► Sept update 4:37
Scale tool	Scale several layers at the same ratio	► Sept update 4:49
Rich text editing	Bold, italic or a gradient on part of a text element	► Sept update 4:59
Permissions per file	Choose who sees which file	► Sept update 5:13
Personal drafts	Announced as next in August	► Aug update 8:48

Replies from the Paper team in the comment sections add: light and dark theme support is coming, components are in R&D with no ETA, and remote MCP is on the roadmap (under [► Aug update 0:00](#)); rulers are being built, guides already exist at the bottom of the right panel when a frame is selected, and an existing Codex subscription can be connected (under [► Sept update 0:00](#)). In the same thread a commenter whose handle suggests Paper staff says lock aspect ratio for resizing an existing frame is coming soon and that more speed-ups for very large files shipped after the video (unverified).

Pitfalls and limits

- Snapshot is only shown as a Chrome extension [► Snapshot 0:34](#). A viewer asked for it to report elements it could not copy, which suggests some components may come in incomplete (comment under [► Resend 0:00](#)).
- Captured layers have generic names and a fixed size [► Snapshot 1:42](#).
- After a structural change, fixed heights stay fixed until you set them to fit [► Snapshot + agent 2:48](#).
- Agents rebuild structure but leave missing assets missing [► Snapshot + agent 2:31](#).
- Manual edits the agent has not re-read can be overwritten in the next pass; resync first [► Resend 4:22](#).
- Code produced from a frame can hotlink Paper assets [► Resend 6:49](#).
- With several tabs open, a prompt that does not name the file can land in the wrong one [► Aug update 1:04](#).
- The comments list only appears in comment mode [► Comments short 0:34](#).
- On Pro, every editor is a paid seat [► Aug update 5:43](#).
- Until remote MCP ships, the desktop app must be open for agents [► Sept update 4:32](#).
- A viewer asked for a save dialog instead of exports going straight to Downloads; there was no reply (under [► Aug update 0:00](#)).
- Labels moved between builds: the May capture video shows "Add flex ⤴A" in the Layout section, the September build shows "Wrap in flex ⤴A" in the same place, so older videos may not match your screen word for word [► Snapshot 1:30](#) [► Sept update 2:08](#).

Apply it to your files

1. **Altitude Sports: drift check.** Capture the KPI row of the live Railway deployment with Snapshot and paste it into the file next to the "Dashboard" board. Rename the captured nodes, then run recipe 1 to compare computed styles with the Paper source and decide, line by line, whether the fix belongs in code or in the design.
2. **Water Dashboard: comment pass.** In comment mode (C), pin five short notes in French on specific KPI cards and chart labels. Run recipe 3. Afterwards filter the panel to unresolved and sort "By page" to see what is left.
3. **Water Dashboard: token upkeep.** In the Theme tab, add opacity tokens ("+" → Opacity), pair one with the muted text color, convert any raw fill you find with the fill's token icon and "+", and tune the cobalt accent with the Previous/New comparison.
4. **File housekeeping.** Before the next big refactor, right-click → "Duplicate" instead of making a manual "Sauvegarde" file. ⌘-click the 14 loose files and use "Move items to..." to sort them into client and personal folders. Find files later with ⌘F.
5. **Ballottage: reference-driven flow.** Put three to five reference flows on a references page (Mobbin's MCP if you have access, Snapshot captures of web flows otherwise), comment on the screens you like, run recipe 4 for three variations, and let a judge agent compare the rows.
6. **Poseidon and the Express Entry dashboard in parallel.** Open both files as tabs and run recipe 5 in two sessions, one per file. Use the dots on each tab to see which agent is still working.

Prompt recipes

Paste these into Claude Code with the Paper MCP connected. Replace the angle-bracket parts; get frame links with ⌘L.

1. Live vs design drift (Altitude Sports)

In Paper, in the Altitude Sports file: the frame "Live capture" is a Paper Snapshot of the deployed dashboard, and the frame "Dashboard" is the design source. First rename the captured nodes by role (KPI row, KPI card, chart card, table). Then compare the two with `get_computed_styles`, never from screenshots: spacing, font family/size/weight, colors and token usage, grid columns and gutters. Output a table of differences with node names and both values. Do not edit the "Dashboard" frame.

2. One incremental change on an existing board (Water)

In Paper, in the Water Dashboard file, frame <%L link>: add one KPI card to the KPI row. Duplicate an existing card with `duplicate_nodes`, keep the 4 × 352 grid with 24 px gutters and the 8 px rhythm, and take the label and value from the frame "KPI copy". Change nothing outside that row. Then take a screenshot, run a checkpoint review (spacing, typography, contrast, alignment, clipping, repetition), set any clipped frame to height fit-content, and call `finish_working_on_nodes`.

3. Implement my comments

In Paper, in the <file name> file: read every unresolved comment thread (list_comment_threads, then get_comment_thread). For each one, quote it, make that change on the element it is pinned to and nowhere else, take a screenshot, then resolve the thread with set_comment_thread_status. If a comment is ambiguous or would break the grid, skip it and tell me why.

4. Reference-driven variations (Ballottage)

In Paper, in the Ballottage file: the frames on page "<references page>" are reference flows, and my comments on them say what I like. Read the frames and the comments. Using only the tokens and components on page "02 Grid and components", design 3 variations of the prediction submission flow. Put each variation in its own row, named "V1 · <pattern> - 01 <screen>", "V2 · ...", "V3 · ...". No new colors or fonts. Screenshot each row when it is done.

5. Two agents, two tabs

Session A: In Paper, in the Poseidon file only, frame <%L link>: <task>. Do not read or edit any other file.
Session B: In Paper, in the Express Entry dashboard file only, frame <%L link>: <task>. Do not read or edit any other file.

6. Resync after manual edits

In Paper, frame <%L link>: I edited this frame by hand. Read it again with get_screenshot and get_computed_styles and list exactly what changed since your last write (node, property, old value, new value). Do not modify anything; wait for my go.

7. Model comparison board (Ballottage figurines)

In Paper, in the Ballottage file, on a new page "Figurine models": take the reference photo layer "<reference>" and generate the same figurine with three image-to-image models (OpenAI GPT Image 2.5 edit, Nano Banana Pro edit, Flux 2 Pro edit) and one shared prompt. Put each result in a white card of the same size on a neutral light ground, the figurine standing on a soft floor shadow, with the model name inside its card, never as a loose caption. Generation spends usage units: run each model once, then stop.

Reference

Facts to look up rather than read through. They come from Paper's documentation, build log, MCP guides and tool schemas, the Paper plugin on your Mac and the videos, checked on 2 October 2026. Paper changes monthly, so treat anything without a video link as the state on that date.

Your setup, checked on 2 October 2026

- **Two Paper connections are active in your Claude sessions.** `plugin:paper-desktop:paper` is the Claude Code plugin (v0.1.0, HTTP to `127.0.0.1:29979`). `Paper` is most likely the Claude Desktop extension (v0.1.1, installed 24 September 2026), which runs the Paper CLI over stdio. They expose nearly the same tools (36 and 35): the plugin has `list_resources` and `rename_resource`, the extension has `list_files` instead. Your calls split 10,313 through the plugin and 3,705 through the extension. Every session loads both tool sets, which costs context for nothing.
- **Your plugin is two versions behind.** Upstream v0.2.1 (since 24 August 2026) connects through the Paper CLI over stdio, so the agent sees Paper's tools even when the app is closed, and it no longer ships the `code-to-design` and `design-to-code` skills or the `ensure-paper-started` rule. Updating removes those two skills from Claude Code; copy their folders into `~/.claude/skills/` first if you want to keep them.
- **Plan.** The Free plan allows 100 MCP tool calls a week and Pro allows 1 million. September averaged about 2,200 calls a week, so this volume needs Pro.
- **What the MCP cannot do yet:** delete a page or a file, write a comment, insert or edit a shader, or choose an export folder (exports land in Downloads).
- **French keyboard.** The Z zoom shortcut was fixed for French and German layouts in June 2026, and the numeric opacity and zoom keys for AZERTY in September 2025.
- **Versions on your Mac.** Paper Desktop 0.5.15 (released 1 October 2026) and the Paper CLI at `~/.paper/bin/paper` (0.0.1).

Keyboard shortcuts

Mac notation. An official consolidated shortcut list was not found: [paper.design/docs/shortcuts](#) returns 404, and web searches returned nothing. The rows below are assembled from the changelog, docs, official videos and the desktop bundle.

Tools and canvas | Keys | Action | Source | |---|---|---| | P | Pen tool | [build log, Jun 2026](#); [paper.design/docs/svg](#); ▶ [SVG editing 3:05](#) | | M | Move tool (move path nodes) | [paper.design/docs/svg](#) | | H | Pan tool | [build log, Apr 2026](#) | | S | Open the shaders menu | ▶ [Cloudflare shader 0:32](#); ▶ [Logo shaders 0:29](#) | | C | Add a comment | [build log, Aug 2026](#) | | I | Color picker (eyedropper) | [build log, Aug 2026](#) ("restored"); ▶ [8 tips 3:05](#) | | Shift + eyedropper click | Add gradient colors with the eyedropper | [build log, 6 Nov 2025](#) | | Z | Zoom (fixed for French and German keyboards in Jun 2026) | [build log, Jun 2026](#) | | Scroll wheel | Zoom

(option added Jun 2026) | [build log, Jun 2026](#) | | Shift + scroll | Scroll the canvas horizontally | [build log, 19 Sep 2025](#) | | ← → ↑ ↓ | Pan the canvas | [build log, 22 Oct 2025](#) | | Number keys | Set opacity; pressing 0 once gives 100% (also numeric zoom keys, numpad supported) | [build log, 3 Dec 2025](#); [build log, 19 Sep 2025](#); [build log, Apr 2026](#) | | Exact numeric zoom bindings (e.g. zoom to fit, zoom to selection) | not found | |

Create and insert | Keys | Action | Source | |---|---|---| | ⌘ I | Create image (AI raster generation) | [build log, Apr 2026](#); ▶ [xAI images 0:02](#) | | ⌘ J | Create SVG (AI vector generation) | [build log, Apr 2026](#); [paper.design/docs/svg](#); ▶ [Quiver SVG 0:11](#) | | ⌘ K | Upload images from the file system | [build log, Apr 2026](#) | | Frame, text, rectangle, ellipse tool letters | not found | |

Layout and structure | Keys | Action | Source | |---|---|---| | ⌘ A | Wrap the selection in a flex frame, or add flex layout to a freeform frame | ▶ [8 tips 0:51](#); ▶ [Flex layout 1:36](#) | | ⌘ F | Put the selected layers into a freeform frame | ▶ [8 tips 0:33](#) | | Double-click a resize handle or frame edge | Resize to fit content (fit height when used on the top or bottom edge) | [build log, Mar 2026](#); ▶ [Flex layout 5:24](#) | | ⌘ + double-click a resize handle or edge | Resize to fill the parent | [build log, Mar 2026](#); ▶ [Flex layout 9:37](#) | | Double-click the 9-point alignment grid | Space-between | ▶ [Flex layout 5:56](#) | | Hold ⌘ | Show distances to edges | ▶ [Flex layout 12:00](#); ▶ [Constraints 2:24](#) | | Arrow / ⌘ + arrow on a selection | Nudge by 1, or by the nudge setting (8 by default in the video) | ▶ [Flex layout 2:04](#) | | ⌘ -drag | Duplicate (repeated rotation now compounds) | [build log, Aug 2026](#); ▶ [New in Paper, Aug 4:27](#) | | ⌘ D | Duplicate, repeating the last offset and rotation | ▶ [New in Paper, Aug 4:40](#) | | ⌘ -resize, ⌘ -resize | Resize both sides equally; snap-aware resize | [build log, 19 Nov 2025](#) (bug fixes naming both behaviors) | | ⌘ L | Collapse all layers (also right-click in the layers list, Collapse all) | [build log, Jun 2026](#); ▶ [New in Paper, Jun 1:58](#) | | Group, ungroup, align, distribute shortcuts | not found | |

Selection | Keys | Action | Source | |---|---|---| | Double-click | Select a child inside a frame | ▶ [Hover selection 0:13](#) | | Hold ⌘ + click | Deep select (hover selection targets shallow frames by default since Jun 2026; deep selection can be switched back on in the file menu or Project Settings) | [build log, Jun 2026](#); ▶ [Hover selection 0:13](#) ▶ [Hover selection 0:28](#) | | ⌘ + right-click | List every layer under the cursor and pick one | [build log, Jun 2026](#); ▶ [Hover selection 0:57](#); ▶ [New in Paper, Jun 1:46](#) | | Shift + click | Add to the selection | ▶ [Flex layout 6:18](#) | | Double-click a layer in the layers list | Zoom to it and center it | ▶ [8 tips 0:21](#) | | Double-click a layer name, or on canvas | Rename | ▶ [Flex layout 1:36](#) |

Text (all added April 2026) | Keys | Action | Source | |---|---|---| | ⌘ ⌘, and ⌘ ⌘. | Font size down / up | [build log, Apr 2026](#) | | ⌘ ⌘, and ⌘ ⌘. | Font weight down / up | [build log, Apr 2026](#) | | ⌘, and ⌘. | Letter spacing down / up | [build log, Apr 2026](#) | | ⌘ ⌘, and ⌘ ⌘. | Line height down / up | [build log, Apr 2026](#) | | ⌘ B / ⌘ I | Toggle bold / italic | [build log, Apr 2026](#) |

Images | Keys / gesture | Action | Source | |---|---|---| | Hold ⌘ while resizing | Crop the image | [build log, 3 Dec 2025](#); ▶ [8 tips 1:37](#) | | Double-click an image edge or corner | Reset the image to its original dimensions | ▶ [8 tips 2:05](#) | | Right-click, Extract colors | Build "magic swatches" from the image | ▶ [8 tips 2:29](#); ▶ [Magic swatches](#) | | Right-click, Remove background | Background removal | ▶ [Remove background](#) (29 Aug 2026) | | Right-click, Vectorize | Raster to vector | ▶ [Paper Shaders 3:47](#); [build log, 9 Sep 2025](#) |

Vector path editing | Keys | Action | Source | |---|---|---| | Enter, or double-click a path | Enter path-editing mode (on an SVG: the first double-click selects a path, the second enters edit mode) | [paper.design/docs/svg](#); ▶ [New in Paper, Jun 1:18](#) | | Esc, or double-click empty space | Leave path-editing mode | [paper.design/docs/svg](#); ▶ [SVG editing 5:43](#) | | Shift while placing pen points | Constrain to 45° | ▶ [SVG editing 3:27](#) | | Hold ⌘ (pen) | Drag out curves; ⌘ on an

edge midpoint bends the edge | [► SVG editing 4:00](#) [► SVG editing 4:40](#) | | Hold `⌘` on a handle | Break handle mirroring (`⌘` restores mirroring) | [► SVG editing 4:15](#) [► SVG editing 4:28](#) | | `⌘`' | Toggle pixel snapping (snaps to half pixels) | [paper.design/docs/svg](#) |

Clipboard, links, code | Keys | Action | Source | `⌘C` / `⌘V` | Copy / paste, including Figma layers, HTML, SVG and Snapshot captures | [paper.design/docs/paste](#); [► Figma paste 0:39](#) | | "Paste on top" and "Paste to replace" (`⌘R`) | Printed on the docs as "`(⌘V)`" and "`(⌘R)`". Paste on top looks like a missing modifier, and in Desktop `⌘R` is also bound to View > Reload. Unverified; check in the app | [paper.design/docs/paste](#); Paper Desktop app | | `⌘R` / `⌘T` | Copy selection as React (CSS) / as Tailwind | [build log, 19 Dec 2025](#) | | Right-click, Copy as PNG / Tailwind / React CSS; also the copy icon in the Export panel (Sep 2026) | Copy as code or image | [► New in Paper, Sep 3:38](#); [build log, Sep 2026](#) | | `⌘L` | Copy a link to the selected frame | [► New in Paper, Aug 1:44](#) |

Interface and collaboration | Keys | Action | Source | `⌘.` | Hide or show the UI (also right-click, Hide UI) | [► 8 tips 0:12](#); [► New in Paper, Jun 4:40](#) | | `⌘.` | Minimize the UI | [► 8 tips 0:21](#); [► New in Paper, Jun 5:01](#) | | `⌘\` | Hide the UI, as shown in a May 2026 video (probably an earlier binding) | [► Design directions 2:26](#) (2 May 2026) | | Right-click on canvas | Cursor chat | [build log, May 2026](#) |

Paper Desktop app menu (bundle 0.5.15) | Keys | Action | Source | `⌘N` / `⌘T` / `⌘T` | New File / New Tab / Reopen Closed Tab | Paper Desktop app | | `⌘N` / `⌘W` / `⌘W` | New Window / Close Tab / Close Window | Paper Desktop app | | `⌘Z` / `⌘Z` | Undo / Redo | Paper Desktop app | | `⌘R` / `⌘R` / `⌘I` | Reload / Force Reload / Toggle Developer Tools | Paper Desktop app | | `⌘D` | Go to Dashboard | Paper Desktop app | | `⌘Tab` / `⌘Tab`, and `⌘→` / `⌘←` (hidden) | Next / previous tab | Paper Desktop app | | `⌘1` to `⌘8` (hidden) / `⌘9` | Go to tab N / go to the last tab | Paper Desktop app | | Help menu | Check for Updates, Documentation, Video Tutorials, Release Notes, Discord, Slack Community, Reddit, X/Twitter, View Logs | Paper Desktop app |

Dashboard: `⌘`-click selects several files or folders; right-click offers copy links, archive, move, Move To and Duplicate; double-click a folder title to rename it ([► New in Paper, Aug 4:56](#) [► New in Paper, Aug 5:19](#); [► Folders 0:24](#) [► Folders 1:20](#)).

Paper Snapshot (Chrome extension): `⌘P` activates (remappable at `chrome://extensions/shortcuts`); `↑/↓` widen or narrow the target; Enter or click captures; `⌘Enter` captures the full page; then `⌘V` in Paper ([paper.design/snapshot-extension](#); [build log, May 2026](#); [► New in Paper, Jun 7:07](#); [► Resend button 0:20](#)).

Connecting an agent

- The MCP server runs inside Paper Desktop. Opening Desktop starts it in the background, and it reads and writes the file that is open ([paper.design/docs/mcp](#); [paper.design/docs/support](#)). An agent edits only the open file, so check that the right file is open ([paper.design/docs/support](#)). Since August 2026 agents can also work across several open files, including background tabs ([build log, Aug 2026](#)).
- Two transports. 1. Stdio through the Paper CLI (current recommendation): the agent launches `/path/to/paper mcp`, which relays to Desktop. The agent can see Paper's tools and instructions even while Desktop is closed, and no localhost URL has to be pasted ([paper.design/docs/mcp](#)). The CLI path is copied from the MCP panel in Desktop ([paper.design/docs/mcp](#)). On this Mac it is `~/ .paper/bin/paper` (Paper CLI; Claude extensions list). 2. Streamable HTTP at `http://127.0.0.1:29979/mcp` (older setup). Desktop

still serves it while a file is open, so existing HTTP configs keep working (paper.design/docs/mcp). In the bundle, port 29979 and path `/mcp` are constants, served by a Hono HTTP server with a streamable HTTP transport, and the server identifies itself as `paper-desktop` with the app version (Paper Desktop app).

- Paper Web (browser) offers no local MCP. Since September 2026 it supports WebMCP for in-browser agents ([build log, Sep 2026](#)). The partner example embeds Paper in an iframe (permissions `tools; clipboard-read; clipboard-write; local-fonts`) and calls `document.modelContext.getTools() / executeTool()`. It needs Chrome 150+ and the WebMCP origin trial, and Paper must allowlist the origin ("available for Paper partners") (github.com/paper-design/webmcp-agent-example).
- One-click setup in the app (Sep 2026): the Connect your agent button in the editor's right panel opens a panel where you pick a harness and click "Add to ..." ([build log, Sep 2026](#); ▶ [New in Paper, Sep 0:56](#)). The bundle's installer handles `claude-code`, `codex-cli`, `chatgpt`, `github-copilot`, `opencode` and `antigravity`; a `vim` id also appears, possibly only in tests. Its tests show it replaces an existing HTTP `paper` entry with a `studio` entry, for example in `~/.claude.json` (Paper Desktop app).
- Coming next, as of 28 Sep 2026: "remote MCP", meaning cloud agents with no Desktop running (▶ [New in Paper, Sep 4:32](#)).

Setup per agent

Agent	Setup	Source
Claude Code	<code>claude plugin marketplace add paper-design/agent-plugins</code> && <code>claude plugin install paper-desktop@paper</code> , restart, check with <code>/mcp</code> . Inside Claude Code: <code>/plugin marketplace add paper-design/agent-plugins</code> then <code>/plugin install paper-desktop@paper</code>	paper.design/docs/mcp ; agent-plugins repo, README.md
Claude Code, March 2026 method	A one-line terminal command copied from the docs that registered an HTTP server; the CLI printed "added HTTP MCP server Paper". The exact command is not shown in the video	▶ Sync with code 0:39 (5 Mar 2026)
Claude (Desktop chat)	Download the <code>.mcpb</code> extension from <code>claude-mcpb.paper.design</code> , then Settings > Extensions > Advanced settings > Install extension (restart on Linux). The manifest runs <code>\${HOME}/.paper/bin/paper mcp</code> and requires Desktop to have been opened once	paper.design/docs/mcp ; Claude extensions list
Cursor	Install from the Cursor Marketplace (<code>cursor.com/marketplace/paper</code>), or <code>/add-plugin paper-desktop</code> in the agent panel; check Settings > Tools & MCP and toggle if stale	paper.design/docs/mcp ; agent-plugins repo, README.md
VS Code	<code>.vscode/mcp.json</code> : <code>{"servers":{"paper":{"type":"stdio","command":"/path/to/paper","args":["mcp"]}}}</code>	paper.design/docs/mcp
Codex CLI	<code>codex mcp add paper -- /path/to/paper mcp</code> . Plugin route: <code>codex plugin marketplace add paper-design/agent-plugins</code> , then <code>codex plugin install paper-desktop@paper</code>	paper.design/docs/mcp ; agent-plugins repo, README.md
ChatGPT app (Codex tab)	Settings > Plugins > Add > Add MCP server: name <code>Paper</code> , command = CLI path, arguments <code>mcp</code>	paper.design/docs/mcp
GitHub Copilot	One-click from the MCP panel, or Customize > Add > MCP Server (command = CLI path, args <code>mcp</code>)	paper.design/docs/mcp
OpenCode	<code>opencode.json</code> : <code>{"mcp":{"paper":{"type":"local","command":["/path/to/paper","mcp"],"enabled":true}}}</code> , verify with <code>opencode mcp list</code>	paper.design/docs/mcp
Antigravity / any other stdio client	<code>{"mcpServers":{"paper":{"type":"stdio","command":"/path/to/paper","args":["mcp"]}}}</code>	paper.design/docs/mcp
Windsurf, Zed, Gemini CLI	Not named on the page (use the generic stdio config)	paper.design/docs/mcp

Test prompts suggested by the docs: a Hello World frame, three color tokens with a swatch, a generated sunset image (paper.design/docs/mcp).

Troubleshooting: restart the agent session first; then restart the MCP host. If the agent invents tools that do not exist, restart everything. On WSL, set `networkingMode` to `mirrored`. Behind a proxy or VPN, allowlist `*.paper.design`. Linux AppImage `paper://` links need an integration tool (Gear Lever, AppImageLauncher or AppMan) (paper.design/docs/mcp; paper.design/docs/support). The plugin rule: if the connection fails, remind the user to open Paper Desktop (Paper plugin, [rules/ensure-paper-started.md](#)). Errors seen in your sessions: "not connected" when Paper was closed at session start, and "Open a Paper file to use this tool" (your sessions).

How agents are told to work in Paper

- Load `get_guide("paper-mcp-instructions")` once per session, before other tools; reload it in long threads (MCP server instructions).
- Start with `get_basic_info`; use `get_selection` for the user's focus. Omitting `fileId` targets the viewed file; omitting `pageId` targets the viewed page. Passing a `pageId` lets the agent work elsewhere without disturbing the user, but the agent cannot switch the user's page (MCP server instructions; MCP guide). The May 2026 changelog says agents can change pages within a file ([build log, May 2026](#)); the current guide says they cannot switch the viewed page.
- Every file-scoped tool returns a file header with file id, name and a `contentHash` of the tokens. Check the file id, and re-read tokens if the hash changed (MCP guide).
- Typography: call `get_font_family_info` before the first text styling of a session; prefer fonts already in the file; font size in px, letter-spacing in em, line-height in px (MCP server instructions; MCP guide).
- New designs: post a brief to the user before any mutation. The brief lists mood candidates, the chosen mood (deliberately not the first instinct), 5–6 palette hex values with roles, type scale and direction (MCP guide, "Before Creating New Designs").
- Build small: one visual group per `write_html`; more than about 15 lines of HTML means split it; never batch a whole component. Clone existing nodes with `<x-paper-clone node-id="..." style="..." />`. Prefer `duplicate_nodes` + `update_styles` + `set_text_content` when that is cheaper (MCP guide; MCP schema).
- Review checkpoints are mandatory: after each section, take a screenshot and give a one-line verdict on spacing, typography, contrast, alignment, artboard fit and repetition. Fix things in place; do not start over. If content clips, set the artboard to `height: "fit-content"`; never guess a fixed height (MCP guide; MCP server instructions).
- Repeated rows need fixed-width slots (`width` + `flexShrink: 0`) for icons and trailing actions; check the vertical lanes with a screenshot after 3+ rows (MCP guide; MCP server instructions).
- Always call `finish_working_on_nodes` when done; pass specific artboard ids when other agents are active (MCP server instructions; MCP schema). An artboard being worked on shows a dot pattern above it ([► Claude Code setup 2:42](#)).
- Never show raw node IDs to the user (MCP server instructions; MCP guide).
- Design to code: use `get_jsx`, `get_computed_styles` and `get_fill_image` for exact values, never screenshots; translate to the codebase's conventions

- (MCP guide; MCP server instructions).
- Tokens: always use existing tokens through `var(--...)` . New token sets follow Tailwind v4 namespaces (`--color-*`, `--breakpoint-*`, `--spacing-*`, `--container-*`, `--opacity-*`, `--font-*`, `--font-weight-*`, `--text-*`, `--tracking-*` (em), `--leading-*` (px), `--radius-*`). Token opacity: `color-mix(var(--color-x) 40%, transparent)` or with an opacity token (MCP guide).
 - Default artboard sizes: Desktop 1440×900, Tablet 768×1024, Mobile 390×844 with the status bar. Artboards default to flex column and are auto-placed; leave 80 px between artboards when positioning them (MCP schema). The guide's step 1 says mobile is 375 wide, which conflicts with 390 in the tool description (MCP guide).
 - Design-quality doctrine (paraphrased): minimalism with a warm human touch; deliberate spacing; asymmetry and scale contrast; Swiss-editorial punchy type with heavy display and light labels; light mode by default; information on surfaces rather than everything in cards; avoid late-2010s gradients and shadows; one intense color moment. Choose a mood word, then derive every color from objects in that scene. The guide lists 19 proven ground × accent pairings, among them editorial (pure white × cobalt), hypertext (pure white × hyperlink blue) and maritime (fog gray × deep navy). Pairings to avoid: warm off-white with red/orange/terracotta; warm off-white with fluorescents; dark navy or charcoal with electric purple, lime or teal; pure white with muted earth tones; tinted warm grounds with saturated accents. Pure white `#FFFFFF` is the normal ground for SaaS dashboards. Text contrast is non-negotiable, and text of 12 px or less is acceptable only in dense productivity UIs. Placeholder content must be realistic and must never mention Figma or Sketch (MCP guide, "Design Quality").
 - Mobile status bar: paste the guide's ready-made Apple status bar (9:41 in system-ui 17/600, plus an 82×22 SVG of signal, wifi and battery) as the first child; change only its color (white by default, black on light grounds); omit it only for full-screen experiences (MCP guide; MCP schema).
 - Figma import: `write_html` understands only literal CSS. Resolve every Figma variable, token or Tailwind class to a concrete value. Inline asset URLs; the Figma MCP's `http://localhost:3845/assets/...` URLs are valid. Tiled backgrounds go on a div with `background-repeat` . For instance node ids like `I700:1100;2000:1000` , use the last segment (MCP guide).

Paper MCP tools, with your calls

Required parameters are in bold. On the plugin server `fileId` is required by every file-scoped tool except `get_basic_info` and `get_selection`; on the `Paper` server most parameters are optional (MCP schema). Your call counts are 3 Jul–2 Oct 2026 (your sessions for `get_basic_info` and `get_guide`).

Tool	What it does	Key parameters	Tips and caveats (source)	Your calls
<code>get_guide</code>	Reads a guide topic	topic : <code>paper-mcp-instructions</code> , <code>mobile-status-bar</code> , <code>figma-import</code> , <code>image-generation</code>	Load the main guide once per session (MCP server instructions)	204
<code>get_basic_info</code>	File name, page, node count, artboards with sizes, fonts used, compact token list, pages with <code>isActive</code>	<code>fileId?</code> , <code>pageId?</code>	Call first; positions are null for unmeasured layout on inactive pages (MCP schema)	316

Tool	What it does	Key parameters	Tips and caveats (source)	Your calls
list_resources (plugin) / list_files (<code>Paper</code>)	Lists open files first (with <code>open</code> , <code>active</code> , <code>draft</code> flags), then recently updated files and folders of the active team	limit (default 50, max 200)	Raise the limit until <code>truncated</code> disappears before treating the list as complete (MCP schema)	17
open_file	Opens a file by id, <code>/file/<id></code> path or URL; returns <code>get_basic_info</code>	fileId , <code>pageId</code> ?	<code>pageId</code> applies only if the file is not already open, so the user is not yanked away (MCP schema)	51
create_file	New file in the active team, optionally cloning one	<code>name?</code> , <code>cloneFileId</code> ?	Follow with <code>open_file</code> . <code>cloneFileId</code> is the MCP route to backups (MCP schema)	5
rename_resource (plugin only)	Renames a file or folder	resourceId , name	Trimmed, max 500 characters; empty name leaves it unchanged (MCP schema)	0
create_page	Adds a page, returns <code>pageId</code>	fileId , <code>name?</code> (default "Page N")	No page-delete tool exists (MCP schema)	20
rename_pages	Renames pages in batch	updates [{ <code>pageId</code> , <code>name</code> }]	Does not switch the user's page (MCP schema)	7
get_selection	Selected nodes: ids, names, types, size, artboard	<code>fileId</code> ?	Use it to learn the user's focus (MCP guide)	1
get_tree_summary	Indented subtree: type, name, id, size	nodeId , <code>depth</code> (default 3, max 10)	Much cheaper than <code>get_jsx</code> for orientation; <code>?</code> marks unmeasured sizes (MCP schema)	643
get_children	Direct children with ids, names, types, child counts, world and relative x/y	nodeId	Errors if the node does not exist (MCP schema)	710
get_node_info	Size, visibility, lock, parent, children ids, text, positions; <code>imageGeneration.status</code> / <code>.output</code>	nodeId	Poll it for AI image completion (MCP schema; MCP guide)	652
find_nodes	Finds nodes by computed style and/or text, across all pages by default	<code>filters</code> [{ <code>styleName</code> , <code>styleValue</code> }] (AND, <code>*</code> wildcards), <code>textValue</code> (case-insensitive, anchored <code>*</code>), <code>nodeId</code> > <code>pageId</code> scope	Colors match by equivalence; searching a literal color also finds token-bound uses, which helps migrate raw colors to tokens; results list what matched (MCP schema)	663
get_screenshot	PNG of a node	nodeId , <code>scale</code> (1 default, 2 for small text)	Size-capped; capture children for detail; does not wait for image generation (MCP schema; MCP guide)	2,805
get_jsx	JSX of a node subtree	nodeId , <code>format</code> <code>tailwind</code> (default, inline fallback) or <code>inline-styles</code>	Element ids in the output can be used as targets; reads generated SVG output (MCP guide)	969
get_computed_styles	Computed CSS per node	nodeIds []	Batch several ids; use for exact values in code (MCP schema; MCP guide)	347
get_fill_image	Image fill as base64 JPEG plus the original URL in metadata	nodeId	Returns a message for SVG nodes (use <code>get_jsx</code>) or while still generating (MCP schema)	33
get_font_family_info	Whether a family is available and its weights and styles (local machine + Google Fonts, plus web-safe and system families)	familyNames []	Mandatory before the first typographic styling (MCP server instructions; MCP guide)	107

Tool	What it does	Key parameters	Tips and caveats (source)	Your calls
create_artboard	New top-level frame, auto-placed	name , styles (whole-px width , height), pageId ?	Default flex column; backgroundImage can carry paper-gen://; see the default sizes in 4.3 (MCP schema)	229
write_html	Parses HTML into nodes	html , targetNodeid , mode insert-children or replace	Inline styles only; rules in section 10; one visual group per call; replace removes the target node (MCP schema). In your sessions, replace dropped children's absolute positioning (your sessions)	3,092
update_styles	Batch style changes	updates [{ nodeId s, styles (camelCase)}]	Accepts token vars; top/left on an artboard move it on the canvas; inert styles are dropped and reported in ignoredStyles ; accepts paper-gen:// backgrounds (MCP schema)	1,139
set_text_content	Replaces text of Text nodes	updates [{ nodeId , textContent }]	Text nodes only; cheaper than write_html replace (MCP schema)	421
duplicate_nodes	Deep clone, optionally under a new parent	nodes [{ id , parentId ?}]	Returns descendantIdMap so clones can be targeted immediately; duplicated artboards are auto-placed (MCP schema)	204
move_nodes	Reorder or reparent, keeping ids	moves : { nodeId , before } / { nodeId , after } / { nodeId , parentId , index ?}	parentId : 'root' means the page root; works across pages; sequential; returns affectedParents ; layout intents may be adjusted to avoid collapse. Prefer it to delete-and-rewrite (MCP schema; MCP guide)	240
rename_nodes	Renames layers in batch	updates [{ nodeId , name }]	Names over 50 characters are truncated (MCP schema)	191
delete_nodes	Deletes nodes and their descendants	nodeIds []	Check suspected wrong parents with get_node_info first (MCP schema)	402
get_tokens	Lists the file's tokens	format json / css (:root)/ tailwind (@theme), namePattern glob, types []	Use the tailwind format for a Tailwind v4 codebase (MCP schema)	31
create_tokens	Creates tokens	tokens [{ type , name --... , value , description ?}]	Types: breakpoint, color, container, fontFamily, fontSize, fontWeight, letterSpacing, lineHeight, opacity, radius, spacing. Alias with var(--x) . Order: semantic colors before palette, neutrals, then primary, secondary, accent; other types smallest first; reuse before creating (MCP schema)	22
set_tokens	Renames, revalues or deletes tokens	tokens [{ name , newName ?, value ?, delete ?, description ?}]	Per-entry errors are reported in-band (MCP schema)	16
export	Exports nodes as image or video files	nodes : "nodes-with-exports-only" or { nodeId : [{ format , scale , pdfQuality ?, pdfResampling ?, durationSeconds ?}]}, type image / video , pageId ?	Formats avif, jpg, mp4, pdf, png, svg (SVG nodes only), webm (transparent), webp. Scale is a string such as 2x , 720p , 512w or 512h ; both format and scale are required in each override; [] uses the defaults. Video duration 1–300 s (default 10). Do not override defaults unless asked (MCP schema). You hit invalid-argument errors on this tool (your sessions)	202

Tool	What it does	Key parameters	Tips and caveats (source)	Your calls
export_combined_pdf	Several nodes into one PDF, one page each	nodeIds[]	Pages are ordered by canvas position, top to bottom then left to right; uses the highest PDF quality set on any of the nodes (MCP schema)	3
finish_working_on_nodes	Clears the "working" indicator	fileId, nodeIds?	Mandatory at the end (MCP server instructions)	265
list_comment_threads	Comment threads, open ones by default	status <code>open</code> / <code>resolved</code> / <code>all</code> , <code>pageId</code> , <code>nodeId</code> , <code>participantUserId</code> / <code>threadAuthorUserId</code> (<code>current-user</code>), <code>search</code> , <code>searchScope</code> , <code>sortField</code> <code>page</code> / <code>createdAt</code> / <code>updatedAt</code> , limit ≤ 200 , <code>offset</code> , <code>previewLength</code> 20–1000	Read-only discovery; there is no tool to write comments (MCP schema)	7
get_comment_thread	Full thread: replies, reactions, attachments, page and node context	commentThreadId	Deleted threads are never returned (MCP schema)	0
list_comment_thread_authors	Users who commented, with counts	<code>pageId</code> ?	Resolve a name to a <code>userId</code> before filtering (MCP schema)	0
set_comment_thread_status	Resolves or reopens a thread	commentThreadId , status	Resolve once the feedback is fully addressed (MCP schema)	0

Not available through the MCP: deleting pages or files, writing comments, inserting shaders, components, undo, choosing an export destination (MCP schema).

The plugin's two skills

Both skills are exposed in Claude Code as `paper-desktop:code-to-design` and `paper-desktop:design-to-code` (session skill list; Paper plugin, `.claude-plugin/plugin.json` sets `"skills": ". /skills"`).

	code-to-design	design-to-code
Purpose	Generate a Paper design from the project's codebase, using its tokens, styles and components as context	Turn a Paper design into production code that follows the project's conventions
Precondition	Paper Desktop running with a file open; if the connection fails, tell the user to open Desktop	same
Steps	1. Read the project's stylesheets, design tokens or theme files to learn the design language. 2. Create a new artboard. 3. Build the UI with the codebase's actual colors, type, spacing and component patterns, not generic defaults	1. Read the selected frame or artboard: structure, styles, text, computed layout. 2. Generate production-ready components in the project's framework and style. 3. Match layout, spacing, type and color through the codebase's conventions (Tailwind classes, CSS modules, design tokens, styled-components)
Source	Paper plugin, skills/code-to-design/SKILL.md	Paper plugin, skills/design-to-code/SKILL.md

- The rules shared by both come from the guide. For design to code: exact values from `get_jsx`, `get_computed_styles` and `get_fill_image`; screenshots only for verification; translate to the codebase's conventions. For code to design: brief first, existing tokens through `var()` , incremental writes, screenshot reviews, `finish_working_on_nodes` (MCP guide).
- Plugin rule file: before any Paper tool, make sure Desktop is running; on connection failure, remind the user to open it (Paper plugin, `rules/ensure-paper-started.md`).

- README examples: design a settings page from the repo's CSS; implement the Paper design with the project's conventions; pull tokens from Figma (Figma MCP required) and build a sticker sheet in Paper (Paper plugin, README.md). The Codex manifest uses the same three as default prompts and declares capabilities Read and Write, brand color #81ADEC (Paper plugin, .codex-plugin/plugin.json).
- Plugin metadata: MIT license, repository github.com/paper-design/agent-plugins, support team@paper.design, docs paper.design/docs/mcp (Paper plugin, .claude-plugin/plugin.json; Paper plugin, README.md).
- Upstream status: v0.2.1 removes the skills and rules keys and folders and points to mcp.claude.json (stdio CLI). Separate MCP files exist for Codex and Cursor (raw.githubusercontent.com/paper-design/agent-plugins/main/plugins/paper-desktop/claude-plugin/plugin.json; github.com/paper-design/agent-plugins/tree/main/plugins/paper-desktop). Updating the plugin would therefore remove the two skills from your Claude Code.

Image models

Where it works: `` or `background-image: url("paper-gen://...")` in `write_html`; `backgroundImage` through `update_styles`; `backgroundImage` in `create_artboard` styles (MCP guide).

URL format: `paper-gen://{model_id}?prompt=...&aspect_ratio=...&reference_nodes=...`. The prompt is required and URL-encoded. `aspect_ratio` defaults to 1:1: an unsupported value falls back to the default with a warning; it is dropped for SVG models; it is ignored when reference images exist (the first reference's ratio wins). `reference_nodes` takes comma-separated node ids: their text is prepended to the prompt, and their images (including descendants) serve as references for image-to-image models. Text-to-image models ignore references with a warning; missing ids are skipped with a warning (MCP guide).

Models (costs in usage units)	Model id	Name and strength	Mode	Cost
Aspect ratios --- --- --- --- ---	openai-gpt-image-2-5	OpenAI Image 2.5, good with text	text→image 1 1:1 3:2 4:3 16:9 2:1 3:1 2:3 3:4 9:16 1:2 1:3	
	openai-gpt-image-2-5-big	OpenAI Image 2.5 Big, highest precision, slower	text→image 2 same as above	
	google-nano-banana-2	Nano Banana 2, general purpose (the default choice)	text→image and image→image 1 1:1 4:3 3:4 16:9 9:16 (image→image: matches the reference)	
	flux-2-pro	Flux 2 Pro, fast and versatile	text→image 1 1:1 3:2 2:3 4:3 3:4 16:9 9:16	
	grok-imagine-quality	Grok, very fast	text→image 1 1:1 3:2 2:3 4:3 3:4 16:9 9:16 2:1 1:2	
	recraft-v4-1	Recraft V4.1, design taste, strong text	text→image 1 1:1 3:2 2:3 4:3 3:4 16:9 9:16 2:1 1:2	
	ideogram-v3-balanced	Ideogram v3, dreamy	text→image 2 1:1 3:2 4:3 16:9 2:1 3:1 2:3 3:4 9:16 1:2 1:3	
	seedream-4-5	Seedream 4.5, precise, good with text	text→image 1 1:1 3:2 2:3 4:3 3:4 16:9 9:16	
	nano-banana-pro	Nano Banana Pro, 2K	text→image 4 1:1 4:3 3:4 16:9 9:16	
	quiver-arrow	QuiverAI Arrow 2, editable SVG, balanced	text→SVG 3 none (size the node with styles)	
	quiver-arrow-telos	QuiverAI Arrow 2 Telos, detailed SVG, slower	text→SVG 4 none	
	openai-gpt-image-edit-2-5	OpenAI Image 2.5 edit, supports transparency	image→image 2 matches the reference	
	openai-gpt-image-edit-2-5-big	OpenAI Image 2.5 Big edit, transparency	image→image 3 matches the reference	
	flux-2-pro-edit	Flux 2 Pro edit	image→image 1 matches the reference	
	grok-imagine-quality-edit	Grok edits	image→image 2 matches the reference	
	nano-banana-pro-edit	Nano Banana Pro edit, 2K	image→image 4 matches the reference	
	quiver-arrow-edit	Arrow 2 SVG		

edit | image→SVG | 3 | none | | `quiver-arrow-edit-telos` | Arrow 2 Telos SVG
edit | image→SVG | 4 | none | Source for the table: MCP guide.

Rules - Generate only when the user explicitly asks, because it spends the user's quota; otherwise use plain placeholders such as solid fills or SVG shapes (MCP guide; MCP schema). - More than 10 images in one call, or a call costing more than the remaining quota, is rejected entirely: nothing generates and every image shows the error visual (MCP guide). - For text inside images, prefer `openai-gpt-image-2-5`, `recraft-v4-1` or `seedream-4-5`. Quiver models output editable SVG nodes, work only through `<img>` in `write_html` (never as a background-image), and suit icons and vector illustrations; Telos is slower and costs more. Image-to-image ids fail without a usable reference image (MCP guide). - Generation is asynchronous. Poll `get_node_info` until `imageGeneration.status` reads `ready` or `error`. Raster output is read with `get_fill_image`, SVG output with `get_jsx`. Screenshots and exports do not wait for generation (MCP guide). - Setting a paper-gen URL always starts a new generation. To make a variant, create a new node that uses the original as `reference_nodes`; do not reuse the original (MCP guide). - Validation problems do not fail the call: the node shows a missing-image visual and a note explains why. Fix the URL with `write_html` replace, or with `update_styles` for backgrounds (MCP guide).

In the app: ⬆️⌘I Create image and ⬆️⌘J Create SVG, each with a model picker ([build log, Apr 2026](#); ▶️ [xAI images 0:14](#)). Selected text elements can serve as the prompt reference, sizes stay consistent across edits, and the prompt stays in the box during generation ([build log, Jun 2026](#)). Background removal and vectorize are on the right-click menu (▶️ [Remove background](#); ▶️ [Paper Shaders 3:47](#)).

Model history: Flux 2, Nano Banana Pro, OpenAI Image Edit 1.5 and Seedream 4.5 (19 Dec 2025); OpenAI GPT Image 2, Nano Banana 2 and Quiver Arrow 1.1 for SVG (Apr 2026); xAI (May 2026); generation through MCP (Sep 2026) ([build log, 19 Dec 2025](#); [build log, Apr 2026](#); [build log, May 2026](#); [build log, Sep 2026](#)). The current lineup (Image 2.5, Arrow 2) comes from MCP guide read on 2 Oct 2026.

Your usage: quiver-arrow-telos 115, quiver-arrow 72, openai-gpt-image-edit-2-5 51, google-nano-banana-2 20, quiver-arrow-edit 2 (your sessions). Counted once each at the guide's prices, that is about 804 usage units, 460 of them from Telos (computed from your sessions × MCP guide).

Paper Shaders, in Paper and in code

What they are: zero-dependency WebGL2 shaders that can be installed from npm or designed in Paper (github.com/paper-design; github.com/paper-design/shaders). The playground and docs live at shaders.paper.design, tagged "ultra fast zero-dependency shaders for your designs" (shaders.paper.design).

Packages and license - `@paper-design/shaders` (vanilla JS) and `@paper-design/shaders-react` (React wrapper, peer dependency `react ^18 || ^19`). Both at v0.0.81 on 2 Oct 2026 (registry.npmjs.org/@paper-design/shaders/latest; registry.npmjs.org/@paper-design/shaders-react/latest). - Pin the exact version: breaking changes ship under 0.0.x (raw.githubusercontent.com/paper-design/shaders/main/README.md). Example: v0.0.81 reworked Paper Texture and renamed parameters (raw.githubusercontent.com/paper-design/shaders/main/CHANGELOG.md). - License: Apache-2.0 since v0.0.77; from v0.0.47 (published 1 Sep 2025) it was PolyForm Shield 1.0.0 (raw.githubusercontent.com/paper-design/shaders/main/LICENSE; [CHANGELOG.md](https://raw.githubusercontent.com/paper-design/shaders/main/CHANGELOG.md); registry.npmjs.org/@paper-design/shaders-react). Commercial use is allowed without visible attribution;

redistributing in derivative libraries requires keeping LICENSE and NOTICE (README). Repository: about 3.5k stars, no GitHub releases (github.com/paper-design/shaders/releases). - Framework support: vanilla and React; Vue and others may come later through community PRs (README).

React usage (README example):

```
import { MeshGradient } from '@paper-design/shaders-react';
<MeshGradient colors={['#5100ff', '#00ff80', '#ffcc00', '#ea00ff']} distortion={1} swirl={0.8} speed={0.2} style={{ width: 200, height: 200 }} />
```

The shader fills its box, so give it a size. Settings can be configured in code or designed in Paper (README). Each component ships named presets, for example `meshGradientPresets` (default, ink, purple, beach) (raw.githubusercontent.com/paper-design/shaders/main/packages/shaders-react/src/shaders/mesh-gradient.tsx).

Common props on every shader: `speed` (0 means static, negative plays in reverse), `frame`, `scale` (0.01–4), `rotation` (0–360), `offsetX` / `offsetY` (–1 to 1), `width`, `height`, `fit` (contain/cover), `worldWidth`, `worldHeight`, `originX` / `originY` (0–1), `minPixelRatio`, `maxPixelCount` (shaders.paper.design/mesh-gradient; [packages/shaders/src/shader-mount.ts](https://packages.shaders/src/shader-mount.ts)).

Vanilla usage: `new ShaderMount(parentElement, fragmentShader, uniforms, webGlContextAttributes?, speed = 0, frame = 0, minPixelRatio = 2, maxPixelCount = 1920*1080*4, mipmaps = []).Methods: setUniforms, setSpeed, setFrame, setMaxPixelCount, setMinPixelRatio, getCurrentFrame, dispose`. Each shader exports a fragment shader string (e.g. `meshGradientFragmentShader`) whose uniforms are `u_`-prefixed (`u_colors` as `vec4[]` built with `getShaderColorFromString`, `u_colorsCount`, `u_distortion`, ...) plus sizing uniforms (`u_fit`, `u_scale`, `u_rotation`, `u_offsetX`, `u_offsetY`, `u_originX`, `u_originY`, `u_worldWidth`, `u_worldHeight`). Utility exports: `getShaderColorFromString`, `getShaderNoiseTexture`, `defaultObjectSizing`, `defaultPatternSizing`, `ShaderFitOptions` (raw.githubusercontent.com/paper-design/shaders/main/packages/shaders/src/shader-mount.ts; .../src/index.ts; .../src/shaders/mesh-gradient.ts).

Performance: the canvas sizes itself to the container with a `ResizeObserver`. Rendering pauses when the tab is hidden or the element is off-screen (v0.0.78). `speed 0` runs no animation loop, so a static shader costs nothing after the first frame. Low float precision is upgraded to highp on weak devices ([packages/shaders/src/shader-mount.ts](https://packages.shaders/src/shader-mount.ts); CHANGELOG.md).

How they work inside Paper - Open the shaders menu with S or the toolbar button, then click or drag a shader onto the canvas. Image filters then need an image uploaded from the right panel. Parameters are sliders with snap points and presets (► [Cloudflare shader 0:32](#); ► [Logo shaders 0:29](#); paper.design/blog/retro-print-cmyk-half-tone-shader; [build log, 22 Oct 2025](#)). - Logo-animation shaders (Heatmap, Liquid Metal, Gem Smoke) take an uploaded logo or shape (shaders.paper.design/liquid-metal; ► [Logo shaders](#), 7 Jul 2026). A dedicated "contour" explainer exists (► [Contour](#)). - Shaders can be chained: export a shader frame as JPG and use it as the input of another filter (paper.design/blog/retro-print-cmyk-half-tone-shader). - The eyedropper samples colors from shaders ([build log, 8 Oct 2025](#)). Shader colors and filters can be reordered by drag ([build log, Mar 2026](#)). - Video export of any frame (Pro), with a dedicated Video panel for shaders: MP4 is opaque, WebM transparent, duration adjustable ([build log, 8 Oct 2025](#); [build log, 22 Oct 2025](#); [build log, 26 Sep 2025](#); MCP schema). - A monthly "shader of the month" series

introduces new shaders, e.g. Lens Distortion (Aug 2026) and Paper Texture 2.0 (Sep 2026) (► [Lens Distortion](#); ► [New in Paper, Sep 4:06](#)). - MCP: no tool inserts or edits shaders (MCP schema). Whether get_jsx emits shader components: not found in official sources. - Related: github.com/paper-design/liquid-logo ("turn your logo into liquid metal with Paper Shaders") (github.com/paper-design).

Shader list (30 shaders on shaders.paper.design; React component names per page; all fragment shaders exported in `src/index.ts`) | Category | Shader (component) | Main shader-specific props | Source | |---|---|---|---| | Image filter | Paper Texture (`PaperTexture`) | image (optional), colorBack, colorPaper, colorShadow, blending, distortion, clip, angle, seed, roughness, roughnessSize, roughnessRows, fiber, fiberSize, folds, foldSizeX/Y, foldOffsetX/Y, wrinkles, wrinkleSize, crumples, crumpleCount (2–15), drops | shaders.paper.design/paper-texture | | Image filter | Fluted Glass (`FlutedGlass`) | image, colorBack, colorShadow, colorHighlight, shadows, highlights, size, shape (pattern, wave, lines, linesIrregular, zigzag), angle, distortionShape (prism, lens, contour, cascade, flat), distortion, shift, stretch, blur, edges, margin(s), grainMixer, grainOverlay | shaders.paper.design/fluted-glass | | Image filter | Water (`Water`) | image, colorBack, colorHighlight, highlights, layering, edges, waves, caustic, size (0.01–7) | shaders.paper.design/water | | Image filter | Image Dithering (`ImageDithering`) | image, colorBack, colorFront, colorHighlight, originalColors, inverted, type (random, 2x2, 4x4, 8x8), size (0.5–20), colorSteps (1–7) | shaders.paper.design/image-dithering | | Image filter | Halftone Dots (`HalftoneDots`) | image, colorBack, colorFront, originalColors, type (classic, gooey, holes, soft), grid (square, hex), inverted, size, radius (0–2), contrast, grainMixer, grainOverlay, grainSize | shaders.paper.design/halftone-dots | | Image filter | Halftone CMYK (`HalftoneCMYK`) | image, colorBack, colorC/M/Y/K, size, gridNoise, softness, contrast, type (dots, ink, sharp), floodC–K, gainC–K, grain controls | shaders.paper.design/halftone-cmyk | | Image filter | Lens Distortion (`LensDistortion`) | image, spread, bias, angle, perspective, count (2–50), dispersion, dispersionShift, dispersionColor, focusCenter, focusEdges, swirl, noise, noiseFrequency, noiseOffset, lensBulge, lensCircle, grain, imageX/Y | shaders.paper.design/lens-distortion | | Logo animation | Heatmap (`Heatmap`) | image, colors (≤ 10), colorBack, contour, angle, noise, innerGlow, outerGlow | shaders.paper.design/heatmap | | Logo animation | Liquid Metal (`LiquidMetal`) | image, colorBack, colorTint, shape (none, circle, daisy, diamond, metaballs), repetition (1–10), softness, shiftRed, shiftBlue, distortion, contour, angle | shaders.paper.design/liquid-metal | | Logo animation | Gem Smoke (`GemSmoke`) | image (optional), shape (none, circle, daisy, diamond, metaballs), colors (≤ 6), colorBack, colorInner, innerDistortion, outerDistortion, innerGlow, outerGlow, offset, angle, size | shaders.paper.design/gem-smoke | | Effect | Mesh Gradient (`MeshGradient`) | colors (≤ 10), distortion, swirl, grainMixer, grainOverlay | shaders.paper.design/mesh-gradient | | Effect | Static Mesh Gradient (`StaticMeshGradient`) | colors (≤ 10), positions, waveX, waveXShift, waveY, waveYShift, mixing, grainMixer, grainOverlay | shaders.paper.design/static-mesh-gradient | | Effect | Static Radial Gradient (`StaticRadialGradient`) | colors (≤ 10), colorBack, radius (0–3), focalDistance, focalAngle, falloff, mixing, distortion, distortionShift, distortionFreq, grainMixer, grainOverlay | shaders.paper.design/static-radial-gradient | | Effect | Dithering (`Dithering`) | colorBack, colorFront, shape (simplex, warp, dots, wave, ripple, swirl, sphere), type (random, 2x2, 4x4, 8x8), size (1–20) | shaders.paper.design/dithering | | Effect | Grain Gradient (`GrainGradient`) | colors (≤ 7), colorBack, softness, intensity, noise, shape (wave, dots, truchet, corners, ripple, blob, sphere) | shaders.paper.design/grain-gradient | | Effect | Dot Orbit (`DotOrbit`) | colors, colorBack, stepsPerColor (1–4), size, sizeRange, spreading | shaders.paper.design/dot-orbit | | Effect | Dot Grid (`DotGrid`) | colorBack, colorFill, colorStroke, shape (circle, diamond, square, triangle),

size (1–100), gapX/gapY (2–500), strokeWidth (0–50), sizeRange, opacityRange | [shaders.paper.design/dot-grid](#) | | Effect | Warp (**Warp**) | colors, proportion, softness, distortion, swirl, swirlIterations (2–20), shape (checks, stripes, edge), shapeScale | [shaders.paper.design/warp](#) | | Effect | Spiral (**Spiral**) | colorBack, colorFront, density, distortion, strokeWidth, strokeTaper, strokeCap, noise, noiseFrequency, softness | [shaders.paper.design/spiral](#) | | Effect | Swirl (**Swirl**) | colors (≤ 10), colorBack, bandCount (0–15), twist, center, proportion, softness, noise, noiseFrequency | [shaders.paper.design/swirl](#) | | Effect | Waves (**Waves**) | colorBack, colorFront, amplitude, frequency (0–2), spacing (0–2), proportion, softness, shape (0–3) | [shaders.paper.design/waves](#) | | Effect | Neuro Noise (**NeuroNoise**) | colorFront, colorMid, colorBack, brightness, contrast | [shaders.paper.design/neuro-noise](#) | | Effect | Perlin Noise (**PerlinNoise**) | colorBack, colorFront, proportion, softness, octaveCount (1–8), persistence (0.3–1), lacunarity (1.5–10) | [shaders.paper.design/perlin-noise](#) | | Effect | Simplex Noise (**SimplexNoise**) | colors, stepsPerColor (1–10), softness | [shaders.paper.design/simplex-noise](#) | | Effect | Voronoi (**Voronoi**) | colors (≤ 5), colorGlow, colorGap, stepsPerColor (1–3), distortion (0–0.5), gap (0–0.1), glow | [shaders.paper.design/voronoi](#) | | Effect | Pulsing Border (**PulsingBorder**) | colors, colorBack, roundness, thickness, softness, aspectRatio (auto, square), intensity, bloom, spots (1–4), spotSize, pulse, smoke, smokeSize, margins | [shaders.paper.design/pulsing-border](#) | | Effect | Metaballs (**Metaballs**) | colors, colorBack, count (1–20), size | [shaders.paper.design/metaballs](#) | | Effect | Color Panels (**ColorPanels**) | colors, colorBack, density (0.25–7), angle1, angle2, length (0–3), edges, blur (0–0.5), fadeIn, fadeOut, gradient | [shaders.paper.design/color-panels](#) | | Effect | Smoke Ring (**SmokeRing**) | colors, colorBack, thickness, radius, innerShape (0–4), noiseIterations (1–8), noiseScale | [shaders.paper.design/smoke-ring](#) | | Effect | God Rays (**GodRays**) | colors (≤ 5), colorBack, colorBloom, bloom, intensity, density, spotty, midSize, midIntensity | [shaders.paper.design/god-rays](#) | Unless noted, numeric ranges are 0–1.

When shaders were added (CHANGELOG / build log): Halftone Dots v0.0.64 ([build log, 19 Nov 2025](#)); Halftone CMYK v0.0.69 ([build log, 19 Jan 2026](#); blog 5 Feb 2026); Gem Smoke v0.0.73 ([build log, Apr 2026](#)); Lens Distortion v0.0.79 (Aug 2026, ► [New in Paper, Aug 8:11](#)); Paper Texture rework v0.0.81 ([build log, Sep 2026](#)). Liquid Metal was fully reworked on 22 Oct 2025 ([build log, 22 Oct 2025](#)) (raw.githubusercontent.com/paper-design/shaders/main/CHANGELOG.md).

Unofficial ports found by search (publisher not verified): a Flutter port [paper_shaders](#) (pub.dev/packages/paper_shaders); a Framer listing "Shaders by Paper" (framer.com/marketplace/plugins/1353); auto-generated docs at [mintlify.wiki/paper-design/shaders](#).

Snapshot

- **What it is:** a Chrome extension that copies part or all of a live web page into Paper as editable HTML/CSS layers instead of a screenshot. Production code becomes the starting point for mockups (chromewebstore.google.com/detail/lidfahaahiogmnlccifabccgplofocck; [paper.design/roadmap](#) lists it as shipped).
- **Install:** [paper.design/snapshot](#) redirects to the Chrome Web Store listing. Install it, then pin it from the browser's Extensions menu ([paper.design/snapshot](#), 307 redirect; [paper.design/snapshot-extension](#)). Listing details: publisher Lost Coast Labs, Inc.; v0.4.4, updated 27 Sep 2026; 105 KiB; about 30,000 users; 4.7/5 from 18 ratings (Chrome Web Store listing). Browsers other than Chrome: not found.

- **Use:** 1. Click the extension icon or press `⌘P` (remap at `chrome://extensions/shortcuts`). 2. Hover the target element; `↑/↓` adjust the selection. 3. Press `Enter` or click to capture, or `⌘Enter` for the whole page (since May 2026; progress shows as a percentage). 4. Press `⌘V` in Paper Web or Desktop (`paper.design/snapshot-extension`; [build log, May 2026](#); [► New in Paper, Jun 7:07](#); [► Resend button 0:20](#)). After pasting, an agent can iterate on the capture ([► Iterate on Snapshot, 2 May 2026](#)), and frames can be set to fit height with a double-click ([► Iterate on Snapshot 2:48](#)).
- **What is kept:** OpenType font features ([build log, May 2026](#)); word breaks were fixed in Sep 2026 ([build log, Sep 2026](#)); the rest is not documented.
- **Local images:** dev servers block cross-origin requests, so allow the origin `https://app.paper.design` in the dev server. The docs give settings for Astro, Django, Express, Fastify, Flask, Next.js (header in `next.config.js`), Nuxt, Rails (Rack::Cors), Vite/Remix/SvelteKit (`server.cors.origin`) and webpack (`devServer.headers`). Vite 6+ blocks `/@fs/` paths, so serve assets from `public/` (`paper.design/docs/support/snapshot-local-images`).
- **Plan requirement:** none mentioned (`paper.design/snapshot-extension`).
Release: announced 26 Mar 2026, in the April 2026 changelog ([alternativeto.net](#) news; [build log, Apr 2026](#)).

HTML and CSS that Paper accepts

Supported or recommended - Inline styles (`style="..."`) are the only styling input (MCP schema; [paper.design/docs/paste/html](#)). - Layout: flexbox, padding and gap are the core tools; flex wrap since Aug 2026, with per-axis wrap gap since Sep 2026; negative gap since Jun 2026 (MCP schema; [build log, Aug 2026](#); [build log, Jun 2026](#); [► New in Paper, Sep 3:22](#)). - Absolute positioning is fully supported. Use it for decoration and never cover the whole artboard with one absolute element, because that blocks the cursor (MCP schema). Elements can ignore flex layout (absolute option, Nov 2025), and a Constraints panel handles parent resizes (Mar 2026) ([build log, 19 Nov 2025](#); [build log, Mar 2026](#)). - Sizing: border-box everywhere; `fit-content` heights; fill and fit by double-click (MCP schema; [build log, Mar 2026](#)). - `display: block` is fine for simple text or decorative shapes, not for layout containers (MCP schema). - Colors: hex, rgb(a), hsl(a), oklch, oklab; sRGB and Display P3 side by side; `color-mix()` on tokens (MCP schema; [paper.design/compare/figma](#); MCP guide). - Design tokens as CSS variables, including inside SVG `fill` and `stroke` (MCP guide; MCP schema). - Fonts: all Google Fonts and locally installed fonts by family name (MCP schema). Variable font axes, automatic optical size, OpenType features ([build log, 19 Nov 2025](#); [build log, 19 Dec 2025](#); [build log, Mar 2026](#)). - Fills and effects: gradients, including text gradients and on-canvas editing; image fills composed with other fills; shadows, outlines, borders, blend modes; filters (blur, saturate, grayscale, brightness, sepia, invert, hue-rotate); backdrop filters; text stroke ([build log, 26 Sep 2025](#); [build log, 6 Nov 2025](#); [build log, 19 Dec 2025](#); [build log, 19 Jan 2026](#); [build log, Mar 2026](#); [build log, Apr 2026](#); [paper.design/compare/figma](#)). - Text formatting: casing, wrapping, truncation; CJK word-keeping ([build log, 19 Dec 2025](#); [build log, Sep 2026](#)). Code blocks use `<pre>` or `white-space: pre` as a single element with one color (MCP schema). - SVG: inline SVG, pasted SVG, path editing (MCP guide; [paper.design/docs/svg](#)). - Paper-specific attributes and elements: `layer-name="..."`, `data-paper-locked`, `hidden`, `<x-paper-clone node-id="...">`; local images through `paper-asset:///absolute/path`; AI images through `paper-gen://` (`paper.design/docs/paste/html`; MCP schema). - Images from `<img>` and `background-image` are uploaded into Paper when pasting HTML (`paper.design/docs/paste/html`). HEIC/HEIF are accepted ([build log, 19](#)

[Jan 2026](#)). - Styles that Paper renders but cannot edit in its panels appear in an "Other styles" panel ([build log, May 2026](#)).

Not supported or to avoid - `margin`, `display: inline`, `display: grid` and HTML tables: do not use them; use padding and gap (MCP schema). CSS Grid is "Planned" on the roadmap ([paper.design/roadmap](#)), although the Sep 2026 release added alignment and gap controls for grid elements ([► New in Paper, Sep 3:38](#)). - Classes, selectors, `<style>` tags and external stylesheets: class names are dropped and selector styles ignored ([paper.design/docs/paste/html](#)). - Rich text, meaning several styles in one text node: not supported yet; announced as coming soon ([paper.design/docs/paste/html](#); MCP schema; [► New in Paper, Sep 4:59](#)). When pasting, a block with inline children flattens into one text node, inline elements become inline-block, and inputs become frames with text children ([paper.design/docs/paste/html](#)). - Emojis as icons: use SVG instead (MCP schema). - Unresolved references (`var()` to tokens that do not exist in Paper, Tailwind classes) fail silently or break the output when importing (MCP guide). - Styles that do nothing in a node's context are dropped and reported in `ignoredStyles` (MCP schema). - Figma paste limits: components, instances and variables are detached; masks hidden; no rich text; no noise, texture or glass effects; no arbitrary dash lengths or gradient strokes; limited pass-through blending; no symmetry or repeat effects; diamond gradients become radial. Image import needs a connected Figma account and is subject to Figma API rate limits ([paper.design/docs/paste/figma](#)). - Pseudo-elements, CSS transforms, `<video>` / `<iframe>` in `write_html`: not documented ([paper.design/docs/paste/html](#)). - Behaviors seen in your sessions (not in the docs): `write_html` replace drops children's absolute positioning; empty divs are ignored; styled inline spans are ignored; backdrop-filter bleeds; `repeating-linear-gradient` and `transform: scale` did not render; rotation pivots on the top-left corner; native PDF export failed on report pages (your sessions).

Field notes from your own sessions

Behaviors your agents ran into while working in Paper between July and October 2026, collected from your project memories. They are not in Paper's videos or docs. Some may change with new releases, so each one carries the date it was observed.

Writing HTML into Paper

Behavior	Workaround that worked	Seen
<code>write_html</code> in replace mode drops <code>position: absolute</code> from the new root's direct children and turns the root into a flex container	Insert the new frame next to the old one (insert-children into the parent), then delete the old node	Sep 2026, Ballottage
An empty <code><div></code> is ignored	Create a frame together with its first child	1 Oct 2026
A styled inline <code></code> inside text is ignored	One text frame per differently styled word	1 Oct 2026
Many writes in a row end in "Your Paper file is currently disconnected from the server", and the writes are lost	Write block by block, pause 250 to 700 ms between writes, stop at the first error	1 Oct 2026
Re-typing long SVG paths by hand corrupts them (a status-bar Wi-Fi path broke this way)	Drive the MCP from a script that reads the HTML from files (local endpoint below)	27 Sep 2026
<code><x-paper-clone></code> of a whole artboard can collapse its responsive layout	Insert a wrapper with a 1 px witness child, <code>duplicate_nodes</code> into the wrapper, set the copy to absolute O/O, delete the witness	1 Oct 2026

Rendering

Works	Does not render (as of the date)
SVG <code><pattern></code> and <code>clip-path</code> ; <code>font-stretch</code> ; <code>zoom</code> on an <code>x-paper-clone</code> to scale a copy (1 Oct 2026)	CSS <code>repeating-linear-gradient</code> , <code>transform: scale</code> (1 Oct 2026)
Faked glass: white at 55%, top-to-bottom sheen gradient, inset white highlight, soft shadow (28 Sep 2026)	<code>backdrop-filter</code> bleeds past the element (28 Sep 2026)

- Rotation pivots on the top-left corner, so a swinging element needs an offset to turn around its real pivot (28 Sep 2026).
- An SVG `clip-path` on an element that also has a `transform` is read in the transformed space: put the clip on a parent `<g>` (Sep 2026).

Files, pages and exports

- `duplicate_nodes` returns a map of new ids (`duplicatedNodes` with `sourceId`, `newId`, `descendantIdMap`). A duplicated artboard lands in a free spot, so set `left` and `top` afterwards, and check the source ids still exist first, or you get orphan copies (Sep 2026).
- There is no page-delete tool in the MCP; pages are removed by hand (1 Oct 2026).
- `export` writes into `~/Downloads` and returns the path; it accepts fractional scales such as `0.25x` for very large boards (1 Oct 2026). Move each export out of Downloads at once.
- The native PDF export failed on the A4 report boards of the budget tool; PNG export worked (28 Sep 2026).
- To place annotations precisely on a screen, export it as PDF and read the text and shape boxes with PyMuPDF (PDF points $\times 4/3 = \text{px}$) (1 Oct 2026).

Image generation (`paper-gen://`)

- With `reference_nodes`, start the generations only once the reference images have finished loading; a batch sent seconds after creating the reference `<img>` nodes failed at once with a bare `error` status (Sep 2026).
- For likeness from photos, GPT Image 2.5 edit with one shared prompt and the same template kept heads equal in size and recognizable, and returned a true transparent background (27 Sep 2026).

Driving Paper from scripts

The Paper desktop plugin's MCP server also listens over Streamable HTTP at `http://127.0.0.1:29979/mcp`. A small client (POST `initialize`, keep the `Mcp-Session-Id` header, POST `notifications/initialized`, then `tools/call` with a `fileId`) calls the same tools as your agent, but reads long HTML from files so it lands byte for byte. It is also the way back in when the session's MCP bridge reports "not connected" because Paper was closed when the session started; restarting the agent session restores the official bridge. Print every text block of each response: validation notes arrive in a separate block.

Plans and pricing

	Free	Pro	Organizations
Price	\$0	\$20 per editor per month billed monthly, or \$16 per editor per month billed yearly ("save 20%")	Custom, contact sales
Editors	Unlimited, free	Paid seats	per contract
Viewers	Unlimited, free ("viewer seats are always free")	Unlimited	Unlimited
MCP tool calls	100 per week	1M per week	Pro features included
AI image generation	"Limited"	"100x more image gen per week" than Free	Pro features included
Max image size	25 MB	100 MB	100 MB
Video export	No	Yes	Yes
Other		Priority feedback	SAML/SSO, admin controls, custom contracts, dedicated support, priority onboarding

Source for the whole table:
[paper.design/pricing](#).

- Collaboration is free, and editor seats only become paid when a team upgrades to Pro (paper.design/pricing). Free teams have been able to invite unlimited editors since August 2026, with free viewer roles that have folder and file access ([build log, Aug 2026](#); ► [New in Paper, Aug 5:43](#), chapter "Free editors and viewer roles").
- Image generation usage per model is metered in "usage" units, 1 to 4 per image depending on the model (table in section 6) (MCP guide). A single tool call that authors more than 10 images, or would cost more than the remaining usage, is rejected as a whole (MCP guide).
- Weekly image-generation allowance in units: not found. Education or student discount: not found (paper.design/pricing). Pricing FAQ: not found on the page (paper.design/pricing).
- If limits do not reset after upgrading to Pro, update the Desktop app (About menu, Check for Updates) and restart (paper.design/docs/mcp).
- Third-party conflict, for the record: banani.co quotes "\$16/month or \$144/year" and "100x more image generations/day" (banani.co/blog/paper-design-mcp-review). The official page above takes precedence.

About Paper

What it is - A design tool whose canvas is real HTML and CSS: layers are DOM nodes with CSS styles, so designs export as code and agents read and write a language they already know (paper.design; [paper.design/compare/figma](#); Paper plugin, README.md). - Current tagline: "the connected canvas for teams shipping with agents" (paper.design). The marketplace manifest describes it as connecting teams, agents, code and data on one design space built on web standards (agent-plugins repo, .claude-plugin/marketplace.json). - Claimed differentiators: web-native rendering with no translation step (Figma uses a proprietary WebGL model); real CSS styles, filters and a shader

effects library; sRGB and Display P3 in the same file, with OkLCH/Oklab pickers; an agent-friendly format (paper.design/compare/figma). Against Pencil (a WebGL desktop app with a proprietary JSON format stored in git): Paper runs on web and desktop, is multiplayer, supports P3, variable fonts and OpenType (paper.design/compare/pencil). - Philosophy: the canvas is where humans supply spatial reasoning, intent and direction while agents handle repetitive work (paper.design/blog/a-real-space-to-design-in-the-age-of-agents, 27 Feb 2026).

Who makes it - Legal entity: Lost Coast Labs, Inc., doing business as Paper, 6789 Quail Hill Parkway #2017, Irvine, CA 92603; California law; terms last updated 30 Mar 2026 (paper.design/legal/tos). The Chrome Web Store lists the same publisher for Paper Snapshot (chromewebstore.google.com/detail/lidfahaahiogmnlccifabccgplofocck). - Founders named on the seed post: Stephen Haney, Vlad Moroz, Ksenia Kondrashova (paper.design/blog/seed). Haney (CEO) co-founded Modulz in 2018; the Modulz team built Radix UI; Modulz was sold to WorkOS in 2022, where he led product until 2024 (designerfounders.substack.com/p/paper-stephen-haney, 23 Jul 2026; accel.com/news/our-investment-in-paper-the-ai-native-design-space). Vlad Moroz is listed as a founder on the seed post; a web-search summary calls him "founding designer", which is unverified. He led Paper Mono (search result pointing to github.com/paper-design/paper-mono). Ksenia Kondrashova co-authored the shader posts (paper.design/blog/retro-print-cmyk-half-tone-shader). - Team: 12 people, fully remote, as of July 2026 (designerfounders.substack.com/p/paper-stephen-haney; paper.design/blog/series-a). One press article puts headquarters in San Francisco (cmswire.com/ai-news/design-startup-paper-raises-34m-to-take-on-figma-in-the-ai-agent-era). - Funding: \$4.2M seed led by Accel with Basecase, 25 Feb 2025; angels included Guillermo Rauch, Des Traynor, David Hoang and Adam Wathan (paper.design/blog/seed). \$34M Series A led by Accel and ICONIQ, 22 Jul 2026, \$38.5M raised in total; Designer Fund and Basecase also took part; angels included Michael Grinich, Anton Osika, and engineers and designers from Anthropic and OpenAI (paper.design/blog/series-a). - Traction claims: ARR grew 25x in the month after the Desktop launch; four consecutive months on Ramp's fastest-growing list; customers include Ramp, Lovable, Harvey (paper.design/blog/series-a), Vercel, PostHog, Quartr and Y Combinator (cmswire.com article above). Daily sales rose 6x after the Desktop launch (same article). - Infrastructure subprocessors: Cloudflare, MongoDB, Fly.io, Redis, WorkOS (auth/SSO), Stripe, PostHog, Sentry, Resend, Plain, ToDesktop (desktop packaging and updates); AI providers Anthropic, OpenAI, Google Gemini and Replicate. List updated 23 Mar 2026 (paper.design/legal/subprocessors).

Platforms | Platform | Facts | Source | ---|---|---| | Paper Web | Browser app at app.paper.design; sign-up at app.paper.design/sign-up. Since Sep 2026 it supports WebMCP for in-browser agents | paper.design; [build log, Sep 2026](#) | | Paper Desktop | macOS (ARM), Windows (x64), Linux (AppImage, Deb, RPM). Current version 0.5.15, released 1 Oct 2026 | [paper.design/downloads](#) | | Desktop internals | Electron app packaged by ToDesktop (bundle id `com.todesktop.2601167vjw8xe`, auto-updates from `download.todesktop.com/2601167vjw8xe`); hosts the local MCP server | `/Applications/Paper.app/Contents/Info.plist`; `/Applications/Paper.app/Contents/Resources/app-update.yml`; Paper Desktop app | | Desktop features | Tabs for several open files (Aug 2026); agents can work in background tabs | [build log, Aug 2026](#) | | Paper CLI | `~/ .paper/bin/paper`, version 0.0.1, one command: `mcp` ("Start the MCP stdio relay") | Paper CLI | | Mobile app | not found. Official route to design from a phone: Claude Code Remote Control driving the desktop MCP | [Remote Control](#) (31 Jul 2026) |

Dated milestones (the full changelog is in section 9) | Date | Milestone | Source

| |---|---|---| | 2018 | Haney co-founds Modulz (later sold to WorkOS, 2022) | designerfounders.substack.com/p/paper-stephen-haney | | 25 Feb 2025 | \$4.2M seed announced | [paper.design/blog/seed](#) | | 9 Sep 2025 | Paper Alpha opens for sign-ups (image generation, shaders, real flex, Copy as React, OKLCH picker, vectorize) | [build log, 9 Sep 2025](#) | | 24 Feb 2026 | Paper Desktop released, with the MCP server | [paper.design/blog/a-real-space-to-design-in-the-age-of-agents](#); [build log, Mar 2026](#) | | 26 Mar 2026 | Paper Snapshot announced (listed in the April build log) | alternativeto.net/news/2026/3/paper-snapshot-lets-users-import-live-sites-as-editable-html-and-css; [build log, Apr 2026](#) | | Apr 2026 | Teams and collaboration (Editor and Admin invites) | [build log, Apr 2026](#) | | May 2026 | Figma copy/paste, PDF export, agents can create and open files | [build log, May 2026](#) | | Jun 2026 | Tokens via MCP, pen tool, folders | [build log, Jun 2026](#); ► [New in Paper, Jun](#) (30 Jun 2026) | | 22 Jul 2026 | \$34M Series A | [paper.design/blog/series-a](#) | | Aug 2026 | Comments, unlimited free editors, desktop tabs | [build log, Aug 2026](#); ► [New in Paper, Aug](#) (12 Aug 2026) | | 24 Aug 2026 | Stdio MCP and Claude Desktop extension land in agent-plugins | [github.com/paper-design/agent-plugins/commits/main](#) | | Sep 2026 | One-click agent setup, image generation through MCP, opacity tokens, WebMCP, Paper Texture 2.0 | [build log, Sep 2026](#); ► [New in Paper, Sep](#) (28 Sep 2026) | | 1 Oct 2026 | Desktop 0.5.15 | [paper.design/downloads](#) |

Official resources

Resource	URL / location	Source
Docs home	paper.design/docs . Pages: MCP (/docs/mcp), Snapshot local images (/docs/support/snapshot-local-images), Tokens (/docs/tokens), Support/FAQ (/docs/support), Paste (/docs/paste, /docs/paste/figma, /docs/paste/html), SVGs (/docs/svg). /docs/shortcuts, /docs/export, /docs/shaders and /docs/comments return 404	paper.design/docs ; WebFetch 404s
Changelog	paper.design/build-log (Help > Release Notes)	paper.design; Paper Desktop app
Roadmap	paper.design/roadmap	paper.design
Blog	paper.design/blog : seed (25 Feb 2025), CMYK halftone (5 Feb 2026), agents essay (27 Feb 2026), Config (1 Jul 2026), design roles (7 Jul 2026), Series A (22 Jul 2026), operations hire (13 Aug 2026)	paper.design/blog
Comparisons	paper.design/compare/figma , paper.design/compare/pencil	paper.design
Apps	app.paper.design (web), paper.design/downloads and download.paper.design (desktop)	paper.design; paper.design/downloads
Snapshot	paper.design/snapshot (Chrome Web Store), guide at paper.design/snapshot-extension	section 8
Shaders	shaders.paper.design; github.com/paper-design/shaders ; npm <code>@paper-design/shaders</code> , <code>@paper-design/shaders-react</code>	section 7
Agent plugins	github.com/paper-design/agent-plugins ; Cursor marketplace cursor.com/marketplace/paper; Claude Desktop extension claude-mcpb.paper.design	agent-plugins repo, README.md; paper.design/docs/mcp
GitHub org	github.com/paper-design: shaders, paper-mono (SIL OFL 1.1 monospace font, about 1.2k stars), liquid-logo, agent-plugins, webmcp-agent-example, an opentype.js fork, google-fonts-scripts	github.com/paper-design; github.com/paper-design/paper-mono
Community	Discord (discord.paper.design), Slack (slack.paper.design), X (x.com/paper), YouTube (youtube.com/@paperdesign), Instagram (instagram.com/paper.design), Reddit (reddit.com/r/paperdesign), Contra (contra.com/company/paper_0ad2ab), community page paper.design/community	paper.design/community ; Paper Desktop app (Help menu)
YouTube catalogue	52 items (33 videos, 19 shorts), 5 Mar 2026 to 28 Sep 2026, including monthly "New in Paper" videos (30 Jun, 12 Aug, 28 Sep 2026), an "8 tips" video, flex/constraints tutorials, agent setup, Snapshot, shaders and tokens	<code>scratchpad/yt/catalogue.json</code>

Resource	URL / location	Source
Support	team@paper.design; FAQ and troubleshooting at /docs/support; desktop logs under Help > View Logs	Paper plugin, README.md; paper.design/docs/support ; Paper Desktop app
Legal	/legal/privacy, /legal/tos, /legal/dpa, /legal/subprocessors (plugins link paper.design/privacy)	paper.design; Paper plugin, README.md
Templates	No public template gallery found. In the app: frame presets (e.g. a YouTube thumbnail preset, Aug 2026) and an onboarding welcome file with a roadmap section (Sep 2025)	build log, Aug 2026 ; build log, 26 Sep 2025
Third party (unofficial)	Raycast extension raycast.com/asifk/paper-design; banani.co review; mintlify.wiki shader docs	web search results

Glossary and timeline

Every Paper feature, tool and integration the guide covers, A to Z, with the place that teaches it. The timeline after it dates each release, so you can tell how old a video's view of Paper is.

Glossary

Absolute position. Takes a child out of its flex parent's flow so it floats; pin it with constraints. [Canvas](#)

Agent loop. Guide, file info, fonts, brief, small writes, screenshot review, then finish_working_on_nodes. [Agents](#)

Aliases. Semantic tokens that point at palette tokens through `var()`; create the palette first. [Tokens](#)

Artboard. A top-level frame. The MCP defaults are 1440×900 , 768×1024 and 390×844 . [MCP tools](#)

Clip content (`\C`). Hides what overflows a frame; smoke and glow shaders need it on their parent. [Canvas](#)

code-to-design, design-to-code. The two agent skills shipped in the Paper plugin v0.1.0, dropped in v0.2.1. [The plugin's two skills](#)

Comments (C). Pins with replies, reactions and resolve. Agents can list, read and resolve them, not create them. [Snapshot and collaboration](#), [Agents](#)

Connect your agent. The button in the right panel's MCP section (September 2026) that installs a stdio connection for your agent. [Connecting an agent](#)

Constraints. Left, Right, Left + Right, Center or Stretch: how a non-flex layer behaves when its parent is resized. [Canvas](#)

Contour. The logo-shader setting for how far in from the edge the effect reacts. [Shaders](#)

Copy as PNG, Tailwind, React CSS (`\⌘C`, `\T`, `\R`). Copies a layer as an image or as code. [Code](#)

Create image (`\⌘I`), **Create SVG** (`\⌘J`). The AI generation panels, each with a model picker. [Vectors and images](#)

Deep selection. `⌘` + click selects inside frames, `⌘` + right-click lists every layer under the cursor, and an option makes hover go deep. [Canvas](#)

Display P3. The wide-gamut color space, a tab next to sRGB in the picker. [Tokens](#)

Export, Export combined PDF. Files land in Downloads; a combined PDF follows canvas position. [Canvas](#)

Extract colors (magic swatches). Right-click an image to get a frame of its colors as plain layers. [Tokens](#)

Eyedropper (I). Samples a color; in a shader input it captures any layer as an image. [Tokens](#), [Shaders](#)

Figma paste. `⌘C` in Figma, `⌘V` in Paper. Images need a Figma connection; components arrive detached. [Vectors and images](#)

Fixed, Fit, Fill. Sizing per axis. Double-click an edge for Fit, `\` + double-click for Fill. [Canvas](#)

Flex layout (`\A`). Paper's name for CSS flex: direction, gap, padding, alignment, wrap. [Canvas](#)

Folders. Nested folders in the dashboard, with Move to and search across folders (`⌘F`). [Canvas](#)

Frame (`\F`). Paper's container, the equivalent of a div, either freeform or flex. [Canvas](#)

Free editors. Unlimited free editors and viewers on the Free plan since August 2026. [Snapshot and collaboration](#)

get_selection. The MCP tool that returns your selection: the cheapest way to point an agent at something. [Agents](#)

Halftone Dots, Halftone CMYK. Image-filter shaders that also work on text. [Shaders](#)

Hide UI (`⌘.`), **minimize UI** (`\.`). Two clean views for reviewing a board. [Canvas](#)

Hover selection. Targets shallow frames by default since June 2026. [Canvas](#)

Image models. The models behind `paper-gen://`, with their cost in usage units. [Image models](#)

layer-name. The attribute agents use in `write_html` to name layers; semantic names make ports easier. [Code](#)

Lens Distortion. An image-filter shader from August 2026 with Default, Fisheye, Duotone and Smoke presets. [Shaders](#)

Liquid Metal, Heatmap, Gem Smoke. Logo animation shaders that take a shape as input. [Shaders](#)

Mobbin. Reference flows pulled onto the canvas through Mobbin's MCP and steered with comments. [Snapshot and collaboration](#)

Opacity tokens. September 2026; pair with a color token to give a transparent value. [Tokens](#)

Paper CLI. `~/paper/bin/paper mcp`, the stdio relay between your agent and Paper Desktop. [Your setup](#)

Paper MCP. The server inside Paper Desktop that lets agents read and write the open file. [Agents](#), [MCP tools](#)

Paper Shaders. WebGL shader layers in Paper, and the same code in `@paper-design/shaders` and `@paper-design/shaders-react`. [Shaders](#), [Shaders in code](#)

Paper Snapshot. A Chrome extension that copies a live page into Paper as editable layers (`\⌘P`, `⌘Enter` for the whole page). [Snapshot and collaboration](#)

Paper Texture. An image-filter shader, rebuilt in September 2026 with folds, wrinkles and crumples. [Shaders](#)

paper-gen://. The URL scheme that starts an AI generation inside `write_html`, `update_styles` or `create_artboard`. [Image models](#)

Pen tool (P). Draws paths: `⌘`-drag pulls curves, `\` breaks handle mirroring, Shift locks 45°. [Vectors and images](#)

Presets (shaders). Named starting values; clicking one overwrites your tuning. [Shaders](#)

Pulsing Border. The effect shader behind glowing buttons. [Shaders](#)

Quiver Arrow, Arrow Telos. SVG generation models; Telos is more detailed, slower and costs more. [Vectors and images](#)

Remote Control (/rc). Steers a Claude Code session, and Paper through it, from the Claude mobile app. [Agents](#)

Remote MCP. Announced in September 2026: agents working in Paper without Desktop open. [Snapshot and collaboration](#)

Remove background. Right-click an image; the cutout keeps the original frame size. [Vectors and images](#)

Selection colors. Lists every color in the selection with its count; edit there to recolor. [Tokens](#), [Code](#)

Stdio and HTTP. The two MCP transports; stdio through the Paper CLI is the current recommendation. [Connecting an agent](#)

Stroke position. Above fill or Below fill on a path. [Vectors and images](#)

Tabs. Several files open in Desktop; agents can work in background tabs (August 2026). [Agents](#), [Snapshot and collaboration](#)

Theme. A file's token set; Copy theme and Paste theme sit in the Paper logo menu. [Tokens](#)

Tokens. Named values (color, type, spacing, radius, container, breakpoint, opacity) used through `var()` . [Tokens](#)

Variable fonts. Exact weights and automatic optical size in the text panel's Variable font tab. [Tokens](#)

Variety pack. Sends one image prompt to four models at once. [Vectors and images](#)

Vectorize image. Right-click a flat image to turn it into an SVG. [Vectors and images](#)

WebMCP. In-browser agent support in Paper Web, September 2026, for partners. [Connecting an agent](#)

write_html. The MCP tool that turns HTML into layers; one visual group per call. [Agents](#), [Field notes](#)

x-paper-clone. An element inside write_html that clones an existing node instead of rewriting it. [Field notes](#)

Timeline

Dated from Paper's build log, blog and the videos' upload dates.

Build-log entries are monthly from March 2026 onward. The dated "New in Paper" videos give release-week dates.

Date	Highlights	Source
9 Sep 2025	Paper Alpha opens: image generation, shaders, real flex layout, Copy as React, OKLCH picker, vectorize	build log, 9 Sep 2025
19 Sep 2025	Radius panel with per-corner controls; rotate button; Shift + scroll horizontal; shader parameter changes (Mesh grain, Fluted Glass size, Swirl center/proportion, Water size); diacritics; AZERTY numeric keys fix	build log, 19 Sep 2025
26 Sep 2025	Text gradients through a new Fill panel; MP4 duration; Pulsing Border and Paper Texture parameter rework	build log, 26 Sep 2025
8 Oct 2025	Video export from any frame (Pro); Paper Mono preview; eyedropper on images and shaders; default export @2x; offline warning	build log, 8 Oct 2025
22 Oct 2025	Video panel for shaders; new sliders with snap points; export size/scale presets; WebP and AVIF export; Liquid Metal rework; arrow-key panning	build log, 22 Oct 2025
6 Nov 2025	Image fills composable with other fills; gradient angle field; Fluted Glass highlights/shadows/grain; Shift + eyedropper adds gradient stops	build log, 6 Nov 2025
19 Nov 2025	Variable fonts with custom axes; Selection colors panel; ignore-flex (absolute) option; auto line height; Halftone Dots	build log, 19 Nov 2025
3 Dec 2025	⌘-drag crop; Tailwind export from Copy as React; pressing 0 sets opacity to 100%	build log, 3 Dec 2025
19 Dec 2025	On-canvas gradient handles; text formatting panel (casing, wrapping, truncation); automatic optical size; Flux 2, Nano Banana Pro, OpenAI Image Edit 1.5, Seedream 4.5; ⌘R / ⌘T copy as code	build log, 19 Dec 2025
19 Jan 2026	Filter panel (blur, saturation, grayscale, brightness, sepia, invert, hue rotation); gap and padding handles on canvas; SVG colors in Selection colors; HEIC/HEIF; Halftone CMYK	build log, 19 Jan 2026
5 Feb 2026	Blog: CMYK Halftone shader	paper.design/blog/retro-print-cmyk-half-tone-shader
24 Feb 2026	Paper Desktop released	paper.design/blog/a-real-space-to-design-in-the-age-of-agents
Mar 2026	Desktop with MCP server; Constraints panel; backdrop filters; OpenType features panel; redesigned dashboard; Wrap in flex for several parents; ⌘ + double-click to fill parent; memory cuts	build log, Mar 2026

Date	Highlights	Source
5 Mar 2026	First MCP videos: Claude Code and Cursor setup; Notion content; shaders	► Sync with code , ► Notion content , ► Paper Shaders
26 Mar 2026	Snapshot announced	alternativeto.net news
30 Mar 2026	Terms of service updated	paper.design/legal/tos
Apr 2026	Snapshot; teams (Editor/Admin), view-only outside the team; GPT Image 2, Nano Banana 2, Quiver Arrow 1.1; ⌘⌘I / ⌘⌘J / ⌘⌘K; refreshed UI; Gem Smoke; agents can reorder and reparent layers (move_nodes); refined agent system prompt; text hotkeys; Stroke panel for text; H pan; RAM down 10–20%	build log, Apr 2026
May 2026	Figma copy/paste with SVGs and images; PDF export (single and combined) to Downloads; agents create and open files and change pages; page reorder; minimal UI; "Other styles" panel for agent-added styles; paste HTML; view-only mode, anonymous visitors; follow a cursor; cursor chat; Snapshot full page (⌘Enter); xAI images	build log, May 2026
Jun 2026	Tokens from MCP, CSS variables or canvas; copy tokens between files (File > Theme); pen tool (P) and path editing; folders and subfolders; shallow hover selection; ⌘ + right-click layer list; presence for teammates and agents; negative gap; better PDFs (shadows, gradients, blend modes); scroll-wheel zoom; ⇧L collapse layers; reduce agent animations option; Z fix for FR/DE keyboards	build log, Jun 2026 ; ► New in Paper, Jun (30 Jun 2026)
1 Jul 2026	Config SF campaign; Paper Mono announced as coming soon	paper.design/blog/paper-at-config-part-1
22 Jul 2026	Series A \$34M	paper.design/blog/series-a
Aug 2026	Comments (C) with resolve, search, filter, sort; unlimited free editors; free viewer roles; desktop tabs; agents across several files and background tabs; create tokens in-app; typography tokens in the font dropdown; Pro file-size limit raised; flex wrap; Figma slots on paste; YouTube thumbnail preset; tokens export correctly in SVG/PDF; I restored	build log, Aug 2026 ; ► New in Paper, Aug (12 Aug 2026) ; alternativeto.net (tabs, 12 Aug 2026)
24 Aug 2026	agent-plugins: stdio MCP and Claude Desktop extension (then Cursor and Codex fixes through 21 Sep)	github.com/paper-design/agent-plugins/commits/main
Sep 2026	Memory roughly halved, faster tabs and pages, smoother pan/zoom; one-click agent setup; copy as code in the Export panel; Paper Texture 2.0; WebMCP; add color tokens from any color field; opacity tokens; CJK word-keeping; image generation through MCP; dashboard search across folders; reworked flex panel (wrap gap per axis; alignment and gap for grid elements); HTML-in-canvas images, videos and exports; larger uploads on Pro	build log, Sep 2026 ; ► New in Paper, Sep (28 Sep 2026)
27 Sep 2026	Snapshot extension v0.4.4	Chrome Web Store listing
1 Oct 2026	Paper Desktop 0.5.15	paper.design/downloads
Announced next (28 Sep 2026)	Remote MCP (no Desktop needed), components on the canvas, scale tool, rich text editing, per-file permissions	► New in Paper, Sep 4:32
Roadmap (no dates)	In progress: code components, native Tailwind, themes/tokens with calc and color-mix, sharing settings, pen and vector editing, more shaders, asset hosting. Coming soon: components with slots; icon packs, shadcn and kits (with Base UI). Planned: CSS Grid, script/prompt engine, right-click Remix, video generation, Lottie/Rive/YouTube embeds, LLM search, particles, advanced image filters, three.js islands	paper.design/roadmap
Vector roadmap	Near term: snapping and guides, crop/scale/resize, shape to path, boolean operations. Later: mirroring, repeating, vector networks, anchoring, brush, shape builder	paper.design/docs/svg
Tokens roadmap	Theme classes that bundle typography; multiple theme modes (dark, compact)	paper.design/docs/tokens

Video library

Every video on the channel, newest first. The search covers titles, summaries and the features each video shows. The three bars on the right rate how useful a video is for your work; the reason sits under its summary. Timestamps open the video at that moment.

28 Sep 2026
5:40
Video

New in Paper: faster performance, agent reliability, opacity tokens, and more

September 2026 changelog: half the memory and faster tabs; a 'Connect your agent' dialog for one-click MCP setup across ten harnesses; image generation via MCP (a four-model 'variety pack'); opacity tokens paired with color tokens, color tokens created from fills, token picker views; dashboard search across folders; a reworked flex panel; Copy as PNG/Tailwind/React CSS from the Export section; HTML in Canvas exports; a rebuilt Paper Texture shader. Coming soon: remote MCP, components on the canvas, scale tool, rich text editing, per-file permissions.

For you. The September release: one-click agent setup over studio, opacity tokens for your glass recipe, image generation through the MCP, faster tabs, and what is next (remote MCP, components).

- ▶ 0:29 Performance improvements ▶ 0:54 Connect your agent dialog
- ▶ 1:09 Add to GitHub Copilot, enable Paper in MCP list ▶ 1:31 Image generation via MCP: variety pack
- ▶ 1:52 Opacity tokens paired with color tokens ▶ 2:19 Color token from a fill

High for you
Snapshot
Intermediate

24 Sep 2026
0:55
Short

What can Opus 5.5 design in five minutes?

After Anthropic's benchmark table (Opus 5.5 ahead of Fable 5.1 on every row shown), a dictated prompt in the Claude desktop app (Opus 5.5, 'Extra' effort) asks for as many good meditation-app home screens as possible in five minutes. The agent names six mood directions (Tidal, Candlelit, Alpine Swiss, Chapel, Editorial pop, Mineral), creates six 402×874 artboards ('01 Tidal — Device (402×874)' etc.), then builds breadth-first (headers on all, then heroes on all). Log shows 'Used 18 tools (6 failed)'. All six finished in just under six minutes.

For you. The Opus your agents run on, building six named directions breadth-first. Borrow that order for Ballottage explorations, and note the failed parallel calls: the same risk as your rapid-write disconnects.

- ▶ 0:02 Benchmark table: Opus 5.5 vs Fable 5.1 ▶ 0:08 Dictated prompt in Claude desktop composer
- ▶ 0:26 Six named mood directions announced ▶ 0:28 Breadth-first build; 6 of 18 tool calls failed
- ▶ 0:30 Six artboards created, filling in ▶ 0:36 All six done in just under six minutes

Medium for you
Agents
Intermediate

23 Sep 2026
0:42
Short

Use Mobbin and Paper to design flows from proven patterns

An agent pulls reference flows from Mobbin's MCP onto the canvas as labelled boards ('Ref · <product> - <flow>'); the designer culls them and pins comments, then prompts (Opus 5, High) with Paper links to the references and the design-system boards to create five variations. The agent writes rows such as 'V1 · Drawer', 'V2 · Toast', 'V3 · Popover', and the loop repeats with comments before manual edits.

For you. Reference flows on the canvas, steered by comments, then variations in labelled rows: a good start for new Ballottage flows.

- ▶ 0:00 Hook over finished Kale designs ▶ 0:06 References from Mobbin's MCP on the canvas
- ▶ 0:10 One board per product and flow ▶ 0:14 Reviewing reference screens ▶ 0:16 Comment pinned for the agent
- ▶ 0:18 Prompt with links: create 5 variations

Medium for you
Snapshot
Intermediate

21 Sep 2026
2:09
Short

How the Skydive team ships with their agents in Paper

Skydive designers create a design agent ('Tallulah, 'UX Design Expert'), train it on their Paper design system via a file link, and ask it in Slack to draft a partnership page artboard and a dark version; they polish it by hand in Paper. A second agent (Moon) receives the design from Tallulah in a Slack group DM and publishes it to Webflow, where the live page matches the Paper design, animation included. How Skydive connects to Paper is not shown.

For you. A different stack, but the split between a design agent and a build agent, with a Paper link as the handoff and the design system read first, is how your design panels could hand over to implementation.

- ▶ 0:18 Skydive Create a new agent dialog
- ▶ 0:30 Train agent with design-system link in Paper
- ▶ 0:43 Slack request: draft a collab artboard
- ▶ 0:53 Follow-up: dark mode
- ▶ 1:00 Manual pixel-perfect edits in Paper
- ▶ 1:17 Group DM: design handed to build agent

Medium for you
Code
Intermediate

18 Sep 2026
1:12
Short

Paper is the design tool for teams building with agents

Pitch that doubles as the read/write model: the canvas is rendered with code so agents read it (reference images, pasted drafts, your manual tweaks as visual context) and write it (first drafts, many variations, boilerplate like applying styles and renaming layers). B-roll prompts: list best practices from reference images; check what I pasted (a frame 'pasted from figma'); apply my changes to the rest of the flow; make a login screen from my style guide; explore 10 variations of my selection. Agent log shows reading the user's selection and a 12,108-node, 56-artboard file; layers renamed into a semantic tree. Vertical 1080×1920, filed by YouTube as a regular video.

For you. Short, but full of prompt patterns for today: "my selection" instead of describing a board, one hand fix propagated across a flow, references first, and a layer clean-up before porting.

- ▶ 0:08 Canvas rendered with code: agents read and write it
- ▶ 0:16 Agent reads a large file and the user's selection
- ▶ 0:24 Read: list best practices from references
- ▶ 0:26 Read: check what I pasted from Figma
- ▶ 0:28 Read: apply my changes to the rest of the flow
- ▶ 0:38 Write: login screen from my style guide

Medium for you
Agents
Beginner

18 Sep 2026
6:09
Video

How to bring your components from code onto the Paper canvas

Harcourt shows two pre-native-components routes to get code components into Paper: an agent connected to the repo rebuilds a labelled component sheet and, unasked, imports and binds the codebase's tokens; or Paper Snapshot copies elements or whole pages from the live site as editable layers with no agent or tokens. She composes a For You section with real API data, tries a smaller-card variant on a duplicate, and has the agent add the final frame to the Explore page, where the reused components keep their behaviour.

For you. Altitude and Water are live, so rebuilding their real components and tokens on each file's components page keeps Paper aligned with production. The variant-on-a-duplicate pattern fits your improve-don't-regress rule.

- ▶ 0:00 Components on canvas coming soon; today's workaround
- ▶ 0:14 Live site's reusable code components
- ▶ 0:28 Prompt: pull relevant components into Paper
- ▶ 1:16 Tokens imported and bound automatically
- ▶ 1:33 Paper Snapshot route, no agent or tokens
- ▶ 2:16 Build For You section with real API data

High for you
Code
Intermediate

3 Sep 2026
0:14
Short

From design to code in Paper

A 14-second vertical teaser cut from 'Push designs in Paper back to code': the code-to-canvas prompt, the redesigned Learn artboard with the reused Search component, the canvas-to-code prompt beside the running page, and the end card. No information beyond the full video.

For you. A 14-second teaser for the push-to-code video. Watch that one instead.

- ▶ 0:00 Code-to-canvas prompt in Cursor
- ▶ 0:02 Redesign with reused Search component
- ▶ 0:04 Canvas-to-code prompt and running result
- ▶ 0:12 paper.design end card

Low for you
Code
Beginner

2 Sep 2026
4:16
Video

Push designs in Paper back to code

Harcourt pulls Paper's Learn page into Paper with a one-line Cursor prompt, redesigns it with categories, a reused Search component and a scrolled-state artboard, then prompts Claude Opus 5 (High, Fast) to build the redesign frame into index.tsx, keep the designed scroll behaviour and reuse existing components. The agent asks questions, reuses the app's sticky-header pattern and SearchField, and the page works locally. She also shows Copy as Tailwind (␣T) and Copy as React CSS (␣R) on a ⌘-selected nested layer and explains the no-translation-layer advantage.

For you. You port your boards to code yourself (969 get_jsx calls). The four-part prompt (frame, target file, behaviour to keep, reuse existing components) and component-sized Copy as Tailwind apply to Altitude, Water and the budget tool.

- ▶ 0:15 Learn page running locally
- ▶ 0:24 One-line prompt: build the Learn tab in Paper
- ▶ 0:29 Redesign plus scrolled-state artboard
- ▶ 1:04 Prompt: build the redesign frame into code
- ▶ 1:23 Reuse components; effort High, Fast mode
- ▶ 1:37 Agent asks clarifying questions

High for you
Code
Intermediate

29 Aug 2026
0:26
Short

Remove image backgrounds in Paper

Right-click an image > Remove background: after a couple of seconds the subject is transparent inside the same 1731 x 2181 frame, then shown over other backgrounds and cropped inside a YouTube thumbnail frame. The captured right-click menu lists every image action and shortcut, including Vectorize image, Extract colors and Open image in new tab (not demonstrated).

For you. Cut out Ballottage figurines and social-post subjects in two clicks instead of regenerating them with transparency. It is a manual step: no MCP tool removes backgrounds.

- ▶ 0:01 Built-in background removal
- ▶ 0:06 Right-click > Remove background
- ▶ 0:08 Processing, frame 1731 x 2181
- ▶ 0:10 Transparent result
- ▶ 0:12 Cutout over other backgrounds
- ▶ 0:16 Placed and cropped in a thumbnail frame

Medium for you
Vectors
Beginner

26 Aug 2026
0:35
Short

Paper: the connected canvas for teams shipping with agents

YC Startup Podcast clip: Paper is agent-native because HTML and CSS are its rendering engine, which agents know well; custom rendering engines mean higher token spend, slower runs and lower accuracy. Team of about twelve designers and engineers. Animated b-roll log: get_selection, a code-file read (labelled read_file), a live API fetch (10 tracks), create_artboard, one write_html per row, set_text_content, then 'now generate the mobile screens' creating 'Mobile / Home', 'Mobile / Playlist', 'Mobile / Library' at 390×844.

For you. Mostly a pitch, but it explains why write_html works so well, and why CSS without a Paper equivalent loses fidelity.

- ▶ 0:00 What is Paper?
- ▶ 0:02 Agent log: selection, code component, live data
- ▶ 0:04 HTML and CSS are the rendering engine
- ▶ 0:06 Rows written one write_html call at a time
- ▶ 0:10 Three 390×844 mobile screens generated
- ▶ 0:14 Custom engines: more tokens, slower, less accurate

Low for you
Agents
Beginner

24 Aug 2026
0:50
Short

Design first, prompt the animations after

Podcast clip in which Stephen Haney explains that Paper's brand designer built the whole paper.design site in Paper in about a week, shaders and Easter eggs included, then prompted the animations on top of the Paper file while an agent turned it into a Next.js codebase. On-screen footage shows Codex reading a Paper selection through get_selection, get_screenshot, get_tree_summary and get_computed_styles before writing a component.

For you. Backs your Paper-first habit: keep screens static in Paper and prompt the motion in code, which also sidesteps the transforms that do not render on the canvas.

- ▶ 0:01 Website designed in Paper in about a week
- ▶ 0:10 Shaders: design you would not prompt
- ▶ 0:18 Animations prompted on top, Next.js codebase
- ▶ 0:20 Codex MCP calls on screen
- ▶ 0:35 Easter eggs: interactive parameters
- ▶ 0:40 Extras there would be no time for otherwise

Medium for you
Code
Beginner

21 Aug 2026
0:35
Short

Paper shader of the month: Lens Distortion

A recut of the August 2026 changelog's shader of the month: Lens Distortion, a new image filter that bends geometry and splits color channels like a camera lens. Content is pulled in with the eyedropper, and the short shows it works on any layer, including text, with five variants of one headline.

For you. A lens look for social-post headlines or a Ballottage splash. Keep the values small: the chromatic effect clashes with deck-grade dashboards.

- ▶ 0:01 Shader of the month: Lens Distortion
- ▶ 0:04 Shaders picker, Image filters section
- ▶ 0:06 Presets and the Image input
- ▶ 0:10 Parameters bend geometry and shift color channels
- ▶ 0:14 Negative Lens bulge, Scale and Position
- ▶ 0:18 Works on any layer, including text

Medium for you
Shaders
Beginner

20 Aug 2026
0:54
Short

Comment, reply, and resolve in Paper

Cut-down of the August comments segment: press C or use the toolbar button to comment anywhere, reply, react with emoji, resolve with the check icon, and use the right-panel list (comment mode only) with search, Newest first / Oldest first / By page sorting, and filters by person, page and resolved status.

For you. How to comment, reply and resolve, and the panel's filters: the human half of the comments-for-agents workflow.

- ▶ 0:00 Comments for team and agent
- ▶ 0:06 Comment button, shortcut C
- ▶ 0:10 Leave a comment
- ▶ 0:16 Reply to a teammate
- ▶ 0:22 Emoji reaction
- ▶ 0:28 Resolve

Medium for you
Snapshot
Beginner

19 Aug 2026
0:27
Short

Leave comments for your agent in Paper

Comments work for agents too: pin comments on layers, then tell the agent you left comments on the frame and to implement them. A comment pinned on a 'Research Agent' card asks for another card; the agent adds an 'Ops Agent' card in matching style and, per narration, fixes the other commented issues in the same pass. Claude, Cursor and OpenAI logos shown. Whether threads are resolved afterwards is not shown.

For you. You almost never use comments (7 list calls, no thread ever resolved). Pinning your notes on exact elements and letting the agent resolve them removes the guesswork on large boards. The MCP reads and resolves threads but cannot create them.

- ▶ 0:00 Team comments around the presenter
- ▶ 0:04 Works with Claude, Cursor, OpenAI agents
- ▶ 0:08 Prompt: I left comments on this frame, implement
- ▶ 0:12 Comment pinned on a card: add another card
- ▶ 0:16 Agent adds the Ops Agent card
- ▶ 0:20 Other commented problems also handled

High for you
Agents
Beginner

19 Aug 2026
13:53
Video

Paper and Conductor: design to code with parallel AI agents

Harcourt and Conductor's CEO capture reference pages with Paper Snapshot, ask an Opus 5 agent for three iterations of a subway-map roadmap, then run several agents at once: one restyling the same Paper file to Geist and dark mode while the first still works, another creating and naming a new Paper file for a musical version. A fresh workspace recreates the chosen frame as a standalone HTML page served locally (get_screenshot, get_jsx, headless screenshot comparison), and they show feedback via Conductor diff comments and Paper canvas comments.

For you. Several agents writing to one file and across files, close to your design panels: name the file in every prompt, fix the fonts in the brief, check ports against the Paper frame by screenshot, steer with comments.

- ▶ 0:32 New Conductor project and workspace
- ▶ 0:53 Agent lists its MCPs, Paper first
- ▶ 1:21 Paper Snapshot of reference pages
- ▶ 2:26 Workspaces are isolated codebase copies
- ▶ 2:40 Prompt: three subway-map iterations
- ▶ 4:26 Agent designs by calling write_html

High for you
Code
Advanced

12 Aug 2026
9:23
Video

New in Paper: tabs, comments, free editors, and more

August 2026 changelog: Series A; tabs in the desktop app with two Cursor agents (Opus 5 High) working in two files while presence shows on tabs, frames and avatars; Theme-tab token creation and editing with live propagation; flex wrap; Option-drag plus ⌘D repeating rotation; dashboard multi-select, file Duplicate and higher Pro limits; unlimited free editors and free viewer roles; comments for teammates and agents with a filterable panel; Lens Distortion shader; personal drafts announced next.

For you. The August release: comments, tabs, free editors, flex wrap, duplicate from the dashboard. Several of the ten changes for you start here.

- ▶ 0:15 Series A announcement
- ▶ 0:29 Tabs: '+' in the tab bar
- ▶ 1:04 Agent prompt naming the file
- ▶ 1:44 Second agent: scale frame 3x with ⌘L link
- ▶ 2:22 Theme tab: create tokens
- ▶ 2:58 Edit a color token; it propagates

High for you
Snapshot
Intermediate

11 Aug 2026
0:33
Short

Tabs, now in Paper

Silent 33-second motion promo for tabs: a fixed Dashboard tab plus file tabs, '+' for new tabs, animated dots on tabs and dotted markers on frames while agents work, a Claude Code panel holding a two-option dashboard spec beside wireframe cards, an agent dock, mixed human and agent presence avatars, and A/B dashboard variants.

For you. Thirty seconds on tabs; the August release video covers them fully.

- ▶ 0:00 Paper app opened
- ▶ 0:02 Title card with Dashboard and file tabs
- ▶ 0:06 '+' opens a new tab
- ▶ 0:08 Tab with animated working dots
- ▶ 0:10 Claude Code panel and agent dock
- ▶ 0:12 Many tabs working at once

Low for you
Snapshot
Beginner

10 Aug 2026
2:57
Video

Using tabs to run agents across multiple Paper files at once

Tabs (August 2026) let agents work in any open file. In Conductor (Opus 4.8 1M, Medium), three agents target three files: by file name, by pasted file link (Copy link ⌘L; the agent loads the guide and calls open_file), or 'in the active Paper file' (resolved when the agent starts). Per-tab dot indicator and robot avatar show where agents work; an agent reads tokens on a Tokens page and applies them on a Design page of the same file. Agent reasoning reveals find_nodes matches substrings (#000000 false positives on 0 values) and SVG strokes need separate checks. Tab menu: Reload, Copy Link ⌘L, Rename, Open in Browser, Move to New Window, Close Tab, Close Other Tabs, Close Tabs to the Right, Close All Tabs.

For you. Run one agent per file in parallel by putting the file link in every prompt. The find_nodes substring caveat matters for your recolors, and parallel writes raise the disconnect risk you already know.

- ▶ 0:30 Prompts must name the target file
- ▶ 0:42 Agent 1: five variations in the Learn Section file
- ▶ 0:52 Agent indicator stays on its tab
- ▶ 1:04 Agent 2: target by pasted file link
- ▶ 1:32 Agent 3: 'in the active Paper file'
- ▶ 1:55 Agent hops between Tokens and Design pages

High for you
Agents
Intermediate

31 Jul 2026
3:00
Video

Use Claude's Remote Control to design in Paper from your phone

Start Remote Control from a terminal Claude Code session (not the desktop app) with /remote-control or /rc; keep the Mac awake via System Settings > Lock Screen display-off timers 'For 3 hours'; open the Connected session in the Claude mobile app's Code tab. From a car, a dictated prompt duplicates an email frame in Paper and fills it with copy from a private Slack canvas and images matched by layer name; review happens through screenshots the agent returns. A follow-up makes a dark mode on a duplicated frame using the codebase's CSS variables. On-screen email lists Paper's tips: prompt in small chunks, use the codebase as token source of truth, have the agent re-read the canvas after manual edits. Session: Claude Code v2.1.220, Opus 5 (1M context), Claude Max.

For you. Review and steer long Paper runs away from the desk. The reminder that agents do not see your manual edits until they re-read the board fits your hand-tweak-then-agent rhythm.

- ▶ 0:11 Start remote session in the terminal with /rc
- ▶ 0:29 Phone notification: session ready
- ▶ 0:38 Keep the Mac awake (Lock Screen, 3 hours)
- ▶ 0:46 Claude mobile app, Code tab, Connected session
- ▶ 1:02 Dictated prompt: duplicate frame, fill from Slack canvas
- ▶ 1:48 Review via returned screenshots; Paper's prompting tips

High for you
Agents
Intermediate

21 Jul 2026
0:34
Short

Create magic swatches in Paper

Magic swatches: paste a photo into Paper (Cmd+V), right-click > Extract colors, and Paper adds a 'Magic Swatches' frame under the image with six swatch layers named 'Color #HEX', ready to use in the UI. The same context menu also offers Vectorize image, Remove background and Open image in new tab.

For you. Palette from a photo for social posts, or a check of Ballottage figurine renders against the screen palette. Promote the picks to tokens: swatches are plain layers.

- ▶ 0:06 Browsing Unsplash for an image
- ▶ 0:12 Copy the image in the browser
- ▶ 0:14 Paste into Paper with Cmd+V
- ▶ 0:16 Image context menu
- ▶ 0:18 Extract colors
- ▶ 0:22 Magic Swatches layers named by hex

Medium for you
Tokens
Beginner

20 Jul 2026
12:29
Video

Flex layout in Paper

Harcourt Allen teaches flex layout from zero (part one of a two-part layout series). Paper uses the CSS term flex because it is built on HTML/CSS and agents and developers share that vocabulary; frames are divs. She builds a menu button and a sidebar step by step: Wrap in flex (⇧A), four ways to set gap, token fill, axis or individual padding, direction, nine-point alignment, Fit via edge double-click, Space between (sliders menu or grid double-click), nested frames to split alignment, Fixed/Fit/Fill sizing (Relative listed but unexplained), Option + double-click edge for Fill, a parent sized by its child, a responsive split-view dashboard, and Absolute position for a close button that still needs constraints.

For you. Confirms that Paper's flex panel is CSS flex, the language your agents write. The sidebar and split-alignment patterns fit Poseidon and Altitude, Fixed and Fill map onto your 4 × 352 grids, and the layer icons expose freeform frames left by agents.

- ▶ 0:13 Why Paper says flex: shared CSS vocabulary with agents and developers
- ▶ 0:29 Frames, children, siblings; frames are divs
- ▶ 1:22 Wrap in flex button and ⇧A
- ▶ 2:00 Gap: four ways to set it
- ▶ 3:17 Padding per axis and individual padding
- ▶ 3:57 Direction row or column, order preserved

High for you
Canvas
Beginner

20 Jul 2026
3:31
Video

Constraints in Paper

Part two of the layout series. Constraints govern non-flex items (children of freeform frames and absolute children of flex frames) when the parent is resized. The UI is a horizontal dropdown (Left, Right, Left + Right, Center, Stretch), a vertical dropdown and a pin diagram. The close button defaults to Left/Top and is pinned Right to hold its corner; a modal centered with the Layout header's align buttons gets Center/Center automatically; Left + Right keeps fixed side margins (500 px) while the width grows, and Stretch scales both and shows the width as a percentage (36%).

For you. The fix for Ballottage overlays (floating bars, badges, figurines on a floor) and the context for replace mode dropping absolute positions. Less central to flex-built dashboards.

- ▶ 0:02 When constraints apply
- ▶ 0:27 Absolute child defaults to Left/Top and drifts
- ▶ 0:38 Constraints UI: dropdowns and pin diagram
- ▶ 1:05 Pin right keeps the close button top-right
- ▶ 1:42 Align center sets Center/Center automatically
- ▶ 2:30 Left + Right keeps 500 px margins

Medium for you
Canvas
Intermediate

20 Jul 2026
0:55
Short

Apply a Halftone Shader to Text

Halftone Dots applied to a letter: add a shader (tooltip Shaders S), pick Halftone Dots, use the Image eyedropper on a text layer, set Type to Gooley, scale to fit, explore Size/Radius/Contrast on a Hex grid, then recolor Foreground (yellow) and Fill (olive).

For you. Halftone letterforms for social posts and Ballottage share stickers, recolored to your palette. Not for dashboards.

- ▶ 0:01 Halftone dots on text
- ▶ 0:04 Toolbar button, tooltip Shaders S
- ▶ 0:06 Halftone Dots added (sample flower image)
- ▶ 0:10 Image eyedropper grabs the letter S
- ▶ 0:14 Type: Gooley
- ▶ 0:16 Scale up so the letter fits

Medium for you
Shaders
Beginner

9 Jul 2026
0:56
Short

Understanding contour in Paper's logo animation shaders

Vertical recut of the Contour explanation from the logo shaders video: each pixel knows its distance to the shape's edge, and Contour sets how far in pixels react. 18% hugs the rim, 0% is a plain mask with no edge reaction, 100% makes deep pixels follow the edges too (shown with Liquid Metal on a triangle).

For you. Contour is the key setting for an animated mark such as the Ballottage icon. Same footage as the logo shaders video, so either one is enough.

- ▶ 0:01 Contour, demonstrated with Liquid Metal
- ▶ 0:06 The shader reacts to the edges of the shape
- ▶ 0:12 Every pixel knows its distance to the edge
- ▶ 0:20 Contour 18%: only edge pixels react
- ▶ 0:28 Contour 0%: no edge reaction, like a mask
- ▶ 0:38 Contour 100%: deeper pixels react

Medium for you
Shaders
Beginner

7 Jul 2026
9:05
Video

How to use logo animation shaders in Paper

Harcourt Allen announces Paper Shaders are open source and free, then covers the three logo animation shaders (Heatmap, Liquid Metal, Gem Smoke), which react to a shape picked with the eyedropper, uploaded, or chosen from presets. She tunes Heatmap on the Paper logo, explains Contour with Liquid Metal on the Vercel logo, and places a faint Gem Smoke inside Resend's real footer captured with Paper Snapshot (Cmd+V, Shift+A, Absolute position, Clip content).

For you. Animated marks for Ballottage (splash, loading), testing a shader inside a live page through Snapshot, and the open-source licence for shipping it. The absolute shader plus Clip content structure is exactly what replace mode can break.

- ▶ 0:02 Paper Shaders are open source and free
- ▶ 0:37 Shaders menu (button or S), Logo animations section
- ▶ 0:58 Default diamond; upload or eyedrop a canvas shape
- ▶ 1:13 Noise grain, Angle, black and white recolor
- ▶ 2:02 Scale down; Inner glow vs Outer glow
- ▶ 3:08 Liquid Metal: preset shapes, then the Vercel logo

High for you
Shaders
Intermediate

30 Jun 2026
8:18
Video

New in Paper: tokens, vector editing, Figma import, and more

Paper's June 2026 changelog. On tokens, a Cursor agent imports color and typography CSS variables, and its log shows two rules: palette tokens must exist before semantic aliases, and Paper rewrites Tailwind's double-dash names (--text-xs--line-height becomes --text-xs-line-height). Token types mirror Tailwind, and themes and modes were in progress. Also covered: SVG path editing, the Cmd + right-click layer picker, Collapse all, panel resizing and page reordering, cursor chat and follow mode, shaders, PDF export (separate or combined, saved to Downloads), Hide UI (Cmd+.) and minimize UI (Shift+.), Figma copy and paste, nested folders, MCP tools to create and open files, and full-page Paper Snapshot (Cmd+Return).

For you. The token ordering and naming rules matter for your Tailwind v4 imports, and it touches your friction points: combined PDF, the layer picker, Selection colors. The rest belongs to other chapters.

- ▶ 0:25 Tokens via MCP; creation order and name normalization
- ▶ 0:42 Token picker on a rectangle
- ▶ 0:54 Token types map to Tailwind; themes and modes in progress
- ▶ 1:16 SVG path editing
- ▶ 1:46 Cmd + right-click layer picker
- ▶ 1:58 Collapse all, panel resizing, page reordering

Medium for you
Tokens
Beginner

26 Jun 2026
0:52
Short

Create Design Tokens From Your Code Base

A vertical cut of the tokens tutorial. One Cursor prompt creates 96 tokens from the codebase's CSS variables through Get Guide, Get Basic Info, Get Tokens and Create Tokens, with the approval card visible. In Paper, the Fill row's 'Color tokens' button lists them, and blue-1 then blue-3 are applied.

For you. The CSS-variables-to-tokens video compressed to 52 seconds. A refresher only.

- ▶ 0:08 Cursor with the Paper repo and a New Agent panel
- ▶ 0:12 Prompt typed: tokens from the CSS variables
- ▶ 0:20 Tool calls; Create Tokens awaits approval
- ▶ 0:28 Summary: 96 tokens by category
- ▶ 0:32 'Color tokens' button tooltip
- ▶ 0:34 Pick blue-1, then blue-3

Low for you
Tokens
Beginner

26 Jun 2026
0:25
Short

Copy and paste tokens between Paper files

How to move a file's tokens to another file while libraries are in progress: Paper logo > Theme > Copy theme in the source file, Cmd+Shift+N for a new file, then Paper logo > Theme > Paste theme; the tokens are available at once. Clear theme is visible in the submenu but not demonstrated.

For you. Your files share no library, so this is how one product's theme reaches its other files (from one Water file to the other, for example). Copy again after every change.

- ▶ 0:02 Theme submenu: Copy, Paste, Clear theme
- ▶ 0:06 Paper logo menu in the source file
- ▶ 0:10 Copy theme
- ▶ 0:12 New file with Cmd+Shift+N
- ▶ 0:14 Paste theme in the new file
- ▶ 0:16 Tokens available in the new file

Medium for you
Tokens
Beginner

24 Jun 2026
4:29
Video

From CSS variables in your codebase to Paper design tokens with MCP

Harcourt uses Cursor with the Paper MCP to turn Paper's own codebase CSS variables into 96 Paper tokens with one prompt (Get Guide, Get Basic Info, Get Tokens, then batched Create Tokens). She tests a color token through the Fill row's four-circle button, has the agent draw a style guide board with Write Html, inspects typography token chips (size, line height, hover name, dropdown with Detach), and copies the whole set into a new file with Paper logo > Theme > Copy theme / Paste theme. As of 24 June 2026 the MCP was the only way to create tokens, and light-dark() values were reduced to their light value.

For you. Your code already defines the CSS variables this one prompt turns into tokens, and you have made 22 create_tokens calls against 3,092 write_html. Seed tokens first so agent HTML binds through var().

- ▶ 0:02 MCP is the only way to create tokens for now; UI promised
- ▶ 0:18 One prompt in Cursor: tokens from the codebase CSS variables
- ▶ 0:48 Get Guide, Get Basic Info, Get Tokens; Create Tokens awaits approval
- ▶ 0:56 Summary: 96 tokens by category and source file ▶ 1:07 Rectangle test with the Fill row tokens button
- ▶ 1:17 Token chip; swap or detach

High for you
Tokens
Intermediate

22 Jun 2026
10:16
Video

Explore Paper's MCP-generated tokens with the Paper team

Harcourt and Dom have Claude (Opus 4.8, Claude desktop app with the paper-desktop plugin) build a style guide from an existing landing page. Unprompted, the agent creates 60 Tailwind v4-named tokens and binds the board through var(). They tour the color and type token controls and the spacing and container tokens in the W dropdown, then build a four-screen onboarding flow on the tokens. Next they detach and recolor two elements teal and have the agent update and rename the tokens with set_tokens, so every bound element changes; the agent warns that the detached elements now hold literal values. Roadmap as of June: in-app creation, theme classes, modes.

For you. Your exact setup (Claude desktop, Opus, the Paper MCP) running the full token update loop, with the traps that matter for incremental edits: detached literals, percentage line heights against your 8 px rhythm, agents creating tokens unasked.

- ▶ 0:48 Prompt: style guide from the Cadence frame ▶ 1:50 Agent created tokens on its own
- ▶ 2:13 Four-circle token picker; pick accent ▶ 2:45 Prompt: onboarding flow using the existing token system
- ▶ 3:13 Switch color tokens, then detach to hex ▶ 3:49 Text tokens: color, size, line height, letter spacing

High for you
Tokens
Intermediate

18 Jun 2026
4:36
Video

Generate and refine SVG illustrations with Quiver AI inside Paper

Create SVG (⇧⌘J) with QuiverAI Arrow 1.1 Max (vs the default Arrow 1.1): prompt anatomy (subject, style, specific colours, restraint), a live build behind a 'Generating with QuiverAI...' placeholder, and an auto-named 256 x 256 SVG made of 3 polygons and about 20 paths. The cleanup pass uses the SVG tools: hide unwanted polygons, normalise uneven strokes (1.28 and 0.99 set to 1), delete stray paths, trim path sections in vector editing with Command+Z as the safety net. About five minutes of edits were skipped.

For you. Quiver is your most-used generator (187 Arrow and Telos runs). The cleanup pass (inventory, one stroke width, hide then delete) turns into agent recipes, and the prompt anatomy works with your palette swapped in.

- ▶ 0:11 Create SVG, ⇧⌘J ▶ 0:24 Arrow 1.1 vs Arrow 1.1 Max ▶ 0:37 Pasting the prompt ▶ 0:56 Prompt anatomy
- ▶ 1:31 Create SVG, loading animation ▶ 1:58 Live build

High for you
Vectors
Intermediate

17 Jun 2026
1:54
Video

How to organize your Paper files with folders and nested folders

In the team dashboard's Files view, + New folder creates a folder (name prefilled Untitled project). Files move via right-click > Move to, with the current location greyed out. Folders nest to any depth (Marketing > Social Media > Instagram, TikTok), move the same way with a confirmation toast, and are renamed via right-click > Rename or by double-clicking the current folder name in the breadcrumb.

For you. Your 14 files sit loose in one folder, including two backup copies an agent could open by mistake. A folder per product is a ten-minute win, done in the dashboard: the MCP cannot move files.

- ▶ 0:08 Files > New folder ▶ 0:24 Right-click a file > Move to ▶ 0:39 Nested folder inside Marketing ▶ 1:03 Move a folder
- ▶ 1:20 Breadcrumb navigation and rename

Medium for you
Canvas
Beginner

15 Jun 2026
3:46
Video

8 Paper tips and tricks every designer should know

Eight canvas tips with on-screen key overlays: ⌘. hides the UI and ⇧. minimizes it into floating pills; double-clicking a layer icon zooms and centers it; ⇧F makes a freeform frame; ⇧A adds flex to a frame (Add flex ⇧A) or wraps loose layers (Wrap in flex ⇧A); ⌘-drag crops an image; double-clicking an edge or clicking the size chip in Fill > Image restores the original size; right-click Extract colors builds a Magic Swatches frame of rectangles named by hex; I opens the eyedropper. Also shows the full right-click menu and the color picker (sRGB / Display P3, LCH, HSL, RGB).

For you. Faster review of agent output (the ⇧. and ⌘. views, jumping to nodes on big boards) and menu actions worth handing to agents: Lock, Copy and Paste styles, Paste to replace, Remove background.

- ▶ 0:12 ⌘. hides the UI
- ▶ 0:21 ⇧. minimizes the UI
- ▶ 0:27 Double-click a layer icon to zoom and center
- ▶ 0:42 ⇧F freeform frame
- ▶ 0:57 ⇧A adds flex or wraps in flex
- ▶ 1:37 Hold ⌘ to crop an image

Medium for you
Canvas
Beginner

10 Jun 2026
1:45
Video

We updated hover selection in Paper based on your feedback

Paper's default hover selection now targets the top-level parent under the cursor, like Figma; double-click goes deeper and ⌘ deep-selects. The old deep selection (hover picks the closest frame) is re-enabled with Use deep selection in the menu under the file name (narration calls it Project Settings), which also holds Back to dashboard ⇧⌘D, Show pixel grid ⌘', Snap to pixel ⇧⌘', Layout guides ⇧G, Multiplayer cursors ⇧M, Open links in desktop app and Nudge settings. ⌘ + right-click lists every layer under the cursor.

For you. You rarely select by hand (get_selection was called once), but ⌘ + right-click reaches any deep layer, and the same menu holds Nudge settings and Layout guides for your 8 px grids.

- ▶ 0:13 New default: hover selects the top-level parent
- ▶ 0:28 Deep selection still available
- ▶ 0:36 Use deep selection in the file-name menu
- ▶ 0:39 Deep selection picks the closest frame
- ▶ 0:57 ⌘ + right-click lists all layers under the cursor

Low for you
Canvas
Beginner

10 Jun 2026
4:22
Video

Claude's Fable 5 vs Opus 4.8: which model designs better in Paper?

Two Claude desktop Code sessions (Fable 5 and Opus 4.8, both High effort) get the same prompt for one expense-entry screen, each titling its artboard with the model name; both write into one Paper file simultaneously. Opus 4.8 finished clearly faster despite starting later; the presenter calls Fable 5 slower and significantly more expensive, possibly better at long independent work. Both loaded the guide, checked fonts (Archivo; Inter Tight), built in chunks, reviewed and released nodes; Fable 5 explained more, Opus 4.8 ended by proposing a variant. June model menu: Fable 5, Opus 4.8, Sonnet 4.6, Haiku 4.5, Opus 4.7/4.6 Legacy, fast-mode toggle. Comments report MCP screenshot timeouts on large artboards; Paper team says naming the frame limits the capture.

For you. Supports running Opus only, on speed and cost. Gives you an A/B harness for panels (compare prompts or effort levels) and flags screenshot timeouts on very large boards.

- ▶ 0:17 Two sessions; model picker
- ▶ 0:24 Same prompt: one expense-entry screen
- ▶ 0:48 Second session on Opus 4.8
- ▶ 1:06 Fable 5 loads guide, checks font, creates artboard
- ▶ 1:22 Both agents designing side by side
- ▶ 2:30 Both finished

Medium for you
Agents
Intermediate

9 Jun 2026
1:27
Short

If your font supports variable weights, your design tool should too!

Variable font controls: with text selected, the Text section's more button (sliders icon) opens a popover with Formatting, Features and Variable font tabs. The Variable font tab sets any weight (100 to 800 shown, including 364 and 560) through a slider and number field, plus optical size with 'Automatic optical sizing'. The presenter explains the benefits: exact weights, smooth weight animation, and one font file instead of one per weight.

For you. Inter, Inter Tight, Geist and Instrument Sans are variable, so KPI numerals and labels can take exact weights. Check how custom weights behave with font-weight tokens and MCP writes.

- ▶ 0:04 User request for custom weights
- ▶ 0:12 More button in the Text section
- ▶ 0:14 Variable font tab
- ▶ 0:28 Weight 400 (Regular)
- ▶ 0:30 Weight 100 (thin)
- ▶ 0:32 Weight 800 (extra bold)

Medium for you
Tokens
Beginner

5 Jun 2026
0:48
Short

SVG editing just launched in Paper

48-second recap of the SVG editing launch: select an SVG, double-click to select one path and move/resize/rotate it alone, double-click again to drag its nodes; a node dragged past the container is clipped at the container edge unless the container is enlarged; recolour the fill, add a stroke (1, Above fill), set its colour and width.

For you. The SVG editing tutorial condensed: a reminder of the double-click ladder and of clipping when an SVG looks cut off.

- ▶ 0:02 SVG editing launched, pen tool included
- ▶ 0:10 Double-click selects a path
- ▶ 0:12 Path resized, moved, rotated alone
- ▶ 0:16 Double-click again for nodes
- ▶ 0:24 Node past the container is clipped
- ▶ 0:32 Fill colour changed

Low for you
Vectors
Beginner

5 Jun 2026
9:15
Video

Using Paper and Resend to turn a design into a ready-to-send email template

Harcourt (Paper) and Chris (Resend) go from a Snapshot capture of a password-reset email to a published Resend template. Claude Code (Sonnet 4.6) duplicates and reskins the email from a style guide on the canvas, self-reviews, and later re-reads Harcourt's manual edits through computed styles. Chris pastes the Paper frame link into his own Claude Code (Opus 4.8), which loads Paper's guide and Resend's skill, builds table-based HTML with {{{RESET_URL}}}, creates a Draft template, and he tests it with JSON variables before publishing.

For you. Handing a frame to another agent by link, here for email HTML. The same pattern hands a board to any build session, and the warning about hotlinked Paper assets applies to your ports.

- ▶ 0:00 Intro: Harcourt and Chris outline the workflow
- ▶ 0:49 Snapshot capture of the email template
- ▶ 1:07 Style guide on canvas and reskin prompt
- ▶ 2:20 Agent reads style guide and structure
- ▶ 2:32 Duplicate reskinned: colors, logo, fonts, copy
- ▶ 3:19 Self-review checklist and change summary

Medium for you
Snapshot
Intermediate

3 Jun 2026
9:17
Video

SVG editing in Paper: pen tool, path editing, and what's coming next

First release of SVG editing (beta). An SVG is a clipping container (default 256 x 256, Clip content ticked) holding Path layers; double-click a path for vector editing (Escape to leave); per-path Fill and Stroke with an Above fill / Below fill position; pen tool (P) with Shift for 45 degrees, Command to pull curve handles, Option to break handle mirroring, click an edge to add a point; drawing outside a container creates a new SVG (Cmd+C/Cmd+V to merge); resizing the container is the crop workaround. Ends with a roadmap board (booleans, improved snapping, path guides, convert shapes, crop/resize near-term) and prototype clips of mirroring, a radial repeater, vector networks and anchoring.

For you. Fix agent-made vector figures by hand (chart figures, Poseidon route arcs, dashboard icons) instead of regenerating them, and understand the 256 × 256 clipping that can cut SVGs in your cards.

- ▶ 0:36 Vector editing roadmap board
- ▶ 1:31 SVG container with Path children
- ▶ 2:04 Double-click a path for vector editing mode
- ▶ 2:25 Per-path Fill and Stroke
- ▶ 2:40 Above fill vs Below fill
- ▶ 3:05 Pen tool (P), half-pixel snapping, Shift 45°

High for you
Vectors
Beginner

3 Jun 2026
4:01
Video

Copy designs from Figma and paste them into Paper

Figma copy/paste (beta): Cmd+C in Figma, Cmd+V in Paper; a toast fetches images and a one-time 'Images from Figma' dialog connects the Figma account; frames arrive 1:1 with their Figma names. A Claude Code agent on Sonnet 4.6 duplicates a frame named in the prompt and translates it to Spanish through the Paper MCP (guide, find, duplicate, set text content, screenshot, finish), revealing new line breaks. Right-click > Copy as offers PNG ⌘⌘C, Tailwind \T, React CSS \R. Components arrive detached; tokens and components were not in Paper yet.

For you. You do not work from Figma, but the agent pattern (target a frame by name, duplicate, change text only, screenshot) gives a cheap French-to-English wrap test for the budget tool, and Copy as PNG keeps files out of Downloads.

- ▶ 0:29 Figma frames to copy
- ▶ 0:42 Cmd+V in Paper
- ▶ 0:49 Figma authorization for images
- ▶ 0:59 One-to-one result
- ▶ 1:20 Agent task: duplicate and translate
- ▶ 1:48 Prompt in Claude, Sonnet 4.6

Medium for you
Vectors
Intermediate

2 Jun 2026
2:49
Video

How to create high quality, searchable PDFs in Paper

Paper prints PDFs through the browser because it renders with HTML/CSS, keeping real searchable text and small files. Select a frame, set Export to PDF, optionally adjust Settings (File suffix, Color space sRGB, Quality High/Medium/Low, Image resampling Detailed/Basic) and click Export ⇧⌘E; files land in Downloads named after the frame. With several frames, Export N elements writes separate PDFs and Export combined PDF writes one file (Combined.pdf, 554 KB for four 1980 × 1080 slides) ordered by canvas position. Preview search finds text, and Claude (Opus 4.8 in the model menu) summarizes the PDF correctly.

For you. Native PDF export failed on your 1400 × 990 report boards and you have made 3 combined PDFs. This gives the path to retest, the page-order rule, and the reminder that exports land in Downloads.

- ▶ 0:03 Why Paper PDFs keep searchable text
- ▶ 0:20 Export section, PDF format
- ▶ 0:36 PDF settings popover
- ▶ 0:42 Export to Downloads
- ▶ 1:04 Export N elements vs Export combined PDF
- ▶ 1:22 Search the PDF in Preview

High for you
Canvas
Beginner

28 May 2026
4:30
Video

Remix Resend's action button with Paper Snapshot and Paper Shaders

Harcourt captures Resend's pill button with Paper Snapshot (structure intact: outer frame, hover rectangle, content frame), duplicates it, and builds an animated border with Pulsing Border: black page background, shader fill removed, colors from the original via the Foreground eyedropper, Scale and Roundness 100%, Radius Full. She nests the shader in the button frame with Absolute position, Fill W/H, X/Y 0, and moves it behind the opaque content frame for hard inner and outer edges.

For you. The exact recipe for a pulsing Ballottage call to action, built on a duplicate beside the original. The absolute-positioned shader is what replace mode breaks, so check it after agent edits.

- ▶ 0:10 Paper Snapshot: hover, arrow keys, Return to capture
- ▶ 0:20 Cmd+V into Paper
- ▶ 0:28 Captured structure; hide the hover rectangle
- ▶ 0:55 Duplicate for before and after
- ▶ 1:00 Add the Pulsing Border shader
- ▶ 1:14 Page background black; remove shader background

High for you
Shaders
Intermediate

26 May 2026
3:37
Video

Generate images in Paper using the xAI model

Create image (⇧⌘I): model menu as of May 2026 (Variety pack default = OpenAI Image 2, Grok, Imagen 4, Seedream 4.5; plus Flux 2 Pro, Recraft V3, Ideogram v3, Nano Banana Pro), aspect ratios (1:1, 3:2, 4:3, 16:9, 2:1, 2:3, 3:4, 9:16, 1:2). Grok returns a 4:3 photoreal image in about two seconds, placed at 592 x 444 from a 1184x888 source. Selecting the image makes it context; one change per prompt (sunset, terracotta pot, healthier plant) keeps the scene consistent, with one artifact; versions are auto-named and placed beside the original. A banner shows a weekly rate limit with Upgrade.

For you. The select-as-context, one-change-per-prompt loop for figurine and prop iterations, with originals kept beside variants. No 4:5 ratio for social posts, and the model list shown is older than the ones you use.

- ▶ 0:02 Create image, ⇧⌘I
- ▶ 0:14 Variety pack default
- ▶ 0:17 Model list, Grok chosen
- ▶ 0:30 Aspect ratio 4:3
- ▶ 0:41 First prompt
- ▶ 0:57 Result in about two seconds

Medium for you
Vectors
Beginner

11 May 2026
1:39
Video

Agent set up: How to connect Cursor to Paper using the Paper MCP server

Two ways to connect Cursor: Cursor Settings (Command+Shift+J) > plugin marketplace > Paper > Get, or paste `/add-plugin paper-desktop` from the Paper docs into a New Agent chat and choose Add for Myself > Add Plugin. Cursor's agent then lists what the paper-desktop plugin bundles: the `paper` MCP server at http://127.0.0.1:29979/mcp, the skills code-to-design and design-to-code, and an auto-applied rule ensure-paper-started. The one hard requirement: Paper Desktop running with a design file open. Docs add: check Cursor Settings > Tools & MCP, reload or toggle the server if the agent can't see it.

For you. You use Claude Code, not Cursor. The useful part is the plugin's anatomy (two skills and the ensure-paper-started rule in your v0.1.0) and the open-Paper-first requirement behind "not connected".

- ▶ 0:00 Intro
- ▶ 0:12 Paper docs: Connect to Cursor, /add-plugin paper-desktop
- ▶ 0:24 Option 1: Cursor Settings (Command+Shift+J), plugin marketplace
- ▶ 0:30 Marketplace search finds Paper, click Get
- ▶ 0:42 Option 2: paste the command into a New Agent chat
- ▶ 0:48 Add for Myself vs Add to Project, Add Plugin

Low for you
Agents
Beginner

Agent set up: How to connect Claude Code to Paper using the Paper MCP server

From zero: install Claude Code (curl installer), run ``claude``, paste ``claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user`` (stored in `~/claude.json`). The first prompt fails because tools added mid-session need a Claude Code restart (Claude suggests ``claude mcp list``). After restart, 'In Paper, design a basic social media home page' produces a design brief (hex palette, Inter weights, 24/16/8 spacing, direction), a 'Social Home' artboard built in chunks under a dotted working indicator, a screenshot review checklist (spacing, typography, contrast, alignment, clipping, repetition) and a summary. UI hidden with `⌘\`; manual edits afterwards (sizing badge 'Fill 191 × Fit 38'). Requires Paper Desktop with a design file open, not Paper web or the dashboard.

For you. Explains the restart rule behind some "not connected" errors, shows the default agent loop and review checklist you can extend with your rules, and shows an agent inventing a terracotta palette when given no brief.

- ▶ 0:24 Install Claude Code with the one-line installer
- ▶ 0:43 Start claude, trust folder, log in
- ▶ 1:05 Paste claude mcp add paper command from Paper docs
- ▶ 1:36 Needs Paper Desktop with a design file open
- ▶ 1:52 First prompt fails: restart needed to load tools
- ▶ 2:22 Say 'in Paper'; agent writes a design brief

Medium for you

Agents

Beginner

Use an AI agent to explore different design directions in Paper

In the Claude desktop app (Code tab, Sonnet 4.6 · Medium), a pre-flight 'Are we connected to the Paper MCP?' returns file, page, artboard 'Social Home' (1440×900), 189 nodes, font Inter. Prompt for two iterations with different colors and layouts: the agent reads the frame structure, proposes named directions 'Ink' (dark, icon-only nav) and 'Slate' (cool, top nav, DM Sans), builds each as a new artboard beside the untouched original, screenshots and critiques each. One variant lost the add button. A combine prompt (Ink's nav + Slate's palette + dark purple accent) yields a planned 'Violet' version (#5B21B6 accent, #1A0F2E rail) with a checkpoint review and `finish_working_on_nodes`; final colour fix done by hand.

For you. Variants beside an untouched original, then combination by name: the mechanics your design panels need, plus the pre-flight check that catches a dead connection.

- ▶ 0:24 Pre-flight: are we connected to the Paper MCP?
- ▶ 0:52 Prompt: two iterations, different colors and layouts
- ▶ 1:28 Agent reads structure, proposes Ink and Slate
- ▶ 2:26 UI hidden with Command+Backslash
- ▶ 2:46 Agent screenshots each variant to check it
- ▶ 3:36 Review: Slate lost the add button

High for you

Agents

Intermediate

Recreate Cloudflare's loading shader from scratch with Paper Shaders

Harcourt rebuilds Cloudflare's teased loading animation: screenshot pasted as reference, Warp added with S and made square, Cauldron Pot chosen as the nearest preset, colors matched first with the color picker's eyedropper (extra blue deleted), then Speed, Scale, Softness and Swirl iterations adjusted. The method is nearest preset, then colors, then motion.

For you. A repeatable, non-random way to build a shader loader for Altitude, Poseidon or Water in cobalt and white, typing exact token values into the picker.

- ▶ 0:02 Goal: Cloudflare's loading shader
- ▶ 0:16 Screenshot of the teaser pasted as reference
- ▶ 0:29 Add a shader with S or the button; Warp
- ▶ 0:42 Make the shader square
- ▶ 0:45 Presets: Default, Live Ink, Cauldron Pot
- ▶ 1:03 Eyedropper the darkest orange and lightest cream; delete the blue

Medium for you

Shaders

Beginner

Design using your live site as a starting point with Paper Snapshot

Harcourt installs the Paper Snapshot Chrome extension, captures the 'Go beyond editing' section of resend.com with a hover-and-highlight selector (click or Return to capture, arrow keys to fine-tune, Esc to cancel) and pastes it into Paper with `⌘V`. The capture lands as editable layers (a 1280 × 900 frame with text layers, a flex row, per-side borders with alpha, radii, Selection colors), which she rearranges with an arrow key and edits in place. Shown on the May 2026 'Paper Alpha' build.

For you. Capture the live Altitude, Poseidon or Water page as editable layers and redesign from production rather than from an older board: improve, don't regress, applied literally.

- ▶ 0:00 Intro: Harcourt Allen, Product Education
- ▶ 0:15 Why the paste is editable: shared HTML/CSS model
- ▶ 0:34 Chrome Web Store, Add to Chrome
- ▶ 0:41 Pin the extension (quick start guide)
- ▶ 0:53 Click the extension; hover shows a blue bounding box
- ▶ 1:08 Fine-tune with arrow keys; capture with click or Return

Medium for you

Snapshot

Beginner

2 May 2026
3:30
Video

How to use an AI agent to iterate on a design you captured with Paper Snapshot

On the Snapshot capture, Harcourt prompts Claude Code in the Claude desktop app's Code tab (Sonnet 4.6, Auto accept edits) to add a third column using copy parked in a side frame. The agent reads the canvas (Get Screenshot, Get Tree Summary, JSX), changes a repeat(2, 1fr) grid to three columns, duplicates a column, updates its text and icon, and posts a Checkpoint review. She approves 'Always allow for project (local)' and finishes by hand, double-clicking media frames' edges to set fit height.

For you. The agent loop on a capture (read the JSX, change the structure, duplicate, review) plus the resync step after your manual edits, the habit that stops agents overwriting your fixes.

- ▶ 0:02 Intro: iterate on the capture with Claude Code
- ▶ 0:13 Context matters: say 'in Paper'
- ▶ 0:26 Prompt typed in the Code tab
- ▶ 0:47 Agent finds file, page and frame via MCP
- ▶ 1:06 Agent reads the canvas (screenshot, tree summary)
- ▶ 1:32 Permission prompt; always allow

High for you
Snapshot
Beginner

5 Mar 2026
3:29
Video

Design with real content: How to pull Notion data into Paper with Claude Code

Earliest agent video (Paper Alpha). Stephen adds the Notion MCP to Claude Code next to the Paper MCP, checks both with /mcp (Notion: connected, authenticated, https://mcp.notion.com/mcp, 12 tools; actions View tools, Re-authenticate, Clear authentication, Reconnect, Disable), and asks Claude to design a Notion 'Core Values' page in Paper. The agent runs notion-search and Paper get_basic_info in parallel, notion-fetch, get_font_family_info for four families, writes a brief, creates an artboard and builds it with semantically named sections (Header, Core Values Section, Tactics Section, Reading Section, Footer). Works with any MCP source, web search, and data viz.

For you. The same pattern fills your dashboards with real data instead of invented values, and shows the opening to require: parallel reads, a font check, semantic names.

- ▶ 0:04 Why real data; any MCP source or web search
- ▶ 0:41 Add the Notion MCP from Notion's docs
- ▶ 0:54 /mcp shows both servers
- ▶ 1:11 Prompt: grab the Notion page, design it in Paper
- ▶ 1:24 notion-search and get_basic_info in parallel
- ▶ 1:54 notion-fetch, then font availability check

Medium for you
Agents
Intermediate

5 Mar 2026
7:19
Video

How to sync designs with your codebase using Paper MCP, Claude Code and Cursor

Stephen Haney connects a fresh Paper file to Claude Code (one-line claude mcp add over HTTP to 127.0.0.1:29979/mcp, verified with /mcp) and has Claude lay out colors.css as a Color System board with variables, oklch values and usage notes, self-reviewing by screenshot. He then installs the server in Cursor via Add to Cursor, retunes five blues by editing hues in the Selection colors list, and has Cursor write only the selected colours back to colors.css, with an accept/reject diff. As of March 2026.

For you. A color round trip between code and canvas that fits the Water blues and the Altitude palette. Its note that the server starts only when a file is open explains "not connected".

- ▶ 0:39 Per-agent setup guides at paper.design/docs/mcp
- ▶ 0:54 claude mcp add run in a normal terminal
- ▶ 1:04 /mcp shows Paper connected
- ▶ 1:11 Prompt: lay out colors.css on the canvas
- ▶ 1:27 Tool permission prompts, don't ask again
- ▶ 2:00 get_font_family_info checks fonts

High for you
Code
Beginner

5 Mar 2026
4:48
Video

How to use Paper Shaders to elevate your designs

Stephen Haney tours Paper Shaders (March 2026): Warp presets as quick starts, Speed 0% for still textures, and right-click Copy as React CSS, which outputs an exact @paper-design/shaders-react component with a link back to the Paper node. He feeds generated images into Fluted Glass with the image eyedropper, chains Halftone CMYK (Vintage) on top of the glass output, and closes with the AI Vectorize command on a flat image.

For you. Copy as React CSS carries a shader straight into React code, exact props and a link to the Paper node included. Speed 0 textures suit social posts and report covers; presets lean warm, so recolor to cobalt and neutrals.

- ▶ 0:22 Warp shader on the canvas; presets as quick starts
- ▶ 0:50 Presets rewrite the parameter values
- ▶ 1:00 Speed 0% turns an animated shader into a still texture
- ▶ 1:12 Right-click, Copy as React CSS, paste exact component code
- ▶ 1:32 Image filters: Fluted Glass
- ▶ 1:44 Built-in image generation with four models (Variety pack)

High for you
Shaders
Beginner

Researched and written with Claude on 2 October 2026, from the 52 videos on youtube.com/@paperdesign, Paper's documentation and MCP guides, and my own Claude session history. The personal sections are addressed to me. Set on the twelve-column programme of [Learn dashboard design with me](#). Web edition: enzosimier.com/projects/paper-guide.